

STUDY GUIDE · FOR BUILDERS OF PRINT SOFTWARE

Computer Graphics for: Print, PDF & Prepress

A Beginner-to-Advanced Study Guide — Color Science, Prepress, PDF
Internals & Typography

Prepared 2026-06-21 · Read the sections in order

Table of Contents

How to Use This Guide

CORE CONCEPTS

1. How Color & Light Work (The Foundation)
2. Color Models: RGB vs CMYK vs Lab
3. ICC Color Profiles & Color Management
4. Rendering Intents
5. Gamut & Out-of-Gamut Handling
6. Spot Colors, Pantone, Overprint & Knockout
7. Total Ink Coverage (TAC) & Dot Gain
8. Resolution: DPI, PPI & Why Print Needs 300
9. Halftoning & Screening
10. Bleed, Trim, Safe Zone & Marks
11. Preflight: Validating a Print-Ready File
12. Trapping
13. Raster vs Vector & Rasterization
14. Image Resampling, Color Depth & Channels
15. Alpha Compositing, Blend Modes & Transparency Flattening
16. Inside a PDF: Document Model & Content Streams
17. Fonts: Embedding, Subsetting & Outlines
18. PDF/X Standards: The Print-Ready Format

19. Typography & Text Rendering

REFERENCE

Glossary of Terms

Frequently Asked Questions

Revision Cheat Sheet

How to Use This Guide

Welcome. If you build, maintain, or work near a print-on-demand product — and especially if you work on **Print-Flow-360**, with its Fabric.js design studio and its Node `pdf-service` — this guide is for you. You don't need a design degree, a prepress background, or any prior knowledge of color science. You need curiosity and a willingness to learn one idea at a time. By the end, you'll understand *why* a customer's bright-orange logo can print muddy brown, *why* text looks crisp on screen but blurry on paper, and *what* a "print-ready file" actually is under the hood.

This guide assumes you are smart but new to graphics and print. Every technical term is defined in plain English the first time it appears. We start from absolute basics — how light and ink even work — and build, section by section, to genuinely advanced topics like PDF content streams and transparency flattening. Nothing is skipped, and nothing is hand-waved.

Who this is for

You might be a backend engineer wiring up the `pdf-service` (which uses `sharp` for images and `PDFKit` for PDFs on port 4000), a frontend developer extending the designer studio, or a founder who needs to understand why customers complain their printed colors look "wrong." Print-Flow-360 today has documented gaps — there is **no RGB → CMYK conversion**, **no preflight** (automated checking of a file before it goes to print), and a weak order-to-production handoff. This guide gives you the vocabulary and the mental models to reason about those gaps and decide how to close them.

The central idea — Screens emit *light* and mix colors as **RGB** (red, green, blue). Print lays down *ink* and mixes colors as **CMYK** (cyan, magenta, yellow, black). These two worlds can't reproduce all the same colors. Almost every print problem you'll ever debug — dull colors, surprise color shifts, "why doesn't it match my screen?" — comes from that single gap. Keep this in mind through every section.

The journey: how the sections build up

The guide moves in four connected arcs, each laying groundwork for the next:

1. **Color science (Sections 1–9)** — the foundation. We start with light and ink, then color models (RGB, CMYK, Lab), then how computers manage color faithfully across devices: ICC profiles, rendering intents, gamut limits, spot colors and Pantone, ink limits and dot gain, and finally resolution (why print needs ~300 DPI). This arc explains the screen-vs-ink gap in full.
2. **Prepress geometry & validation (Sections 10–12)** — preparing a file to physically print: bleed, trim, and safe zones; preflight checks; and trapping. This is where color knowledge meets the cutting blade.
3. **Raster, vector & PDF internals (Sections 13–18)** — what files are made of: pixels vs. math-based shapes, image resampling and channels, transparency and blend modes, then the anatomy of a PDF itself and the PDF/X print-ready standard.
4. **Typography (Section 19)** — how text is rendered and embedded, the detail that makes work look professional or amateur.

Each section is self-contained enough to revisit later, but they're written to be read **in order** the first time. Section 6 (spot colors) leans on Section 2 (color models); Section 18 (PDF/X) leans on nearly everything before it. Resist the urge to jump ahead until you've read it once front to back.

How to read the callout boxes

Throughout, you'll see colored boxes that flag the most important moments. They mean exactly this:

Key takeaway

The one idea from this spot you must not forget. Each section also ends with a summary box.

Best practice

A concrete, do-this recommendation — often tied directly to the Print-Flow-360 stack.

Common mistake

A pitfall people hit repeatedly. Read these closely; they save real debugging hours.

Analogy

An everyday comparison to make an abstract idea click.

Worked example

A real-world or numeric walk-through showing the concept in action.

You'll also see ASCII diagrams in monospaced boxes — simple pictures of curves, layouts, and before/after states. They reward a slow look.

Best Practice — As you read, keep mapping each concept back to a real Print-Flow-360 surface: the designer canvas (where customers build artwork in RGB), the `pdf-service` (where that artwork becomes a deliverable file), and the storefront's brand-color theming. Asking "where would this live in our stack?" turns passive reading into engineering insight.

Analogy — Think of this guide as learning to cook for a restaurant, not just for yourself. First you learn ingredients (color and light), then food safety and plating standards (prepress), then how the kitchen and recipes actually work (file internals), and finally presentation (typography). Skip a stage and the dish — your print job — comes out wrong in ways you can't explain.

Key takeaways —

- This guide is for anyone building or supporting a print SaaS like Print-Flow-360 — no prior graphics knowledge assumed; every term is defined on first use.
- It moves beginner → advanced in four arcs: **color science** → **prepress** → **raster/vector & PDF internals** → **typography**. Read it in order the first time.
- The one theme tying it all together: **screens emit light (RGB)**, **print uses ink (CMYK)**, and most print problems live in that gap.
- Callout boxes are signposts — Key takeaway, Best practice, Common mistake, Analogy, Worked example — and every section ends with a summary.
- We'll repeatedly connect ideas to the real stack: the Fabric.js designer, the `sharp` / `PDFKit` `pdf-service`, brand theming, and the known gaps (no RGB → CMYK, no preflight).

How Color & Light Work (The Foundation)

Before we touch a single color model, file format, or print setting, we need to agree on one surprising idea: **color is not a property of objects**. A strawberry is not "red" the way it is "round." Color is something your brain *builds* out of light. Once you really understand that, everything else in this guide — why your screen and your prints never quite match, why neon green turns to mud on paper, why color management even exists — stops being mysterious and starts being obvious.

1. Light is the starting point

Light

A form of energy that travels in waves. The eye can only detect a tiny slice of all the light in the universe — that slice is called **visible light**.

Wavelength

The distance between the peaks of a light wave, measured in **nanometers (nm)**, a billionth of a meter. Wavelength is what we perceive as different colors.

Visible light runs roughly from **380 nm to 700 nm** (some people see a bit further, up to ~780 nm — the edges are fuzzy and vary by person and brightness). Short wavelengths carry more energy and look violet/blue; long wavelengths carry less energy and look red. The familiar rainbow order, short to long:

380nm		700nm
violet	blue cyan green yellow orange	red
short wavelength	----->	long wavelength
higher energy	----->	lower energy

"White light," like sunlight or a typical light bulb, is simply *all* these wavelengths mixed together. An object looks red because its surface **reflects** the long (~700 nm) wavelengths back to your eye and **absorbs** the rest. Change the light — say, put that strawberry under a pure blue lamp — and it looks nearly black, because there are no red wavelengths for it to reflect. The object never had a "color" of its own.

Analogy — Think of an object's surface as a bouncer at a club for wavelengths. A "red" surface lets red wavelengths bounce back out to your eye and turns all the others away (absorbs them). No red light in the room? Then nobody gets bounced back, and the surface looks dark.

2. How the eye actually sees color

The back of your eye (the retina) has two kinds of light sensors:

Rods

Work in dim light (night vision). Very sensitive but see **no color** — this is why a moonlit room looks gray.

Cones

Work in bright light and give us **color vision**. There are only **three types**.

The three cone types are named by which wavelengths they respond to most: **S** (short, peaks ~440 nm, our "blue" cone), **M** (medium, peaks ~545 nm, "green"), and **L** (long, peaks ~565 nm, "red"). The L and M cones overlap heavily. Your brain doesn't read an exact wavelength — it reads the **ratio** of how strongly S, M, and L are firing, and interprets that ratio as a color. Yellow, for example, isn't a special signal: it's just L firing slightly more than M.

Key Takeaway — The eye is a **3-channel sensor**. This single fact is the reason all of color technology works. Because you only have three color sensors, *any* color you can see can be matched by mixing just **three primary colors**. That's why screens use three (Red, Green, Blue) and print uses three inks (Cyan, Magenta, Yellow). We're not reproducing the real light — we're just fooling three cones.

Metamerism

When two physically different mixes of light produce the *same* S/M/L ratio, so they look identical even though they aren't the same light. This is both a gift (we can fake colors with 3 primaries) and a curse (two colors can match under one light and clash under another).

3. The two opposite ways to make color

Here is the heart of the whole guide. There are exactly two strategies for producing color, and a **screen** and a **printed page** use **opposite ones**.

Additive color — light (screens)

You start in **darkness** (black) and *add* colored light. The primaries are **Red, Green, Blue (RGB)**. The more light you add, the brighter you get, heading toward white. This is how everything that **emits** light works: monitors, phones, TVs, projectors, and the sensors in cameras and scanners.

- Red + Green = Yellow
- Green + Blue = Cyan
- Red + Blue = Magenta
- Red + Green + Blue = **White**

Subtractive color — ink/pigment (print)

You start with **white paper** (which reflects all wavelengths) and *add inks that subtract* — each ink absorbs some wavelengths and reflects the rest. The primaries are **Cyan, Magenta, Yellow (CMY)**. The more ink you add, the darker you get, heading toward black.

- Cyan absorbs red; Magenta absorbs green; Yellow absorbs blue.

- Cyan + Magenta = Blue
- Cyan + Yellow = Green
- Magenta + Yellow = Red
- Cyan + Magenta + Yellow = a muddy dark brown — *not* a clean black

Notice the beautiful symmetry: the print primaries (CMY) are exactly the **secondary colors of light (RGB)**, and vice versa. The two systems are mathematical inverses of each other.

Worked example — why the K in CMYK? — Because mixing all three inks gives muddy brown, not deep black, print adds a fourth ink: **K = Key (black)**. Three reasons: (1) CMY can't make a true rich black; (2) aligning three separate ink layers perfectly ("registration") is imprecise, so text would look fuzzy and tinted; (3) one black ink is far cheaper than soaking the paper in three inks. So nearly all color printing is **CMYK**, four inks.

Analogy — Additive light is like **stage spotlights** on a black stage — point red, green, and blue lamps at the same spot and you get white. Subtractive ink is like **stacking colored gel filters** over a white window — each filter removes more light, and stacking them all gives near-darkness.

ADDITIVE (screen, light) start: BLACK +light => brighter <pre> R / \ Mag Yel \ / B---+---G -> all = WHITE </pre>	SUBTRACTIVE (print, ink) start: WHITE paper +ink => darker <pre> C / \ Blu Grn \ / M----+----Y -> all = mud </pre>
--	--

4. Why a screen and a print can never match exactly

Now the consequence. A screen **emits** its own light, so it can be vivid, bright, and even glowing in a dark room. A print only **reflects** whatever light is already in the room, through a layer of ink, off the paper. A print can *never be brighter than the room it's sitting in*, and never brighter than the paper underneath. So the same photo that glows on your phone will always look a little darker and less saturated on paper. Different starting points (black vs. white), different colorants (light vs. ink), different physics — they **cannot** be made identical.

Key Takeaway — The goal of color work is never "make the print identical to the screen." That's physically impossible. The realistic, professional goal is **close and predictable**: you know in advance how a color will shift, and there are no nasty surprises.

5. Color spaces and gamut — naming and bounding the colors

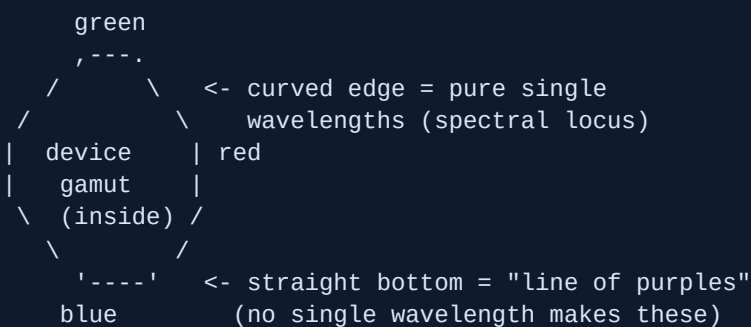
Color space

A defined system — a coordinate map — for describing colors with numbers, so "this exact red" means the same thing on two different machines. Examples: sRGB, Adobe RGB, a specific CMYK press profile.

Gamut

The **range** of colors a particular device or space can actually reproduce — its palette. Every real device's gamut is a *subset* of all the colors a human can see.

The master reference for "all colors humans can see" is the **CIE 1931 XYZ** system, created in 1931. Its famous **chromaticity diagram** is a horseshoe/"tongue" shape:



The curved outer edge is the **spectral locus** — the purest possible single-wavelength colors. The straight bottom edge is the **line of purples**, colors (like magenta) that no single wavelength can produce; they only exist as mixes. Every device's gamut is a smaller shape *inside* this horseshoe. Important caveat: this diagram is **not perceptually uniform** — equal distances on it do *not* mean equal-looking color differences. (Later spaces like **CIELAB**, with its **Delta-E** difference metric, were built to fix that.)

Common RGB color spaces, from smaller to larger gamut: **sRGB** (the web/general default, created by HP & Microsoft in 1996), then **Adobe RGB**, **DCI-P3**, and **Rec.2020**. A standard **CMYK** print gamut is *smaller than sRGB* — and, crucially, a *different shape*. It's not just "less"; some colors a press can print (certain deep cyans) actually fall outside sRGB, while many bright screen colors fall outside CMYK.

6. Out-of-gamut: the colors print simply can't reach

Out of gamut

A color that exists in one space (e.g. on your sRGB screen) but has **no equivalent** in the target space (e.g. CMYK ink). The printer has to substitute the nearest color it *can* make.

Subtractive ink can never out-shine emitted light, so the brightest, most electric screen colors are unprintable on standard CMYK. Classic offenders: neon green (`#39FF14`), electric blue (`#7DF9FF`), hot pink (`#FF69B4`), and vivid oranges. On a press these all shift noticeably duller. The printer decides *how* to substitute using a **rendering intent**:

Rendering intent	What it does	Best for
Relative Colorimetric	Keeps in-gamut colors exact; "clips" each out-of-gamut color to the nearest printable one	Logos, brand colors, designs with few extreme colors
Perceptual	Gently squeezes the <i>whole</i> gamut inward so the <i>relationships</i> between colors are preserved	Photographs with many saturated colors

And remember the paper itself is your literal "white": **glossy/coated stock has a wider gamut** than matte/uncoated, and a warm-white sheet vs. a bright-white sheet shifts every color, because the paper is the starting point everything is built on. In standard CMYK, **white ink doesn't exist** — "white" is just the parts of the paper you leave unprinted.

7. How this maps onto Print-Flow-360

This isn't abstract — it explains real, documented behavior and gaps in this platform:

- **The Fabric.js design studio (`designer/`) is pure RGB.** Browsers, canvas, and `<canvas>` all work in additive sRGB. So every swatch a customer picks — including via the storefront **brand-color theming** — is a glowing screen color with no awareness of whether ink can reproduce it.
- **The platform has no RGB → CMYK conversion and no preflight** (a documented gap). "Preflight" means an automated pre-print check that flags problems — like out-of-gamut neon colors or missing bleed — *before* a job goes to press. Without it, a customer's electric-blue design is accepted as-is and only disappoints when it arrives on paper duller than the screen.
- **The Node PDF service (`pdf-service` , port 4000) uses `sharp` for images and PDFKit for PDFs** — both operate in sRGB and do not perform color-managed CMYK separation. That's exactly where a future gamut-warning or soft-proofing step would belong, since it already owns the heavy image/PDF pipeline.
- **The weak order-to-production spine** is where these physics bite hardest: a beautiful on-screen proof with no color expectations set is a recipe for "this isn't what I designed" support tickets.

Pro Tip (actionable, no invented code) — The cheapest first win is *expectation-setting*, not engineering: when a customer picks a highly saturated brand or design color, plan a plain-language note such as "Bright screen colors print a little softer on paper." That single sentence prevents the most common print complaint and costs nothing in the press.

8. The mistakes beginners make (and how to dodge them)

Common Mistakes —

- **Designing in RGB and auto-converting to CMYK at the last second** → ugly shifts, worst in saturated blues, greens, and oranges. Convert early and *look* at the result.
- **Expecting neon/vivid screen colors to print as seen.** They can't; they're out of gamut.
- **No ICC profiles or soft-proofing.** An *ICC profile* is a small file describing a device's exact gamut; *soft-proofing* means simulating the print on your screen using the press + paper profile. Skip these and screen and print drift apart blindly.
- **Embedding a printer/media profile in a file when the printer expects plain sRGB** → unpredictable double-conversions.
- **Forgetting paper stock changes identical CMYK values.** The same numbers print differently on gloss vs. matte vs. warm-white.
- **Trusting a bright, uncalibrated monitor.** Most screens are brighter than paper, so prints look "too dark" — the screen lied, not the press.
- **Believing screen and print can be made identical.** They can only be made *close and predictable* through color management.

Key takeaways —

- **Color lives in your brain, not in objects** — it's how three cone types (S/M/L) interpret light. That's why three primaries can fake any color.
- **Screens are additive (RGB):** start at black, add light, get brighter toward white — they *emit* light.
- **Print is subtractive (CMYK):** start at white paper, add ink, get darker toward black — it *reflects* light. CMY are the inverses of RGB; K is added for true black, sharp text, and cost.
- **Because the two work in opposite directions, they can never match exactly** — only "close and predictable." This is the theme of the entire guide.
- **A color space names colors with numbers; gamut is the range a device can reproduce.** CMYK's gamut is smaller and a different shape than sRGB, so bright screen colors (neon green, electric blue, hot pink) go "out of gamut" and shift on paper.
- **Paper is your literal "white"** — stock and brightness change every color; there is no white ink in standard CMYK.
- **In Print-Flow-360** the Fabric.js studio, brand theming, and the **sharp** /PDFKit pipeline are all RGB with no CMYK conversion or preflight — so setting customer expectations is the highest-value, lowest-cost first step.

Color Models: RGB vs CMYK vs Lab

Every color you have ever seen on a phone, a billboard, or a printed brochure is described by a *color model* — a system of numbers that tells a device exactly which color to produce. There are three you must understand to do print work well: **RGB** (how screens make color with light), **CMYK** (how presses make color with ink), and **Lab** (a neutral, device-independent way to describe color that acts as a translator between the other two). The single most expensive misunderstanding in print is assuming these three are interchangeable. They are not, and the gap between them is exactly where Print-Flow-360 has a documented production weakness. Let's build it up from scratch.

RGB — making color by adding light

RGB

Red, Green, Blue. A color model that *adds* colored light together. Used by anything that *emits* light: phone screens, monitors, TVs, cameras.

Additive color

You start in total darkness (black = no light) and *add* light to create color. The more light you add, the brighter and closer to white you get.

Channel

One of the primary colors, given its own intensity number.

In RGB each channel is an 8-bit number from 0 to 255. (0, 0, 0) is black (no light at all), (255, 255, 255) is white (all three at full blast), and (255, 0, 0) is pure red. Three channels × 256 levels = about 16.7 million colors (this is "24-bit color"). The standard RGB working space on the web is **sRGB**; wider ones like Adobe RGB and Display P3 exist for richer screens.

Analogy — Think of three theater spotlights — one red, one green, one blue — aimed at the same white wall in a dark room. All off = a black wall. All on full = a white spot. Mix them and you can paint any color. That is RGB: you are *adding* light.

CMYK — making color by subtracting light

CMYK

Cyan, Magenta, Yellow, Key (black). The model used for printing with ink on paper.

Subtractive color

You start with white paper (which reflects all light) and lay down ink that *absorbs* — subtracts — certain wavelengths. What bounces back to your eye is the color. The more ink, the darker the result.

CMYK channels are **percentages from 0 to 100%** — the amount of each ink laid down. This is also called *process color* printing. Cyan ink absorbs red light, magenta absorbs green, yellow absorbs blue. Stack them and you remove more and more light.

Analogy — RGB is spotlights in a dark room. CMYK is the opposite: a bright white sheet of paper, and ink acts like colored sunglasses you stack on top to *block* light. Stack enough and you approach darkness — but you can never make the paper glow brighter than the room's light. That ceiling is why ink can't match a screen.

Why the "K" (black) ink exists

If cyan, magenta, and yellow each remove one third of the spectrum, then 100% of all three should produce black. In theory. In practice real inks are impure, so **C100 M100 Y100** prints as a muddy dark brown, not a true black. It also soaks the paper, dries slowly, and lacks density. So presses add a fourth, dedicated black ink.

Key (K)

The black plate. "Key" refers to the registration/key plate that black text aligns to — it is not "the K from black." Using K instead of "B" also avoids confusion with Blue.

GCR (Gray Component Replacement)

Swapping the gray part of any CMY mix for black ink across the whole tone range — saves ink and keeps neutrals steady.

UCR (Under Color Removal)

The same idea but only in dark/shadow areas.

TAC / Total Ink Coverage

The sum of all four ink percentages at one spot. Presses cap this — coated sheetfed (GRACoL) around 300–340%, digital around 260–280%.

A dedicated K ink gives true neutral black (about 30% denser than a CMY mix), crisp black text without three-plate misalignment, and less total ink. Two practical recipes matter:

- **Plain K100** — fine for small black text, but over a large area it looks washed-out gray.
- **Rich black** — e.g. **C60 M40 Y40 K100** (= 240% TAC) for large dark fills, giving a deep solid black.

Common Mistake — Using **registration black** **C100 M100 Y100 K100** (400% ink) for body content. It exceeds every TAC limit, smears, and never dries. Registration black is only for crop/registration marks.

Lab — the device-independent bridge

Lab (CIELAB / CIE L*a*b*, 1976)

A color model built to match *human vision*, not any device. It describes a color absolutely, no matter what produced it.

Its three values: **L*** = lightness, 0 (black) to 100 (white); **a*** = green (negative) to red (positive); **b*** = blue (negative) to yellow (positive), with a* and b* roughly spanning -128 to +127. Lab's gamut is larger than both RGB and CMYK — it can describe every color a human can see, including ones no screen or press can reproduce. That is exactly why it works as the neutral translator: any device's colors can be mapped *into* Lab and back out to another device.

Gamut

The full range of colors a particular device or model can actually produce.

```
Lab gamut (all visible color)
+-----+
| RGB / sRGB gamut (screen light) |
| +-----+ |
| | CMYK gamut (ink on paper) | |
| | smallest | |
| +-----+ |
| ^ bright greens/blues live here |
| in RGB but NOT in CMYK |
+-----+
```

Why RGB → CMYK shifts and dulls color

The RGB gamut is much wider than the CMYK gamut. When you convert, colors that exist in RGB but fall outside CMYK are *out of gamut* and must be squeezed into the smaller printable set — called **gamut clipping** or **compression**. Those colors lose saturation and look duller, darker, or dustier. The worst losses are bright, saturated **greens**, **blues**, and neon/glowing oranges and reds — precisely the colors that emitted light can hit but reflected ink cannot. Pastels and most skin tones survive the trip well.

Worked Example — vivid blue and green going dull — A common (naive) conversion formula:

normalize $R' = R/255$, then $K = 1 - \max(R', G', B')$; $C = (1 - R' - K)/(1 - K)$; $M = (1 - G' - K)/(1 - K)$; $Y = (1 - B' - K)/(1 - K)$.

- **Pure screen blue** $RGB(0, 0, 255) \rightarrow C100\ M100\ Y0\ K0$. On a real press this prints as a dull *violet/indigo*, not electric blue — that screen blue is outside the CMYK gamut. In Lab terms its very-negative b* shifts toward a darker purple.
- **Pure screen green** $RGB(0, 255, 0) \rightarrow C100\ M0\ Y100\ K0$. Prints a muted *leaf/olive* green, never the glowing on-screen green. This is the single most dramatic dulling case.

The key catch: that 0–100% formula ignores real ink behavior. Professional conversion uses ICC profiles with Lab in the middle and produces *different, usually smaller* numbers — and the *same* RGB yields *different* CMYK on different paper and presses.

Color management: how the bridge actually works

ICC profile

A file describing one device's gamut — how its numbers map to real Lab colors. You convert through a source profile (e.g. sRGB) and a destination profile (e.g. a CMYK press).

PCS (Profile Connection Space)

The neutral hub every profile maps to and from — it is Lab (or XYZ). Reference white is **D50** for printing (most other industries use D65), per ISO 13655.

Rendering intent

The rule for handling out-of-gamut colors during conversion.

Delta E (ΔE)

A number measuring how different two colors look in Lab. About **1** is a just-noticeable difference; used to check a proof against the press.

Real conversion is always **RGB** → **Lab** → **CMYK**. Named CMYK destination standards include **GRACoL2013** (US sheetfed coated), **SWOP** (US web offset, smaller/cheaper), and **FOGRA39/FOGRA51 (PSO Coated v3)** in Europe. The four rendering intents:

Intent	What it does	Best for
Perceptual	Compresses the whole gamut proportionally; shifts everything slightly but keeps relationships	Photos, gradients
Relative Colorimetric	Keeps in-gamut colors exact, clips out-of-gamut to nearest edge; PDF default	Logos, brand color
Saturation	Maximizes vividness over accuracy	Charts, business graphics
Absolute Colorimetric	Simulates paper white too	Proofing / press match

The "CMYK gap" in Print-Flow-360

This is the practical payoff. PF360's **Fabric.js design studio** ([designer/](#)) and the **storefront** — including the brand-color theming — work *entirely in RGB/sRGB*, because they are web tools. A printing press needs CMYK separations with an ICC output profile, an ink-limit, and proper black generation. **Nothing in the current pipeline converts RGB → CMYK or embeds an output profile.** The Node **PDF service** ([pdf-service/](#) , port 4000, using [sharp](#) for images and [PDFKit](#) for PDFs) is where that conversion *belongs*, but it does not yet do it.

What goes wrong if unaddressed —

- A customer's vivid on-screen design prints duller and shifted — no managed conversion.
- Large black areas print muddy or weak — no rich-black / K handling.
- No gamut warning, so customers approve colors that physically cannot be reproduced.
- Files may blow past press TAC limits (smearing, slow drying).
- No way to pick a per-paper / per-press profile.

This is a real production-readiness gap (it sits alongside the missing *preflight* step and the weak order-to-production spine), not a settings toggle.

Pro Tip — Color conversion belongs *server-side*, at proof/print-file generation in the PDF service — using an ICC profile, a chosen rendering intent, an ink-limit and black-generation pass — **never in the browser**. Lab is the engineering bridge that makes it correct and repeatable per paper and press.

Common Mistakes to call out — exporting in RGB then being shocked print looks dull; expecting neon screen greens/blues to print; using K100 for large fills (looks gray) when you need rich black; using 400% registration black for body content; ignoring which CMYK profile/paper applies; assuming one fixed RGB → CMYK formula (it's profile-dependent); treating Lab as a paint space instead of a reference; forgetting D50 vs D65 when matching a proof to a screen.

Key takeaways —

- **RGB** = adds light, 0–255 per channel, for screens; **CMYK** = subtracts light with ink, 0–100% per channel, for presses; **Lab** = device-independent, perceptual, the neutral bridge with the widest gamut.
- K (black) ink exists because C+M+Y can't make a true black — it gives dense neutral black, sharp text, and less ink (GCR/UCR); respect TAC limits and use rich black, not K100 or 400% registration black, for large dark areas.
- RGB's gamut is wider than CMYK's, so conversion clips out-of-gamut colors — bright blues and greens dull the most (screen blue → indigo, screen green → olive).
- Correct conversion is RGB → Lab → CMYK via ICC profiles (GRACoL/SWOP/FOGRA), a rendering intent, and D50 reference white — the same RGB yields different CMYK per paper/press.
- PF360's designer and storefront are RGB-only; nothing converts to CMYK. The fix lives server-side in the PDF service (profile + intent + ink-limit + black generation), and it's a genuine production gap, not a toggle.

Section 3: ICC Color Profiles & Color Management

In the last section you learned that RGB and CMYK are two different ways of describing color, and that a screen and a printer live in different worlds. But here is the uncomfortable truth that section hinted at: the numbers `R=255, G=0, B=0` or `C=80, M=0, Y=90, K=0` do not actually mean a specific color. They are just numbers. Hand the same numbers to two different monitors, or two different printing presses, and you get two different colors. This section is about the system the printing industry built to fix that problem — **color management**, powered by **ICC profiles**.

The core problem: device numbers are ambiguous

Imagine you tell two painters "use color number 47." Painter A reaches for a tube labelled 47 in her set; Painter B reaches for a totally different tube labelled 47 in his set. The number is the same; the paint is not. That is exactly what happens with raw RGB and CMYK values across devices. A color number is meaningless until you know *which device's dictionary* it came from.

ICC profile

A small standardized data file that describes what a specific device's color numbers actually *mean* — i.e., what real-world color you get when that device produces those numbers. "ICC" stands for the International Color Consortium, the industry body that maintains the spec (also published as the ISO standard ISO 15076-1). The files end in `.icc` or `.icm`.

Analogy — An ICC profile is a *translation dictionary* for one device. The device speaks its own private language ("my R=255"), and the profile translates that into a universal, device-independent description of the actual color a human would see. Every device gets its own dictionary.

Profiles come in three roles depending on what the device does:

- **Input profiles** — for cameras and scanners (devices that *capture* color).
- **Display profiles** — for monitors (devices that *show* color).
- **Output profiles** — for printers and presses (devices that *print* color). These are the stars of this section.

The Profile Connection Space — the universal middle language

If every device had its own dictionary but no shared language, you would still be stuck: how do you go from "this camera's numbers" to "this press's numbers"? The answer is a fixed, neutral middle language that every profile translates to and from.

Profile Connection Space (PCS)

A device-independent reference color space that acts as the universal meeting point. Every profile knows how to translate its device's numbers into the PCS, and back out of it. The ICC spec allows

two encodings for the PCS: **CIEXYZ** or **CIELAB (Lab)** — both are math-based descriptions of color built on how the human eye actually sees (the CIE 1931 standard observer), not on any one device.

Analogy — Think of the PCS as English at an international airport. A Japanese speaker and a Portuguese speaker don't each need to learn every other language — they each translate to and from English. The PCS is that pivot language for color. Because everyone agrees on the pivot, a profile made by Adobe and a profile made by some European press vendor can be paired together at runtime and still produce predictable results.

One detail that trips people up: the print PCS is defined under the **D50** viewing condition (a standardized 5000-Kelvin "daylight" white, at 500 lux, measured per ISO 13655). Screens use D65 (a cooler ~6500K white). So the world of editing on screen (sRGB, D65) and the world of print color science (D50) reference different whites — a frequent source of "why does my proof look warmer than my screen?" confusion.

How a conversion actually runs: source, destination, and the CMM

Every real color conversion is a two-step pipe through the PCS, run by a piece of software called the CMM.

CMM (Color Management Module)

The software engine that does the actual color math. It reads the source profile and the destination profile, then computes: source numbers $\rightarrow$ PCS $\rightarrow$ destination numbers. Common CMMs: Adobe ACE, Apple ColorSync, Microsoft ICM, and the open-source **LittleCMS (lcms2)**.

Source profile

Describes where the color is coming *from* (e.g., the sRGB working space your design was edited in). It translates the device numbers into the PCS.

Destination profile

Describes where the color is going to (e.g., the specific press + paper you'll print on). It translates from the PCS into that target's numbers.

```

RGB triplet          Lab / XYZ          C, M, Y, K
(sRGB image)        (the PCS)          (press output)

|                   |                   |
| source profile    | destination    |
+-----> | profile |                   |
          |                   |                   |
          "what do these   +-----> |
            numbers mean?"         "what numbers
                                   reproduce that
                                   same color here?"

          [ all of this run by the CMM ]

```

Common mistake — Assign vs. Convert. "Assign profile" changes only the *dictionary* attached to the file — the pixel numbers stay identical, so the color appearance changes. "Convert to profile" recomputes the pixel numbers to *preserve* the appearance under a new profile. Assigning when you meant to convert (or vice versa) silently wrecks color. Rule of thumb: *Convert* to move into a new space while keeping the look; *Assign* only to fix a file whose profile was wrong or missing.

Working spaces vs. output profiles — the distinction everyone blurs

Not all profiles play the same role. There's a big difference between the space you *edit* in and the profile that describes the *final printing condition*.

Working space

A device-independent *editing* space — a clean, neutral RGB room to design in, not tied to any real monitor or printer. Examples below.

Output profile

A profile describing a *specific, real printing condition*: this press + this ink + this paper + measured under a defined condition. This is what you must convert to for print.

Profile	Type	What it is / when to use
sRGB	RGB working space (D65)	The web/consumer default. Smallest common gamut (~72% of NTSC). Safe for screen; clips some saturated cyans/oranges in print.
Adobe RGB (1998)	RGB working space (D65)	Wider in cyan-greens, ~50% larger volume than sRGB; roughly encompasses CMYK gamut. Good for print-bound photo work.
ProPhoto RGB	RGB working space	Enormous — larger than any real device. Powerful but you <i>must</i> soft-proof or you'll edit colors nothing can show or print.
GRACoL 2013 (CRPC6)	Output (US sheetfed)	Premium coated #1 paper. ISO 12647-2:2013, M1 measurement, total ink limit 300%.
Fogra39 (ISOcoated_v2)	Output (Europe)	The most widely used European coated profile. ISO 12647-2:2004, M0. _300 variant caps ink at 300% vs 330%.
Fogra51 (PSOcoated_v3)	Output (Europe, newer)	2015 update; ISO 12647-2:2013, M1 — handles modern optical-brightener papers.
SWOP	Output (US web/publication)	Higher dot gain. Use only if your printer specifically asks for it.

Gamut

The full range of colors a given device or space can reproduce. Bigger gamut = more colors possible.

TAC / total ink limit

The maximum allowed sum of C+M+Y+K (e.g., 300% or 330%). Exceed it and ink can't dry, smears onto the next sheet ("set-off"), or causes registration problems. This is why output profiles cap ink.

M0 / M1 measurement

How the profile's color was measured. M0 doesn't define UV light; M1 (D50, defined UV) correctly handles papers with optical brighteners. Mixing M0 and M1 profiles shifts color — don't.

Rendering intents: how out-of-gamut colors get squeezed in

When you convert from a big gamut (a vivid RGB photo) into a smaller one (a press), some colors simply can't be reproduced. The **rendering intent** decides how to handle them. There are four:

Intent	What it does	Use for
Perceptual	Squeezes the whole source gamut to fit, preserving smooth relationships and gradients. Some in-gamut colors shift slightly so everything stays harmonious.	Photographs.
Relative Colorimetric	Keeps in-gamut colors exact; clips out-of-gamut ones to the nearest printable color. Remaps source white to the paper white.	Most print work (the common default).
Saturation	Pushes colors to the gamut edge for maximum vividness, sacrificing accuracy.	Charts, logos, business graphics — not photos.
Absolute Colorimetric	Like Relative, but does <i>not</i> remap white — it simulates the source paper's white tint on the destination.	Proofing (a proofer mimicking another press's paper).

Black Point Compensation (BPC)

Scales the tonal range so the source's darkest black maps to the destination's darkest black, preventing shadows from blocking up into flat dark blobs. Always on for Perceptual, optional for Relative/Saturation, off for Absolute.

Pro tip — For 90% of print jobs the safe combo is **Relative Colorimetric + Black Point Compensation on**. Use Perceptual only when a photo has lots of vivid out-of-gamut color that would otherwise clip harshly. A correct print instruction always names *both* a target profile and an intent — e.g., "convert to Fogra39, Relative Colorimetric, BPC on."

Soft proofing — seeing the print before you print

Soft proofing means using your calibrated monitor to simulate, on screen, how the file will look once printed on a chosen output profile. In Photoshop this is `View > Proof Setup`. The CMM runs the destination profile's transform live on your display, optionally also simulating the dull paper white and the not-quite-black ink. The "Gamut Warning" overlay highlights colors the press physically can't print, so the designer sees the dullness and clipping *before* committing.

Common mistake — Soft proofing on an uncalibrated or narrow-gamut laptop screen. If the monitor itself lies about color, the simulation is worthless. Soft proofing requires a calibrated, wide-enough display.

Why "just convert to CMYK" is meaningless

This is the single most important idea in the section. **CMYK is not a color space — it's just four ink channels.** The same CMYK numbers print differently on every press, ink set, and paper. A conversion is only defined relative to a *specific destination output profile*.

Key idea — "Convert to CMYK" without naming a target profile silently falls back to some arbitrary default (often US Web Coated SWOP) and produces wrong, frequently dull or over-inked output. The correct instruction is always "convert to [*specific output profile*] using [*rendering intent*]" — for example, "convert to GRACoL 2013, Relative Colorimetric."

Real-world case — A designer sends a bright blue logo (`R=0, G=80, B=255` in sRGB) to a printer and says "I converted it to CMYK." Their software used the default SWOP profile. But the printer runs Fogra39 on premium coated stock. Two things go wrong: that saturated blue was outside *both* CMYK gamuts and got clipped to a duller blue, and the SWOP-vs-Fogra39 mismatch shifted every other color. The fix wasn't "convert to CMYK" — it was "convert to Fogra39, soft-proof first, and accept the blue will be a touch less electric."

Tying it to Print-Flow-360

Now the concrete part for our stack. PF360 today has a documented gap: **no RGB → CMYK conversion and no preflight**. Everything created in the Fabric.js **design studio** (`designer/`) and stored through the storefront is sRGB. The brand-color theming, the canvas, the customer's uploaded photo — all sRGB. There is currently no color-management step before a file would go to a press.

Here is where the pieces would live, mapped to the system you actually have:

- **The PDF service is the right home for color management.** The Node `pdf-service` (port 4000) already does heavy file work with sharp (images) and PDFKit (PDFs). A real color-managed pipeline belongs here, behind a feature flag like every other pdf-service delegation — never inside the designer or the storefront.

- **Store output ICC profiles, not "a CMYK setting."** The service must hold a library of destination output profiles (Fogra39, GRACoL 2013, an uncoated profile, newsprint, etc.) — one per real press/paper/ink condition — and let the right one be selected *per product or per substrate*. A single global "CMYK" toggle would reproduce exactly the "just convert to CMYK" mistake above.
- **Use a real CMM.** LittleCMS (lcms2) or Ghostscript can perform the source-profile → PCS → output-profile transform, apply the chosen rendering intent and BPC, and enforce that press's TAC limit. sharp can be wired to use lcms under the hood for the raster side.
- **Declare the destination in the file via preflight.** The industry-standard way to make a print file unambiguous is **PDF/X-4**, which carries an "output intent" (the embedded or referenced destination ICC profile) right inside the PDF. A preflight step would also flag out-of-gamut colors, missing fonts, and over-TAC ink — closing both the "no preflight" gap and the weak order-to-production spine.

Actionable takeaway for PF360 — The smallest correct first step is *not* "add CMYK conversion." It's: (1) attach a chosen output profile to each press/substrate the store offers, (2) in pdf-service, run the customer's sRGB artwork through a CMM into that profile with Relative Colorimetric + BPC, (3) enforce the profile's TAC limit, and (4) emit a PDF/X-4 with the output intent embedded. That sequence turns "we have no color management" into a print-correct file a real press can trust.

Key takeaways —

- An ICC profile is a *translation dictionary* that says what one device's color numbers actually mean; without it, RGB/CMYK numbers are ambiguous.
- Every conversion runs through the **Profile Connection Space** (Lab or XYZ, at D50): source profile → PCS → destination profile, executed by a **CMM** like LittleCMS.
- **Working spaces** (sRGB, Adobe RGB, ProPhoto) are for editing; **output profiles** (Fogra39, GRACoL 2013, SWOP) describe a real press+paper+ink condition. Don't confuse them.
- Pick a **rendering intent** for out-of-gamut colors — Relative Colorimetric + BPC for most print, Perceptual for vivid photos — and watch the result with **soft proofing** on a calibrated display.
- "Just convert to CMYK" is meaningless: CMYK is four channels, not a space. A conversion is only defined against a named output profile + intent.
- For PF360, color management belongs in the **pdf-service**: store an output ICC profile per press/substrate, convert via a CMM with the right intent and TAC limit, and embed it as a PDF/X-4 output intent — directly addressing the no-conversion and no-preflight gaps.

Rendering Intents

In the last section you learned that every device — a phone screen, a camera, a printer — can only reproduce a limited range of colors, called its *gamut*. A bright neon-green you see on screen often simply **does not exist** in the set of colors a printer with a given ink and paper can physically make. So when you send that image to print, the software has to make a decision: *what color do I print instead?*

A **rendering intent** is the rule the software follows to make that decision. It is the strategy for squeezing colors from a big space into a smaller one.

Gamut

The full set of colors a device or profile can reproduce.

Source space

Where the color is coming *from* — e.g. a photo in sRGB or Adobe RGB.

Destination space

Where the color is going *to* — e.g. a specific printer + paper combination, described by an ICC profile.

Out of gamut (OOG)

A source color the destination cannot reproduce. It *must* be changed to something printable.

Rendering intent

The gamut-mapping strategy: the rule for how OOG (and sometimes in-gamut) colors are moved to fit the destination.

Key idea — The destination (printer) gamut is almost always *smaller* than the source (photo) gamut. The rendering intent only matters when colors fall outside what the printer can do. An intent never *adds* new printable colors — it only *redistributes* the ones you have.

Two basic strategies: clip vs. compress

Before the four named intents, understand the two underlying mechanics. Everything else is a flavor of these:

Clip

Out-of-gamut colors get "snapped" to the nearest color the printer *can* make. In-gamut colors are left **exactly as they were**.

Compress

The *entire* source gamut is gently squeezed inward to fit, so even in-gamut colors shift a little — in exchange, the *relationships* between colors stay smooth.

CLIP	COMPRESS
source gamut (big)	source gamut (big)
* * * *	* * * *
OOG colors	\ \ \ \ everything
v v v v pushed to edge	\ \ \ \ squeezed in
[=====] dest gamut	[=====] dest gamut
in-gamut: unchanged	in-gamut: shifted, but
OOG: piled at boundary	spacing preserved

The four ICC rendering intents

The ICC (the standards body for color profiles) defines exactly four intents, each with a fixed number that appears in the profile's lookup tables:

#	Intent	What it does	Best for
0	Perceptual	Compresses the whole gamut; preserves smooth gradients & detail	Photos with many vivid/OOG colors
1	Relative Colorimetric	In-gamut exact; OOG clipped; adapts to paper white	Logos, brand colors, most photos
2	Saturation	Maximizes vividness, sacrifices accuracy	Charts, infographics, simple graphics
3	Absolute Colorimetric	Like relative but simulates the source's exact white	Proofing one device on another

1. Perceptual (compress everything)

Perceptual takes the *whole* image and scales it inward — the most saturated and darkest colors get pulled toward neutral, and in-gamut colors shift along with them. Nothing is perfectly accurate, but the *relationships* between colors stay intact, so gradients in skies and shadows stay smooth with no abrupt jumps (banding). Images can look slightly duller overall, but they look *natural*.

Analogy — Perceptual is like shrinking a group photo to fit a smaller frame. Everyone gets a little smaller, but the spacing between people stays proportional — the picture still "looks right." Nobody gets cropped out.

2. Relative Colorimetric (keep what fits, nudge the rest)

Every color that the printer *can* make is reproduced **exactly, 1:1**. Colors that don't fit are clipped to the nearest printable color at the gamut edge. It also maps the source's white to the paper's white — meaning it treats the actual paper as "white" and adapts everything else around that. This is the most accurate option for the bulk of an image.

The downside: when many different OOG colors all clip to the *same* boundary color, you can lose subtle detail there (e.g. several shades of deep blue all becoming one identical blue).

Analogy — Keep every guest who fits in the room; anyone who doesn't fit gets moved to stand next to the closest person already inside. Most people are exactly where they belong.

3. Absolute Colorimetric (simulate another device)

Identical to relative — in-gamut exact, OOG clipped — with one crucial difference: it does **not** remap white. It preserves the *source* white absolutely. If your source white is a cool bright white but the paper is cream, absolute will actually add a touch of cyan ink to simulate that cooler white, and let the cream paper tint show where intended.

Why would you ever want tinted whites? **Proofing**. When an inkjet proofer must mimic exactly what a big offset press on a specific paper will produce — tint and all — absolute reproduces that target faithfully. It is a *simulation* tool, never a normal-printing tool.

4. Saturation (maximum punch)

Saturation pushes colors toward the gamut edge for maximum vividness, sacrificing accurate hue and lightness. Great for a pie chart that needs to pop on a slide; terrible for a portrait, because it turns skin tones orange and unnatural.

Black Point Compensation (a separate, important switch)

Black Point Compensation (BPC) is *not* an intent — it's a separate on/off toggle you use *alongside* an intent.

Black Point Compensation

Maps the source's darkest black to the destination's darkest achievable black, smoothly scaling all the shadow tones in between.

It does for *black* what relative colorimetric does for *white*. Without BPC, shadows can "block up" — large dark areas turn into one flat lump of black with no detail — or, conversely, the printer's deepest black goes unused and shadows look washed-out gray. BPC applies to the colorimetric and saturation intents (perceptual handles black internally). The ICC authority is *White Paper 40* on color.org.

Best Practice — The industry default in Adobe apps and automated RIP (print-processing) workflows is **Relative Colorimetric + BPC ON**. Adobe enables BPC by default. Unless you have a specific reason, this is your safe starting point.

When to choose which

- **Photos with lots of vivid / OOG colors** (sunsets, deep blues, slide scans) → **Perceptual** — preserves gradients and detail.
- **Photos that mostly fit in gamut** → **Relative Colorimetric + BPC** — more accurate and slightly more saturated than perceptual.
- **Logos, brand colors, spot colors, accuracy critical** → **Relative Colorimetric** — exact match for in-gamut brand colors. (But a brand color that is itself OOG will still clip — relative cannot make an unprintable Pantone printable; flag it to the customer.)
- **Press proofs (simulate paper tint & white)** → **Absolute Colorimetric**.
- **Charts & vivid business graphics** → **Saturation**.

Real-world case — A customer uploads a logo using Pantone 285 C (a specific blue) to Print-Flow-360 for business cards. With **relative colorimetric**, if that blue is inside the printer's gamut, it prints essentially perfectly — the brand looks correct. If you had instead used **saturation**, the software might shove it toward a more electric, "wrong" blue for punch — unacceptable for a brand. This is why accurate intents matter for stationery, the bread-and-butter of a print storefront.

Connecting to the Print-Flow-360 stack

This is where theory becomes actionable. Print-Flow-360 has documented gaps that touch directly on rendering intents:

- **No RGB → CMYK conversion yet.** The Fabric.js **design studio** ([designer/](#)) works in screen RGB, and the Node **PDF service** ([pdf-service](#) , port 4000, using [sharp](#) for images and PDFKit for PDFs) currently passes color through without applying a printer ICC profile. That means *no rendering intent is being applied today* — colors are sent as-is, and the press's own RIP picks an intent blindly. This is exactly the missing link this section describes.
- **Where an intent *would* live.** When RGB → CMYK conversion is added, it belongs in the PDF service (the documented home for "heavy file work"), because [sharp](#) can apply an ICC destination profile with a chosen intent during image processing. The platform should default to **relative colorimetric + BPC** for logos/cards and offer **perceptual** as an alternative for photo-heavy products (canvas, photo prints).
- **No preflight.** A preflight step (checking files before print) is the natural place to *detect* OOG brand colors and warn the customer in plain language — "Your logo blue is brighter than this paper can print; here's the closest match" — instead of silently clipping it. That honesty is the difference between a trustworthy storefront and a surprised, angry customer.
- **Storefront brand-color theming** lives entirely in screen sRGB and is never printed, so rendering intents do not apply to it — don't confuse the two color pipelines.

Common Mistakes —

- Thinking an intent *adds* gamut. It only redistributes; an OOG brand color can only be approximated, never made printable.
- Using **Absolute** for normal printing → muddy, tinted whites and wasted ink simulating paper color.
- Turning **BPC off** → blocked-up, detail-less shadows.
- Using **Saturation** for photos → orange, unnatural skin tones.
- Assuming **Perceptual is always best for photos** — for in-gamut images it needlessly desaturates. Always soft-proof and compare both.
- Forgetting that intent *only matters when the source gamut exceeds the destination*. Converting into a *larger* space, all intents look nearly identical.
- Trusting a cheap profile's perceptual table — it's hand-tuned by the profile maker and may be poor, making relative the safer bet.

Key takeaways —

- A rendering intent is the rule for fitting colors from a big source gamut into a smaller printer gamut; it only matters when colors are out of gamut.
- The four ICC intents: **Perceptual** (compress, smooth gradients, photos), **Relative Colorimetric** (in-gamut exact + clip, the default), **Saturation** (max vividness, charts), **Absolute Colorimetric** (simulate another device, proofing only).
- **Black Point Compensation** is a separate toggle that protects shadow detail; keep it ON.
- Industry and Adobe default = **Relative Colorimetric + BPC**; that is the right storefront default for logos, cards, and stationery, with Perceptual offered for photo products and Absolute reserved for internal proofing only.
- For Print-Flow-360: no intent is applied today (no RGB → CMYK); when added, it belongs in the **pdf-service** via **sharp**, and a preflight step should warn customers about OOG brand colors rather than silently clipping them.

Section 5: Gamut & Out-of-Gamut Handling

You pick a gorgeous electric-blue logo on your screen. It glows. You order 500 business cards. They arrive looking like a tired, dusty navy. Nobody made a mistake at the print shop — the color you chose was simply *impossible to print*. This section explains why that happens, what the industry does about it, and how a print platform like Print-Flow-360 can warn a customer **before** they hit "Order" instead of after.

5.1 What is a gamut?

Gamut

The complete range of colors that a particular device or color space can capture, display, or reproduce. Think of it as a "menu" of colors — anything not on the menu can't be served.

Color space

A defined system for describing colors with numbers (for example, RGB or CMYK). Each color space has its own gamut.

Device-independent reference space

A neutral "map of all colors a human can see," used so we can compare different devices fairly. The two common ones are **CIE 1931 xy** (the famous horseshoe-shaped diagram) and **CIELAB / CIE L*a*b*** (a 3D space).

A gamut is best pictured as a 3D *solid* sitting inside that reference space. The outer horseshoe boundary represents every color the human eye can perceive. Every real device — your phone, your monitor, a printing press — can only reach *part* of that horseshoe. Its gamut is a smaller blob inside.

Analogy — The human-visible horseshoe is the whole box of 120 crayons. Your monitor got a 64-crayon box. The printing press got a 40-crayon box — and a few of *its* crayons aren't even in the monitor's 64. A color is "out of gamut" simply when the crayon you want isn't in the box you're using right now.

Key idea — "Out of gamut" (OOG) is never absolute. A color is out of gamut *for a specific target* — this monitor, or this printer + this paper + this ink. Always ask "out of gamut for *what?*"

5.2 Why CMYK can't print everything your screen shows

This comes down to physics — two completely different ways of making color.

RGB (additive color)

Used by screens. You start with a black screen and **add** red, green, and blue *light*. All three at full = white. Color comes from *emitted light*, so it can be intensely bright and saturated.

CMYK (subtractive color)

Used by printing. You start with *white paper* and lay down Cyan, Magenta, Yellow, and black ink. Ink **absorbs (subtracts)** some wavelengths and reflects the rest. More ink = darker. Color comes from *reflected light*.

Because ink can only *absorb* light and never *emit* it, two hard limits apply: the brightest a printed color can ever be is limited by the paper's whiteness, and the most saturated it can be is limited by the pigment. Glowing neon greens, vivid cyans, electric blues, hot oranges, and saturated purples are exactly the colors pigments struggle with. So the CMYK solid is a *smaller* blob that mostly nests *inside* RGB.

Human-visible (CIE horseshoe)

```

/
|  RGB monitor gamut
|  .....
|  :   CMYK print gamut   :
|  : (smaller, inside)   :
|  :.....:
|      ^ a tiny sliver of CMYK
|      pokes OUT of sRGB
|
\
```

Common mistake — Believing "CMYK is always smaller than RGB everywhere." Mostly true, but not strictly. Offset CMYK profiles like **FOGRA39** or **GRACoL/G7** are smaller than Adobe RGB overall, yet slightly *exceed* plain **sRGB** in a few spots (very saturated cyans and yellow-oranges). A handful of printable colors actually live outside sRGB.

Some rough, illustrative figures: people often cite RGB as ~16 million addressable shades versus CMYK reproducing maybe ~16,000 — treat those as ballpark, not gospel. On wider RGB spaces: **Adobe RGB** covers about **86.2%** of Pointer's gamut (the range of real-world surface colors) and was deliberately

designed to include more cyan-green for cleaner CMYK conversion; **ProPhoto RGB** reaches ~90% of real-world colors; **sRGB** is similar to Adobe RGB in overall coverage but weaker in the cyan-green region.

5.3 What happens to an out-of-gamut color? Clipping vs. compression

When a color can't be printed, software must decide where to *put* it instead. That decision is controlled by the **rendering intent**.

Rendering intent

The rule for mapping out-of-gamut colors into the destination gamut. The ICC standard defines four: **Perceptual**, **Relative Colorimetric**, **Absolute Colorimetric**, and **Saturation**.

They fall into two big strategies:

Strategy	Intents	How it works	Risk / best for
Clipping	Relative & Absolute Colorimetric	In-gamut colors stay <i>exact</i> ; only OOG colors get snapped to the nearest printable boundary color.	Many different OOG colors can collapse onto the same edge color → lost detail (posterization, blocked saturated areas, hue shifts). Best when few colors are OOG (logos, brand colors you want kept exact).
Compression	Perceptual	Squeezes the <i>entire</i> source gamut to fit the destination, preserving the <i>relationships</i> between colors.	Even in-gamut colors lose a little saturation, but gradients stay smooth and nothing snaps abruptly. Best for photos with <i>many</i> OOG colors.

The fourth intent, **Saturation**, maximizes vividness over accuracy — good for charts and business graphics, bad for photos.

Analogy — Clipping is shoving everyone who's too tall against the back wall of a photo — they all line up at the same spot and you lose who was taller. Compression is asking the *whole crowd* to take one small step back so everyone still fits in frame and you can still tell them apart.

Black Point Compensation (BPC)

An option that maps the source's darkest black to the destination's darkest black so shadows don't get crushed or muddy. Keep it **ON** for most work.

Common mistakes — (1) Using **Absolute** when you wanted **Relative** — Absolute even simulates the source *paper white* (it's for proofing one printer on another), giving an odd tint. (2) Forgetting BPC and getting blocked, muddy shadows. Sensible default for most graphics: **Relative Colorimetric + BPC on**.

Key idea — Conversion by clipping is *irreversible*. Once two distinct colors are snapped to the same boundary color, the difference is gone forever. Never treat RGB → CMYK as something you can undo.

5.4 Seeing the problem early: soft proof & gamut warning

Soft proof

An on-screen *simulation* of how the print will actually look, viewed through a specific printer + paper ICC profile, before any ink is used. In Photoshop: **View > Proof Setup > Custom**, then toggle **Proof Colors** with **Ctrl/Cmd+Y**.

Gamut warning

An overlay (Photoshop: **Shift+Ctrl/Cmd+Y**) that paints a flat gray mask over the exact pixels that fall *outside* the destination gamut and will be remapped significantly.

ICC profile

A standardized file describing a specific device's gamut, so color can be translated accurately between devices.

The difference matters: **Soft Proof shows you the simulated final look** (more useful for judging the real outcome), while **Gamut Warning just flags which pixels are risky**. The Color Picker also shows a small triangle "gamut alarm" with a swatch of the nearest in-gamut substitute you can click to accept.

Common mistakes — Soft-proofing on an *uncalibrated* monitor or with the wrong/generic profile — garbage in, garbage out. And panicking at every gamut warning: a flagged color that still looks fine in the soft proof needs no edit. A warning means "inspect," not "broken."

5.5 Handling it gracefully: suggesting the nearest printable color

The kind move is to warn early, non-blockingly, and offer the closest color that *will* print. "Closest" is measured with a perceptual distance called **Delta E (ΔE)**.

Delta E (ΔE)

A number for how different two colors look to the human eye. Smaller = closer. A **just-noticeable difference (JND)** is roughly ΔE 1–2; most tools treat $\Delta E < 2$ as "close enough to be indistinguishable."

The ΔE formulas, oldest to best: **CIE76** (plain Euclidean distance in Lab — crude), **CIE94**, **CIEDE2000** (the industry standard, perceptually tuned), **ΔE_{OK}** (Euclidean in Oklab), and **ΔE_{ITP}** (for HDR).

The modern web approach — the CSS Color Module 4 default, used by libraries like colorjs.io — is smart: instead of naively clipping channels, it works in **OkLCh** (a perceptual hue/chroma/lightness space). It *holds the hue and lightness fixed* and does a binary search reducing **chroma** (saturation) until the color fits, comparing each guess to a channel-clipped version with ΔE_{OK} and stopping when $\Delta E < 2$. (If lightness $\leq 0 \rightarrow$ black; $\geq 1 \rightarrow$ white.) Chroma-reduction keeps the color recognizably "the same hue, just calmer," which beats naive clipping for most hues; yellows are a known weak spot.

5.6 Bringing it to Print-Flow-360

Here's the honest state of the stack: PF360 has **no documented RGB $\rightarrow$ CMYK conversion and no preflight step**. The Fabric.js **design studio** (in [designer/](#)) lets customers pick any RGB/hex color, the storefront brand-color theming is RGB, and the Node **PDF service** ([pdf-service](#), port 4000, using [sharp](#) for images and [PDFKit](#) for PDFs) outputs RGB PDFs. So a neon hex chosen in the studio sails straight through to a PDF with *no* warning — the customer discovers the dull result only on the doorstep. That's a textbook **silent-lie** bug: the screen promised something the print can't keep.

Worked example — a graceful PF360 gamut warning.

1. Customer picks `#00FF88` (electric mint) for a flyer headline in the Fabric.js studio.
2. On color-pick, check it against a target print profile (a FOGRA-like CMYK gamut for the product's print process). ΔE to its nearest printable color is large $\rightarrow$ it's OOG.
3. Show an *inline, non-blocking* message in **plain language**: "This bright color may print noticeably duller than it looks on screen." (Per PF360 UX rules: reserve the layout space so nothing jumps, never a raw error, never block.)
4. Render a **live swatch of the nearest printable color** (computed via CIEDE2000 or the OkLCh chroma-reduction method, stopping at $\Delta E < 2$) with a one-click **"Use printable color"** button.
5. **Suggest, never substitute.** If the customer keeps their color, honor it. Silently swapping it would be its own silent-lie bug.
6. If no color is out of gamut, behavior is byte-for-byte unchanged — no warning, no surprises.

Pro tip — The gamut check, the ΔE math, and a true RGB $\rightarrow$ CMYK transform all belong on the **Node PDF service**, not bolted into the browser studio. It's the heavy-file, server-side layer that already owns rendering — the natural home for a future preflight step that flags OOG colors, low-res images, and missing bleed, strengthening the platform's weak order-to-production spine.

Key takeaways —

- **Gamut** = the colors a device can reproduce; OOG is always relative to a specific target (printer + paper + ink), never absolute.
- CMYK (subtractive ink) is mostly smaller than RGB (additive light) because pigment can't emit — neons, bright greens/blues/oranges are the usual casualties — though a few CMYK cyans/yellow-oranges poke outside sRGB.
- **Clipping** (Relative/Absolute Colorimetric) keeps in-gamut colors exact but snaps OOG colors to the edge, losing detail; **compression** (Perceptual) squeezes everything to keep relationships smooth — best for photo-heavy art. Conversion is irreversible.
- Default to **Relative Colorimetric + Black Point Compensation on** for most graphics; Perceptual for photos with lots of OOG color.
- **Soft proof** simulates the real printed look; **gamut warning** just flags risky pixels. Both need a calibrated monitor and the correct profile, or they lie.
- Suggest the nearest printable color by minimizing ΔE (CIEDE2000, or OkLCh chroma-reduction stopping at $\Delta E < 2$ — one JND).
- PF360 today has no RGB → CMYK conversion or preflight. The right fix: a friendly, non-blocking gamut warning in the studio plus a real CMYK/preflight step in the Node PDF service — warn and suggest, never silently substitute or block.

Section 6: Spot Colors, Pantone, Overprint & Knockout

So far you've thought about color as something a screen makes with light. Print makes color with *ink on paper*, and that changes everything. This section covers two big topics that trip up almost every beginner: how printers create exact, named colors that go beyond the standard four-ink mix, and how overlapping inks decide whether to *mix* or *cover* each other. Get these wrong and your file looks perfect on screen but prints wrong — or worse, parts of it vanish entirely.

6.1 Two ways to make color: process vs. spot

There are two fundamentally different ways a printing press lays down color.

Process color (CMYK)

Color built from four inks — **C**yan, **M**agenta, **Y**ellow, and **K**ey (black) — printed as tiny overlapping dots. Your eye blends the dots into a full-color image. CMYK is how photos and gradients get printed.

Spot color

A single ink that is *pre-mixed* in a bucket to be exactly one color, then printed as one solid layer (one "plate"). No dots — just a flat, uniform coat of that exact color.

Plate / print station

One ink's worth of machinery on the press. CMYK needs four plates (one per ink). Each spot color needs its own additional plate.

Here's the giveaway: if you put a magnifying loupe on a CMYK area you'll see a tiny dot pattern called a *rosette*. On a spot-color area you'll see a smooth, solid coat — no dots at all. That solidity is why spot colors look razor-sharp for logos, line art, and text.

Analogy — CMYK is like a TV mixing red, green, and blue pixels to fake every color. Spot color is like buying a can of "Tiffany Blue" paint at the hardware store — it's already exactly the color you want, mixed to a published recipe, no faking required.

6.2 Pantone (PMS): the universal color dictionary

If a designer in New York says "make it green," that's useless to a printer in Shenzhen. The **Pantone Matching System (PMS)** solves this. It's a catalog of roughly 1,800+ pre-mixed spot inks, each with a number and a published recipe (mixed from about 18 base inks). When you specify **PANTONE 185 C**, any printer in the world mixes the identical ink. The Pantone number is a precise, contractual spec that travels between people.

Notice the letter suffix — it matters a lot:

Suffix	Means	Why it looks different
C	Coated paper	Glossy/smooth stock; ink sits on top, looks vivid
U	Uncoated paper	Porous stock soaks ink in, looks duller/softer
M	Matte paper	In between

The *same* Pantone number prints noticeably different on coated vs. uncoated paper. A brand guideline that says just "Pantone 185" is incomplete — it should say "Pantone 185 C" so everyone knows the finish.

Common mistake — Writing "Pantone 185" with no suffix, or worse, just handing over a HEX/RGB value and expecting an exact ink. Without the C/U/M suffix the printer is guessing, and your brand color drifts run to run.

6.3 Gamut: why some brand colors can't be faked in CMYK

Gamut

The full range of colors a given system can physically produce.

The spot/Pantone gamut is **wider** than the CMYK gamut. A commonly cited figure is that more than 30% of Pantone colors simply *cannot* be reproduced accurately in CMYK — especially vivid oranges, bright greens, and deep blues. When you force such a color into CMYK, the press prints the closest it can and the color visibly shifts. This is called *gamut clipping*.

Pantone publishes **Color Bridge** guides that show, side by side, each spot color and its best CMYK approximation plus an sRGB/HEX value — so you can see how much a color will shift before you commit.

Real-world case — Brands like Tiffany Blue (1837), UPS Brown (Pantone 476), and John Deere green are defined by exact Pantone inks precisely because CMYK would drift press-to-press and never hit them reliably. The Pantone number is the legal source of truth for the brand color.

6.4 When spot is required (impossible in CMYK)

Some effects aren't a "nice to have" spot — they are the *only* way:

- **Metallics** — gold, silver, copper inks contain real metal-flake pigment. CMYK has no shine.
- **Fluorescents / neons** — the Pantone 800-series glows; CMYK can't.
- **Ultra-vivid or very pale tints** outside the CMYK gamut.
- **Opaque white** — there is *no white ink* in CMYK (white = the bare paper). To print white on clear plastic, kraft paper, or a dark stock, you need a separate white spot plate, often called a "white

pass."

6.5 Cost: when spot saves money, when it burns it

Job	Cheaper choice	Why
1–2 flat colors (e.g., a green logo on a white box)	Spot	1 plate vs. 4 — less setup, less ink
Full-color photo / many colors	Process (CMYK)	4 plates cover everything
CMYK photo + 1 exact brand color or varnish	Mixed (costs more)	"5th color" = extra plate, press unit, and wash-up

Wash-up

Cleaning the press and swapping inks between runs. Every extra spot ink adds a wash-up, which adds time and money.

Pro tip — Rule of thumb: few flat colors → spot; many colors or photos → process; brand-critical or specialty (metallic/neon/white) → spot regardless of count. And always delete *stray* spot colors from a CMYK-only job — a forgotten spot becomes an unexpected extra plate and an unexpected bill.

6.6 Overprint vs. knockout: how overlapping inks behave

When one colored object sits on top of another, the file must decide what happens to the ink underneath. There are two behaviors.

Knockout (the default)

The top shape "punches a hole" in the ink below, so the top ink prints on bare paper. Inks don't mix. This keeps colors clean.

Overprint

The ink below is *left in place* and the top ink prints directly on top of it. Because process inks are see-through (transparent), the two colors blend optically.

KNOCKOUT

```
+-----+
| BLUE field |
|   _____   |
| | (hole) | |
| | yellow | |
| |_____ | |
+-----+
```

yellow prints on
bare paper = YELLOW

OVERPRINT

```
+-----+
| BLUE field |
|   _____   |
| | yellow | |
| | ON blue | |
| |_____ | |
+-----+
```

yellow + blue mix
optically = GREEN

Set yellow type to overprint a blue field and it reads **green** — almost always a surprise. You control this with the "Overprint Fill" and "Overprint Stroke" checkboxes in Illustrator/InDesign's Attributes panel.

6.7 Legitimate uses of overprint (it's not always a bug)

Overprint exists for good reasons, mostly involving *black* and *registration*.

Registration

How precisely the press lines up all the plates. Plates shift a hair — that's *misregistration*.

Trapping

Deliberately overlapping adjacent colors by a tiny amount so a misregistration gap doesn't show as a white sliver.

The classic good use: **small black text**. If solid black (100% K) type is knocked out and the plates shift slightly, you'd see a thin white or colored halo around every letter. Set that black to *overprint* instead and it prints on top of whatever's behind it — no hole, no halo. Black is opaque enough that the color underneath doesn't visibly change it. Many apps and RIPs (the software that drives the press) turn on "overprint black" by default. Spot varnish, foil, spot UV, and white-pass layers also rely on overprint.

6.8 Black done right: 100% K vs. rich black

100% K (plain black)

Just the black plate. Sharp, single plate — perfect for small text and fine lines.

Rich black

Black plus supporting CMYK (a common build is C60 M40 Y40 K100; recipes vary by shop) for a deeper, darker black on large fills.

Common mistake — Using rich black on small text (under ~10 pt). Because it uses multiple plates, any misregistration shows colored fringes around the letters — fuzzy and amateur. Rule: small/fine text → plain 100% K set to overprint; large solid areas → rich black.

6.9 THE classic trap: white text set to overprint disappears

This is the single most famous, most damaging beginner trap in print. Walk through the logic:

- Overprint means "don't knock out the background — print on top of it."
- But there is *no white ink* in CMYK. White is the bare paper showing through.
- So when white (or "[Paper]") is set to overprint, the press prints... nothing. The background shows straight through and your white element is simply **gone**.

On screen in normal view it looks perfect. On paper, the white logo or white headline vanishes. There's even a long-standing InDesign defect (CS3 through CC): make a style with black + Overprint Fill, then

change its color to white/[Paper] — the overprint checkbox grays out visually but the overprint flag *stays on* in the file, silently.

Pro tip — how to catch it — Turn on **Overprint Preview** (View menu in InDesign/Illustrator, or Acrobat's Output Preview). This simulates what the press will actually do — disappearing white instantly reveals itself. Normal screen view *lies*; overprint preview tells the truth.

Preflight

An automated pre-print check that scans a PDF for problems (overprinting white, low-res images, RGB colors, stray spots) before it goes to press. Tools: Adobe Acrobat Preflight, callas pdfToolbox, Enfocus PitStop, Markzware FlightCheck.

6.10 The standard behind it: Ghent Workgroup (GWG)

The **Ghent Workgroup** is an international prepress-standards body (printers, publishers, software vendors). It publishes preflight profiles built on the **PDF/X-4** exchange standard, with a test suite of 260 files and 14 certified variants. Among other things, GWG profiles **flag overprinting white line art** as a warning and **flag overprint on text ≤5 pt** (≤8 pt for newsprint). Its "Universal Proof of Preflight" embeds metadata in the PDF documenting how it was checked. **PDF/X-4** and **PDF/X-1a** are the standard formats for sending files to print.

6.11 How this connects to Print-Flow-360

PF360 has a **brand-color storefront feature**: a store owner picks one primary brand color — a HEX value like **#0FB5A6** — and a CSS **color-mix** cascade rebrands the whole storefront via **--theme-color-primary**. Crucial point for this section: *that HEX is a screen/RGB color — additive light on a monitor*. It is not a print spec.

The platform's documented gaps make this concrete and actionable:

- **No RGB → CMYK conversion** — when a customer designs a product in the Fabric.js design studio (**designer/**), they pick RGB colors. Nothing currently converts those to CMYK or warns when a vivid brand color is *out of CMYK gamut*. The on-screen color may simply not be printable.
- **No preflight** — the Node PDF service (**pdf-service/**, port 4000, using sharp for images and PDFKit for PDFs) generates the print artifact, but there is no preflight step to catch overprinting-white, low-res images, or stray spot colors before output. This is exactly where a GWG-style check would live.
- **Weak order-to-production spine** — there's no structured handoff that carries a Pantone reference or print-color intent from the storefront brand color through to the press-ready file.

Actionable guidance for a print SaaS — Treat the screen brand color and the printed brand color as two coordinated-but-different specs. Capture the merchant's HEX for the storefront UI, *and* let them store a Pantone/PMS reference (with the C/U suffix) for physical products. When an RGB brand color falls outside CMYK gamut, warn the merchant before they print. For any print output, flag white-on-dark elements and overprint risks — the disappearing-white trap is the highest-value thing a preflight step could catch.

Key takeaways —

- **Process (CMYK)** = four inks dotted together for photos/full color; **spot** = one pre-mixed solid ink, sharp and exact, on its own plate.
- **Pantone (PMS)** is the universal color dictionary; always include the finish suffix — **C** (coated), **U** (uncoated), **M** (matte) — because the same number prints differently on different paper.
- 30%+ of Pantone colors fall **outside CMYK gamut**; check a Color Bridge before assuming a brand color is printable.
- Metallics, fluorescents, and **white** are impossible in CMYK — they require spot inks (white = a separate "white pass").
- **Spot saves money** on 1–2 flat colors (fewer plates); it **costs more** as a 5th color added to CMYK (extra plate + wash-up).
- **Knockout** (default) punches a hole so inks don't mix; **overprint** prints on top so transparent inks blend.
- Overprint solid **100% K small text** (avoids halos); never use rich black on small type. Use rich black only on large fills.
- **White set to overprint disappears** — there's no white ink to lay down. Always check with **Overprint Preview**; normal screen view lies.
- **Preflight** (GWG/PDF-X profiles) catches overprint-white and tiny-text overprint before press.
- In PF360, the storefront brand **HEX is screen color, not print color**; the missing RGB → CMYK conversion and preflight steps in the design studio + PDF service are exactly where gamut and overprint warnings belong.

Section 7: Total Ink Coverage (TAC) & Dot Gain

So far you've learned how colors are built. Now we tackle two problems that only show up when ink actually meets paper on a real press: putting down *too much* ink, and ink dots quietly growing larger than you asked for. Both are invisible on your screen. Both can ruin a print run. And both are exactly the kind of thing a good platform should catch *before* a customer's job goes to production. Let's start from zero.

What "Total Ink Coverage" actually means

Recall that in print, color is made from four inks: Cyan, Magenta, Yellow, and black (CMYK). Each is laid down as a percentage from 0% to 100%. A bright red might be something like C0 M90 Y80 K0.

Total Area Coverage (TAC) / Total Ink Coverage (TIC) / Total Ink Limit

The sum of the four CMYK percentages at a single spot on the page. These three names mean the same thing — Acrobat says "Total Area Coverage," InDesign says "Total Ink Coverage." If a dark shadow is C70 M60 Y60 K90, its TAC is $70 + 60 + 60 + 90 = \mathbf{280\%}$.

Because each channel maxes out at 100%, the absolute ceiling is $100 + 100 + 100 + 100 = \mathbf{400\%}$. TAC matters most in the darkest areas of a design — deep shadows, black backgrounds, dense photos — because that's where all four inks pile up at once.

Analogy — Think of painting a wall. One thin coat dries fast. Slap on four heavy coats at once and the bottom layers stay wet, the paint sags, and if you stack a wet board on top, paint transfers onto it. Ink on a fast press behaves the same way: there's a limit to how many wet layers paper can hold and still dry.

Why too much ink fails on press

A printing press runs fast and stacks freshly printed sheets right on top of each other. The paper can only absorb and dry so much ink. Push past its limit and you get real, costly defects:

Drying failure

Wet ink simply piles up and won't dry — the sheet stays tacky.

Set-off (also called offsetting)

A still-wet sheet rubs ink onto the *back* of the sheet stacked on top of it in the delivery pile. Don't confuse this with **show-through**, which is ink visible *through* the paper from the other side.

Smearing

The job smudges the moment anyone handles it.

Trapping failure

The last ink laid down can't stick ("trap") to the wet layers beneath it. Instead of a clean neutral black you get a muddy, dirty **brown**.

Picking, blistering, sticking

Paper fibers tear off, heatset presses can blister, and pages glue together.

The core mechanism is simple: **ink laid down last cannot adhere to wet layers underneath it**. More layers than the paper can dry = trouble.

Typical TAC limits — they depend on paper and press

There is no single magic number. Rougher, more absorbent paper and faster web presses hold *less* ink. The smoother the paper and slower the press, the *more* ink it tolerates.

Press / paper	Typical TAC limit
Sheetfed offset, coated paper	300–340% (often 320–340%)
Heatset web offset (magazines), incl. SWOP	300% (some say 300–320%)
SWOP standard (U.S. web offset publications)	300%
GRACoL / coated commercial	~300–320%
Non-heatset web, uncoated / newsprint	240–260% (newspapers ~240%)
Inkjet / digital copiers	300–350%
Safe "unknown printer" default	~300% (board-game guides: ≤320%)

Pro Tip — When in doubt, design to ~300% TAC. It's safe almost everywhere. But always defer to the printer's spec sheet or ICC profile — that document is the real authority, not a rule of thumb.

The three "blacks": plain, rich, and registration

This is where beginners make the single most expensive mistake in print, so go slowly.

Plain (standard) black — K100 only, TAC = 100%

Just the black plate, nothing else. Perfect for small text (under ~24pt, ideally under 12pt) and thin lines. Because only one plate prints, the edges stay crisp and there's no risk of colored halos. Downside: on a big filled area it can look flat and grayish.

Rich black — K100 + support inks, deeper black for large areas

Add some C, M, and Y under the black so big backgrounds and large display type look truly deep. The recommended neutral recipe is **C60 M40 Y40 K100 = 240%** — dense, no color cast, and safely under most limits. Other safe versions: **C40 K100** (140%) or **C30 M30 Y30 K100**. A "cool black" leans more cyan; a "warm black" leans more magenta.

Registration black — C100 M100 Y100 K100, TAC = 400%

The built-in "[Registration]" swatch. It exists for **one purpose only**: crop marks and registration marks, so they print on every single plate. It is *not* a fill color.

Common Mistake — Filling a large area or background with [Registration] black (400%). On screen it looks like a perfect black, so people pick it by accident constantly. On press it blows past every TAC limit: it won't dry, it sets off onto other sheets, it goes muddy, and it can damage the press. This is the #1 catastrophic ink mistake in print — and exactly what a preflight check must flag.

Common Mistake — Using rich black on *small* text. Each plate on a press can shift a hair (misregistration). On big areas you'd never notice, but on tiny letters the C/M/Y plates peek out from behind the black as colored, fuzzy halos. Rule: **small text = K100 only; large fills = rich black.**

Dot gain: ink dots grow, midtones darken

Halftone printing makes shades by printing tiny dots of varying size — a "50% gray" is dots covering half the area. The problem:

Dot gain (modern term: TVI — Tone Value Increase)

Halftone dots physically and optically grow during platemaking and printing, so the print comes out **darker** than you intended. Midtones fill in the most.

The measurement convention trips everyone up, so memorize it: **"20% dot gain at 50%" means a 50% tint prints as 70%** (50 + 20), *not* 60%. The default reference patch is the 50% midtone unless stated otherwise.

```
You requested          What actually prints (20% gain)
  50% gray  ----->      70% gray  (darker!)

0% ..... 50% ..... 100%
          ^^^
          midtones                gain PEAKS in the
                                   midtones (~40-60%)
```

Why do midtones gain most? Around 50%, the dots have the most *edge* (perimeter), and it's the edge that spreads. In deep shadows the dots have already merged together, so there's less edge left to grow.

Typical magnitudes: sheetfed offset on coated paper gains ~12–20% at 50% (a 20% TVI is a common standard → 50% prints as 70%). Newsprint and uncoated stock gain *more* than coated. There are two flavors:

- **Mechanical (physical) dot gain** — ink squeezes outward during transfer and absorbs/spreads into the paper (worst on newsprint).
- **Optical dot gain** — light scatters inside the paper around each dot, making it *look* bigger than it measures.

How dot gain is compensated

The fix is delightfully counterintuitive: **print smaller dots than your target so they grow into the right value.**

- **Plate curves / linearization** on the RIP or platesetter: if gain is 5%, the device images a 50% as 45% so it lands at 50%.
- **ICC output profiles / standards:** ISO 12647-2 (offset), GRACoL, and SWOP define target TVI curves, so files separated to those profiles are pre-compensated.

- **Photoshop** auto-compensates during RGB → CMYK conversion using its CMYK working space / "Dot Gain" setting.

Common Mistake — Building flat tints in a *vector* app like Illustrator and ignoring dot gain. Vector apps and hand-built flat tints get **no** automatic compensation — that manually built 50% gray can print muddy and dark. Photoshop helps; Illustrator does not.

How this connects to Print-Flow-360

Right now PF360 has documented gaps that map straight onto this section: **no RGB → CMYK conversion** and **no preflight**. That matters because the Fabric.js design studio ([designer/](#)) works entirely in RGB on screen, and the storefront brand-color theming is RGB/hex too. A customer can design a gorgeous near-black background, hit order, and have it convert to a 380%+ four-color black that no press can dry — and nothing today warns them.

Real-world case — A customer picks pure black for a full-bleed poster background in the studio. On screen: perfect. After a naive RGB → CMYK conversion it becomes roughly C95 M85 Y80 K90 ≈ 350% TAC. The print shop gets a job that sets off all over the delivery pile. The fix the platform should have offered: remap it to rich black **C60 M40 Y40 K100** (240%).

The natural home for the check is the Node PDF service ([pdf-service/](#) , port 4000). It already uses **sharp** for image processing and **PDFKit** for PDF generation, so it can composite/rasterize the artwork and inspect actual ink coverage region by region — which is essential, because some checkers only test vector objects and miss the worst offenders hiding in images and overprints. A preflight stage there would strengthen the platform's weak order-to-production spine.

The preflight check PF360 should run

- Flag any region whose TAC exceeds a configurable limit (default 300%; coated 320%, uncoated 260% — settable per paper/press).
- Specifically detect **[Registration]/400% fills used as artwork** (not just on marks).
- Detect rich-black text below a threshold point size (halo risk).
- **Rasterize/composite first** so images, transparency, and stacked overprints are caught — not just vectors.
- **Warn, don't hard-block** — let the user override, consistent with the platform's UX rules.

Pro Tip — Follow PF360's "never surface raw technical output" rule. Don't say "TIC 400% violation." Say: *"This area uses 400% ink (registration black). It won't dry and may smear. Use rich black C60 M40 Y40 K100 (240%) instead,"* with a one-click "Fix it" button. Always offer the fix, not just the error. Reference tools that do this analysis: Acrobat Pro → Output Preview → Total Area Coverage; InDesign Separations Preview → Ink Limit; Enfocus PitStop.

Key takeaways —

- **TAC = TIC = Total Ink Limit:** the sum of C+M+Y+K at one spot. Max is 400%; real presses want roughly **240–340%**, with ~300% a safe default and uncoated/newsprint as low as 240–260%.
- Too much ink causes drying failure, **set-off**, smearing, and muddy-brown trapping failure — always defer to the printer's spec sheet/ICC profile.
- **K100** for small text (crisp, no halos); **rich black C60 M40 Y40 K100 (240%)** for large areas; **never** use 400% registration black as a fill — it's the #1 catastrophic ink defect.
- **Dot gain (TVI)** darkens prints by growing dots; "20% at 50%" means 50% prints as **70%**, peaking in the midtones. Compensate by imaging smaller dots via plate curves and ICC profiles — Photoshop auto-handles it, Illustrator/flat tints do not.
- PF360's missing RGB → CMYK + preflight is a real production risk; the right place to add a TAC/registration-black check is the **pdf-service** (sharp + PDFKit), as a plain-language *warning with a one-click fix*.

Resolution: DPI, PPI & Why Print Needs 300

You design something beautiful on screen. It looks crisp, colorful, perfect. You send it to print and it comes back soft, fuzzy, jagged at the edges. Nothing was wrong with your eyes — the file simply did not carry enough detail to survive the jump from glowing screen to physical paper. This section explains the number behind that disaster: **resolution**. Get it right and your prints look professional. Get it wrong and no printer on earth can save you.

First, two words everyone confuses: PPI and DPI

These two acronyms are the single biggest source of confusion in print. People use them interchangeably, including software you'll use. Let's pin down what each one actually means.

Pixel

The smallest square of an image. One pixel holds one full color (for example, a specific shade of red). A digital photo is a grid of these — say 3000 pixels wide by 2000 tall.

PPI — Pixels Per Inch

A property of your **file**. It tells the printer how many of those image pixels to squeeze into one inch of printed space. This is the number you control.

Dot

A single tiny droplet of ink (or speck of toner) the printer physically sprays onto paper. Each dot is one ink color only.

DPI — Dots Per Inch

A property of the **printer hardware**. It tells you how many ink dots the machine can lay down per inch. You don't set this in your file — the machine does.

Key distinction — When you talk about a *file*, the correct word is **PPI**. When you talk about the *printer*, the correct word is **DPI**. The famous "300 DPI rule" for artwork is technically a 300 *PPI* rule — it's about your file, not the machine.

Why does everyone say "300 DPI" then? Because old habits and Adobe's own labels mixed the terms decades ago, and it stuck. We'll teach it correctly, but don't be surprised when a print shop emails you asking for "300 DPI files" — they mean 300 PPI in the image.

Pixels are not printer dots — and that's important

Here's the part that surprises beginners: printer DPI is usually *much* higher than image PPI. A home inkjet quotes 1200–5760 DPI. A laser printer does 600–1200 DPI. An offset press (the big commercial machines) is different again. Yet we only ask for 300 PPI in the file. Why the mismatch?

Because one pixel is a full color, but one printer dot is a single ink color (Cyan, Magenta, Yellow, or Black — the CMYK set from the color section). To reproduce one continuous-tone pixel — say a soft pinkish-gray — the printer must place a tiny cluster of cyan, magenta, and black dots that your eye blends together. This trick is called **halftoning** (or **dithering** for the random-dot version).

Analogy — A pixel is a single chord played on a piano — one rich, complete sound. A printer dot is one finger pressing one key. To play that chord, the printer needs several fingers (dots) working together. That's why it needs thousands of dots per inch to render a few hundred pixels per inch.

One more myth to kill early: on a **screen**, the PPI tag is meaningless for display size. A 1500×1000 file looks identical whether it's tagged 72 PPI or 300 PPI — only the pixel dimensions matter on screen. The PPI tag only wakes up at print time, when it decides the physical size on paper.

The 300 PPI rule — and where it actually comes from

The industry standard for quality print viewed at normal reading distance (about 10–16 inches from your face) is **300 PPI**. The acceptable range people cite is 200–400 PPI; 300 is the safe default. But why exactly 300? There are two real reasons, not folklore.

Reason 1: your eyes

At a 12-inch reading distance, the human eye can resolve detail down to about one arcminute of angle, which works out to roughly 300 dots per inch. Below that, you'd see individual dots; above it, the extra detail is invisible to the naked eye. Printing a brochure at 600 PPI for reading-distance work is wasted data — you literally cannot see the difference.

Reason 2: the 2× halftone rule (the engineering reason)

Commercial presses build images out of halftone rows measured in **LPI** (Lines Per Inch). The rule of thumb: image resolution should be about **twice the line screen**.

- Magazine / commercial offset: $150 \text{ LPI} \times 2 = \mathbf{300 \text{ PPI}}$ — this is where "300" was born.
- High-end work: $175 \text{ LPI} \times 2 \approx 350 \text{ PPI}$.
- Newspaper: $\sim 85 \text{ LPI} \times 2 \approx 170 \text{ PPI}$ (coarse paper hides the rest).

Common mistake — Treating 300 PPI as a universal law. It's a *reading-distance* rule. A billboard viewed from 50 feet away can print at 20–50 PPI and look perfect, because your eye can't resolve fine detail from far away. Large-format banners commonly run 100–150 PPI. Spec'ing a 30-foot banner at 300 PPI creates a monstrous file for zero visible benefit.

Viewing distance	Sensible minimum PPI	Example product
Nose-to-print (~12 in)	~300 PPI	Business card, brochure, book
~1 meter	~180 PPI	Poster, retail sign
~2 meters	~100 PPI	Trade-show banner
Across a parking lot	20–50 PPI	Billboard

Effective resolution — the scaling trap that ruins prints

This is the most practically important idea in the whole section. **PPI is not fixed in a file.** It changes the instant you resize the image on the page. The number that matters is the **effective PPI**:

```
effective PPI = pixel dimension / printed size in inches
(do width and height separately)
```

Worked example — You have an 800×1200 pixel photo. Print it at 4×6 inches: $1200 \div 6 = 200$, and $800 \div 4 = 200 \rightarrow$ **200 PPI** (marginal). Now a 1500-pixel-wide logo: at 5 inches wide that's 300 PPI (great); at 10 inches it's 150 PPI (risky); at 20 inches it's 75 PPI (it fails). Same file, three completely different outcomes — only the print size changed.

So a "300 PPI" image dragged 200% bigger in your layout silently becomes 150 PPI. The file's tag still proudly says 300, but the effective resolution has halved. Professional preflight tools (Adobe Acrobat and InDesign's Live Preflight) check the *effective, scaled* PPI — not the file's stored tag — precisely because the tag lies after scaling.

Pro tip — To find the biggest size a photo can safely print at 300 PPI, divide its pixel dimensions by 300. A 3000×2000 px photo $\rightarrow$ max $\sim 10 \times 6.7$ inches. Beyond that, quality drops.

Why "72 PPI" web images print blurry

People grab an image off a website or Instagram, drop it into a print layout, and it looks terrible. The reason is *not* the "72 PPI" tag — remember, the tag means nothing on screen. The real problem is that web images have **few pixels** to begin with, kept small so pages load fast.

The math is brutal: a 300 PPI image holds about $17\times$ more pixel information than a 72 PPI image at the same physical size ($300^2 \div 72^2 \approx 17.4$). Stretch that thin web image across a print page and there simply aren't enough pixels per inch.

The killer misconception — "I just changed the number to 300 in Photoshop, so it's print-ready now." With *resampling off*, changing the PPI number adds **zero** new detail — it only reassigns the same pixels to a smaller physical area. With *resampling on* (upsampling), the software *invents* pixels by guessing — that produces bigger, blurrier pixels, not real sharpness. **You cannot manufacture detail that was never captured.**

A reliable warning sign: web PNG exports often carry no density metadata at all, or default to 72. That's a strong signal the file is web-sourced and too low for print.

Line art is a different animal — it needs much more

Line art (1-bit)

Pure black-and-white imagery with no gray tones at all — logos, text, barcodes, technical drawings. Stored as 1 bit per pixel (black or white only), so files are tiny.

Because line art has hard edges and no halftone dots to soften jaggies, it needs **far higher** resolution than photos: typically 600–1200 PPI, and 1200–2400 PPI for crisp logos, barcodes, or tiny text where jagged edges would be obvious. The 2×-LPI rule does *not* apply here — there's no line screen for pure line art. Scanned line art and screen-printing files are commonly 600–1200 DPI 1-bit TIFFs.

Pro tip — Better still, keep logos and text as **vector** (math-defined shapes) instead of rasterizing them to pixels. Vector is resolution-independent — it stays razor-sharp at any size. This is exactly why a designer should export a logo as vector, not a 300 PPI PNG.

How this lands in Print-Flow-360

PF360 already enforces these rules in real code, and understanding the theory tells you exactly where it's strong and where the gaps are.

The Node PDF service (sharp) and DPI metadata

The PDF service on port 4000 uses the `sharp` image library. When it resizes artwork (`imageProcessor.js`), it calls `.withMetadata({ density: dpi })` to stamp the output DPI tag, and `pdfGenerator.js` defaults to `dpi = 300`, converting millimeter trim/bleed sizes into pixels at that density. The crucial theory tie-in: `withMetadata` only writes the *tag* — it does not add pixels. Correct print sizing must compute *target pixels = inches × 300*, which the service's `buildDimensions` does properly from the product's physical size. There's also a telling fallback in `getMeta : dpi: meta.density || 72` — the 72 you now know to distrust.

The preflight engine

The file `pdf-service/src/services/preflight/preflightChecker.js` is PF360's automated quality gate for uploaded artwork. Its `checkDpi` function reads sharp's `meta.density` and applies thresholds

(`MIN_DPI_PASS = 300` , `MIN_DPI_WARN = 150`):

- **300+** → PASS: "suitable for professional print."
- **150–299** → WARN: "acceptable for small prints like business cards but may pixelate on larger formats."
- **Under 150** → FAIL: "too low for print — re-export at 300 DPI."
- **No density tag** → WARN: "usually exported for web (72 DPI) — re-export at 300 DPI."

Notice these are plain-language messages with a recovery action — exactly what a non-technical store owner needs ("re-export at 300 from Illustrator/Photoshop/Canva Pro"), never a raw "low res" error.

The documented gap — Around line 86 of `preflightChecker.js`, for PDF uploads the code admits "PDF resolution cannot be verified without rendering each page" and returns an info note rather than checking each placed image. This is the *effective-resolution* blind spot we just learned: it trusts the embedded density tag, which lies for scaled images and web PNGs. A real preflight (Acrobat / Ghent Workgroup standard) renders each page and measures the *scaled PPI* of every placed image.

Actionable rule to teach the platform — The strongest preflight check ignores the embedded tag entirely. Compute *effective PPI* = *pixel dimensions* ÷ *the ordered product's trim size* (the storefront already knows the product size), then FAIL under 300 and WARN under 150. This catches the customer who uploads a small logo for a large banner — the case the current tag-based check misses.

For context, the authoritative standard here is the **Ghent Workgroup (GWG)** prepss specs, built on **ISO PDF/X-4** (the modern print-exchange PDF format). GWG's test suite specifically checks "correct detection of minimum image resolution," and its Sign & Display variant formalizes the viewing-distance scaling we covered.

Key takeaways —

- **PPI** describes your *file*; **DPI** describes the *printer*. The "300 DPI rule" really means 300 PPI in the file.
- One pixel = one full color; one printer dot = one ink color. Printers use many dots (halftoning) to fake one continuous-tone pixel — so high printer DPI $\neq$ high image PPI.
- **300 PPI** is the canonical print resolution because it matches eye acuity at reading distance AND equals 2× the typical 150 LPI press. It is a reading-distance rule, not a law — billboards print at 20–50 PPI.
- **Effective PPI = pixels ÷ printed inches.** PPI changes the moment you scale an image; always check effective, scaled resolution, not the file's stored tag.
- 72 PPI web images print blurry because they have *few pixels*, not because of the tag. Changing the number to 300 adds no detail; upscaling invents blurry fake pixels.
- Line art needs 600–2400 PPI (hard edges, no halftone). Better: keep logos/text as resolution-independent vector.
- PF360's preflight checks the embedded density tag (300/150 thresholds) but skips per-image effective resolution inside PDFs — the right fix is to divide pixel dimensions by the ordered product size and flag low-res before production.

Section 9: Halftoning & Screening

Imagine you have a stamp and a single pot of black ink. You can press the stamp down (solid black) or not press it (bare white paper). That's it. Now I ask you to print a smooth, photographic gray sky that fades from light to dark. How do you make "50% gray" when you only have "ink" and "no ink"? You can't — not as a single tone. This is the central problem of every printing press, and the entire craft of **halftoning** exists to solve it.

Binary press

At any single point on the paper, a press (offset/litho, flexo, screen) lays down either a full dot of solid ink or nothing. There is no "half-strength" ink.

Halftoning (screening)

The trick of faking smooth, continuous tones by printing tiny dots of solid ink, varying their *size* or *number* so your eye blends them into shades of gray (or color).

Continuous tone (contone)

A real smooth gradient like a photograph, where any shade is possible. Your screen shows contone; a press cannot — it must simulate it.

Analogy — Stand far from a pointillist painting (Seurat) and the thousands of tiny colored dots melt into smooth tones and shapes. Step close and you see only separate dots. A halftone is the same illusion: your eye does the blending. This is called *spatial integration* by the human visual system.

How a halftone fakes gray: small dots vs big dots

The simplest, most common method varies **dot size** on a fixed grid. Light areas get tiny dots with lots of white paper showing. Dark areas get fat dots that nearly touch, so ink dominates. A "50% gray" is where dots cover roughly half the paper area.

AM clustered screening — dots GROW on a fixed grid by tone

Highlight ~10%	Midtone ~50%	Shadow ~90%
.	o o o o o	# @ # @ #
.	o o o o o	@ # @ # @
.	o o o o o	# @ # @ #
(more paper)	(half & half)	(mostly ink)

AM vs FM: the two ways to vary tone

AM (Amplitude Modulated) screening — also "conventional" or "clustered"

Dots sit on a fixed grid at fixed spacing; the **size** of each dot changes with tone. This is the classic halftone screen you've seen in old newspapers.

FM (Frequency Modulated) / stochastic screening

All dots are the same tiny fixed size, scattered in a random-looking pattern; the **number** of dots per area changes with tone. More dots = darker.

FM stochastic — dots stay tiny & fixed-size, COUNT varies

Highlight

Midtone

Shadow

FM spots are extremely small (often 10–20 microns) and behave like a very fine 250–480 LPI-equivalent screen. There's also **hybrid (XM)** screening: AM in the midtones, FM in the delicate highlights and shadows — aiming for the best of both.

The three letters everyone confuses: PPI, LPI, DPI

These sound alike and are mixed up constantly. Get them straight and you'll avoid the most common print mistakes.

Term	Stands for	What it measures	Lives where
PPI	Pixels per inch	Resolution of your digital image (the photo)	Input — what you set in Photoshop / the design studio
LPI	Lines per inch	Frequency of the halftone screen (rows of dots per inch — the "ruling")	The screen itself — a print choice, not a device
DPI	Dots per inch	Addressable resolution of the output device (platesetter/imagesetter)	The machine — many device dots build ONE halftone dot

Key distinction — PPI is your image. LPI is the screen. DPI is the machine. A 300 PPI photo, printed at 150 LPI, on a 2400 DPI platesetter are three different numbers describing three different things.

Why DPI must be much bigger than LPI

The device builds each halftone dot out of a little cluster of its own tiny device dots — like drawing one big dot inside a grid of graph-paper squares and filling in more squares to make it bigger. The more device dots per halftone cell, the more dot *sizes* (gray levels) you can make. The rule:

$$\text{gray levels} \approx (\text{DPI} / \text{LPI})^2 + 1$$

Worked example — At $2400 \text{ DPI} \div 150 \text{ LPI} = 16$, so $16^2 + 1 = 257$ gray levels — just enough to hit the 256-level maximum, giving smooth gradients. But at $600 \text{ DPI} \div 150 \text{ LPI} = 4$, you get only $4^2 + 1 = 17$ levels. A sky printed with 17 grays shows ugly visible steps — **banding** (also called posterization). *That's* why platesetters run at 2400+ DPI.

Typical LPI by process (worth memorizing)

Process / paper	Typical LPI
Newspaper (newsprint, uncoated)	65–100 (often ~85)
Magazine / commercial coated sheetfed	133–175 (150 is the workhorse)
High-end art books / coated gloss	175–200+
Flexo / screen print / packaging	50–150 (varies widely)

Higher LPI means finer detail and smoother tones — but it demands smoother, higher-quality paper. On rough or absorbent stock, tightly packed small dots spread into each other and the image turns muddy.

The 1.5×–2× rule: how much image PPI do you need?

This is the single most-asked number in prepress. Your image PPI should be about **1.5× to 2× the LPI** — a ratio called the **Quality Factor (QF)**.

- Standard advice: $\text{PPI} = 2 \times \text{LPI}$. So 150 LPI → **300 PPI**; 175 LPI → **350 PPI**.
- This comes from the **Nyquist–Shannon sampling theorem**: to faithfully reproduce a signal you need roughly twice its frequency in samples. 1.5× is the practical floor — below it, fine detail softens. Above 2× is wasted: a bigger file with no visible gain ("overscanning").
- At a different print size: $\text{required PPI} = \text{LPI} \times \text{QF} \times (\text{final size} \div \text{original size})$. Enlarging an image to 2× its size doubles the PPI you need from the source.

Common mistakes — (1) Sending 72 PPI web images to print — they look fine on screen but pixelate badly on paper. (2) Setting PPI to match DPI — e.g. making a 2400 PPI image because the platesetter is 2400 DPI (confusing image with device). (3) Over-scanning to 600+ PPI thinking "more is always better" — it just bloats the file.

Screen angles, moiré, and rosettes (the color part)

Color printing overlays four screens — Cyan, Magenta, Yellow, Black (CMYK). If two grids of dots line up at the same or nearly the same angle, they "beat" against each other and produce **moiré**: a coarse, shimmering, ugly interference pattern.

Moiré

An unwanted pattern that appears when two regular grids overlap at a bad angle — like the wavy shimmer you see through two stacked window screens.

Rosette

The tiny, tight, neutral flower-shaped pattern formed when the screens *are* correctly angled. This is the intended, benign texture of good 4-color AM printing.

The fix is to rotate each color's screen. The ideal separation is **30°**. But a square-dot screen repeats every 90°, and $90^\circ \div 3 = 30^\circ$ — so only *three* colors fit at the ideal spacing. Standard angles:

Ink	Angle	Why
Cyan	15°	C, K, M sit 30° apart — the strong, visible inks get the safe spots
Black	45°	
Magenta	75°	
Yellow	0° (=90°)	Forced into the bad spot, only 15° from cyan — but yellow is the lightest ink, so its residual moiré is nearly invisible

Some shops also run yellow at a slightly different frequency (~108%) to hide it further. Note there are *two* moiré sources: screen-to-screen (fixed by angles) and **subject moiré** — the image content itself (fine fabric, herringbone suits, window screens) beating against the halftone grid.

Pro tip — FM/stochastic screening eliminates moiré entirely: with no fixed grid and no angles, random dots simply can't beat against each other. It also gives a near-photographic look, holds color better under slight misregistration, widens the gamut, and uses ~10–15% less ink. The trade-off: tiny FM dots are fragile — they suffer heavy dot gain, are harder to hold on press, need stochastic-capable platemaking, and can look grainy in flat midtones.

Dot gain and dot shape

Dot gain

Ink dots spread when pressed into paper, so a 50% screen prints darker than 50% (often ~65%). Worse on uncoated/absorbent stock and at higher LPI (smaller dots have more edge to spread). Prepress compensates with a correction curve in the separation.

Dot shapes matter too: **round** is simple; **square** dots can abruptly connect at the midtone, causing a visible tone jump ("plugging"); **elliptical/chain** dots transition more smoothly through the midtones and are a common default.

Common mistake — Ignoring dot gain. Designers proof on a bright screen, approve it, then the printed piece comes back muddy and too dark because the midtones gained on press. Always soft-proof with the press profile (or pull a real proof).

How this maps to Print-Flow-360

Halftoning happens at the press/RIP stage, downstream of everything PF360 produces — but several platform realities connect directly:

- **Design studio (`designer/` , `Fabric.js`):** Fabric renders to a screen `<canvas>` in RGB at screen resolution. A 300 DPI print piece needs the exported raster to carry enough pixels for the `PPI = 2 × LPI` rule. If the studio exports at 72–96 PPI screen resolution, a customer's photo upload will halftone poorly when a print shop finally screens it. Surface the effective output PPI at the chosen print size so users aren't shipping under-resolution art.
- **PDF service (`pdf-service/` , `port 4000`, `sharp + PDFKit`):** `sharp` is the right place to enforce a minimum PPI and warn on under-resolution images during processing — it can read pixel dimensions and compute PPI against the target print size. This is the natural home for a future *preflight* check (resolution, plus the documented missing CMYK conversion), since PF360 currently has no RGB → CMYK step and no preflight.
- **The documented gaps are exactly the halftoning prerequisites:** no RGB → CMYK conversion means the four screens (and their angles) are never even prepared on-platform; no preflight means nothing catches a 72 PPI logo before it reaches a press; the weak order-to-production spine means there's no place to attach press intent (target LPI, paper, dot-gain profile) to an order. These are real, actionable gaps — fixable as preflight rules in the PDF service rather than new code in the studio.
- **Storefront brand-color theming** is purely on-screen RGB (CSS custom properties) — it is unrelated to print halftoning, and that's worth stating plainly so no one assumes the brand color "is" the printed ink.

Key takeaways —

- A press is binary — it prints solid ink or nothing. Halftoning fakes smooth tone with tiny dots the eye blends together.
- **AM** varies dot size on a fixed grid; **FM/stochastic** varies the *number* of fixed tiny dots; **hybrid (XM)** mixes both.
- **PPI** = your image, **LPI** = the screen ruling, **DPI** = the output device. Keep them straight.
- DPI must be far bigger than LPI: $\text{levels} \approx (\text{DPI}/\text{LPI})^2 + 1$. 2400/150 → 257 levels (smooth); 600/150 → 17 levels (banding).
- The **1.5×–2× rule** (Nyquist): set image $\text{PPI} = 2 \times \text{LPI}$ → 150 LPI needs 300 PPI, 175 LPI needs 350 PPI. Don't ship 72 PPI; don't over-scan past 2×.
- Typical LPI: newspaper 85, commercial 150, high-end 175–200+. Higher LPI needs smoother paper.
- CMYK screen angles (C 15°, K 45°, M 75°, Y 0°) are 30° apart to avoid moiré; correct angling yields a benign **rosette**. Yellow takes the bad angle because it's the lightest ink. FM screening sidesteps moiré entirely.
- **Dot gain** makes printed tones darker than specified — compensate, or output looks muddy.
- In PF360, enforce export PPI in the design studio and add resolution/CMYK *preflight* in the **pdf-service** (sharp) — that's where the documented print-readiness gaps should be closed.

Section 10: Bleed, Trim, Safe Zone & Marks

Every printed thing you've ever held — a business card, a flyer, a book cover, a cereal box — started life as a digital file that had to be printed onto a big sheet and then *cut down* to its final size. That last step, cutting, is where beginners get burned. This section teaches the page geometry that makes the cut come out clean: the boxes that define where the page is, where the blade lands, and where your important content is safe. Get this right and your prints look professional. Get it wrong and you ship cards with ugly white slivers down one edge.

Why this exists: cutting is never perfect

Here's the core truth that everything in this section flows from: **a paper cutter is a mechanical blade slicing through a tall stack of sheets, and it does not land in exactly the same spot every time.** The blade can drift a fraction of a millimeter. Paper shifts under the clamp's pressure. In a stack of 500 sheets, the bottom ones creep slightly. The normal, accepted variance in where the cut actually lands is roughly **±1 mm** (about ±0.0625 inch). This is called the *cut tolerance* — it's not a defect, it's physics. The entire system of bleed and safe zones exists to make your design survive that wobble.

Analogy — Think of wrapping a gift. If you cut the wrapping paper to the *exact* size of the box, the tiniest misalignment leaves bare cardboard showing at a corner. So you cut the paper *bigger* than the box (that extra is your "bleed") and you keep the gift tag away from the edges so it doesn't get folded under (that's your "safe zone"). Print works the same way.

The four nested boxes

A print file is defined by four rectangles, one inside the next. From outermost to innermost:

```
+-----+
| filename.pdf v3 2026-06-21 [color bars] | <- Media / Slug
|                                           | (marks & notes)
+-----+
| | BLEED (art extends out to here) | | <- Bleed box (+3mm)
| | +-----+ | |
| | | TRIM (cut line) | | <- Trim box (final size)
| | | +-----+ | |
| | | | SAFE ZONE | | <- Safe box (-3mm)
| | | | keep text inside | |
| | | +-----+ | |
| | | +-----+ | |
| | +-----+ | |
+-----+
|__ crop marks sit just outside the trim corners
```

Trim box (trim line / cut line / cut size)

The finished size of the page after cutting — the dimensions of the thing the customer actually holds. This is the **single source of truth**; every other box is measured from it. A standard business card's trim box is 3.5 × 2 inches (or 85 × 55 mm).

Bleed (bleed box)

The area where you extend backgrounds, colors, and photos *past* the trim line, so that when the blade drifts there's still printed art — not bare paper — at the cut. Standard bleed is **0.125 inch (1/8 inch) = 3 mm** on every side.

Safe zone (safety / live area / margin)

An inner buffer, set *in* from the trim line, where all important content must stay. Standard inset is **3 mm (0.125 in)**; cautious printers use 5 mm (0.25 in). Keep text, logos, phone numbers, and QR codes inside this line.

Slug / Media box

The outermost area, beyond the bleed, that holds printer's marks and human notes (job name, version, date, instructions). In PDF terms this is the *MediaBox*, the largest box that contains everything.

Notice the elegant symmetry: **bleed and safe zone are mirror images of the same trim line**. Bleed extends 3 mm *outward* to survive a cut that lands too far out; safe zone sits 3 mm *inward* to survive a cut

that lands too far in. Together, 3 mm of bleed plus 3 mm of safe margin comfortably absorb the normal ± 1 mm cut tolerance on both sides.

Key Takeaway — Trim is where the blade *aims*. Bleed protects you when it lands a hair outside. Safe zone protects you when it lands a hair inside. You design assuming the blade will miss — because it will.

Why files without bleed get white edges

This is the number-one rookie mistake, so let's make it vivid. Suppose you design a flyer with a solid blue background sized to *exactly* the trim — the blue stops precisely at the cut line. The press prints it on a larger sheet and the cutter aims for the trim line but lands 0.8 mm to the right (well within tolerance). On the left edge, the blade now cuts *inside* the blue, into the unprinted paper. Result: a thin **white sliver** running down the left side of every flyer. The job is ruined and a non-technical store owner has no idea why.

With 3 mm of bleed, that same 0.8 mm drift simply trims off a little of the excess blue. The customer never sees the difference. **Bleed gives the blade a margin of error so what's lost is throwaway art, not bare paper.**

Common Mistake — The sneakiest bleed error isn't "no bleed" — it's setting a bleed box but *forgetting to extend the artwork into it*. The file looks correct (it has a bleed setting!) but the background still stops at trim, so you still get white edges. A bleed setting with no art filling it is worse than useless: it gives false confidence. This is exactly the kind of silent failure a real preflight check must catch.

Printer's marks: the symbols in the margins

Marks live in the slug area, *outside* the trim, so they're cut away and never appear on the final product. They exist to guide the press operator and the cutter.

Mark	What it looks like	What it's for
Crop marks (= trim marks)	Short lines at the 4 corners	Show the cutter exactly where to slice. Offset outside the bleed so the marks themselves aren't cut into the page.
Bleed marks	Short lines marking the outer bleed edge	Indicate how far the art should extend.
Registration marks	Crosshair / bullseye target	Printed on <i>all four</i> CMYK plates. The operator aligns the plates so the targets stack perfectly. Misalignment = visible color fringing/ghosting ("mis-registration").
Color bars (control strip)	Row of solid & tint swatches of C, M, Y, K	The operator reads them with a densitometer to confirm ink density and coverage are correct across the sheet.
Page information	Tiny text in the margin	Filename, date, plate/separation name — so a sheet is traceable to its job.

CMYK

The four printing inks — Cyan, Magenta, Yellow, black — layered to make full-color images. Each ink is printed from its own "plate," which is why they must be aligned (registered).

How this maps to PDF page boxes

When you export a print-ready PDF, those boxes become real metadata the print partner's software reads. The hierarchy is **MediaBox** $\geq$ **BleedBox** $\geq$ **TrimBox/ArtBox**. The professional print standard, **PDF/X**, requires these be set correctly: PDF/X-1a and PDF/X-3 require MediaBox + TrimBox + BleedBox; the ArtBox/TrimBox must never exceed the BleedBox. Imposition software (which arranges many pages onto one big sheet) positions every page by its **TrimBox** — so if your TrimBox is wrong, everything downstream is wrong.

Real-world case — The **Ghent Workgroup (gwg.org)** publishes the industry's authoritative preflight specifications — 14 certification variants built on PDF/X-4, with 260 test files that check minimum image resolution, color spaces, ink coverage, overprint, and *bleed presence*. When a serious print broker says "send us a GWG-compliant PDF/X-4," this geometry is a big part of what they're verifying.

Tying it to Print-Flow-360

This is where the theory becomes actionable for our platform. PF360 has a Fabric.js **design studio** ([designer/](#)) where customers create artwork, and a Node **PDF service** ([pdf-service/](#) , port 4000, using sharp for images and PDFKit for PDFs) that produces files. Today the platform has documented

gaps: **no RGB → CMYK conversion, no preflight, and a weak order-to-production spine.**

Bleed/trim/safe is squarely in that gap, and here's how it should be addressed.

1. The canvas must show three guides — in plain language

The Fabric.js canvas should render three distinct guide lines over the artboard: the **cut line** (trim), the **bleed line** (outer, dashed), and the **safe-zone line** (inner, dashed). Critically — our users are non-technical store owners who have never heard "bleed." Label the guides the way a shopkeeper would understand:

- Bleed line → *"Fill your background all the way to this edge"*
- Cut line → *"This is where we'll cut"*
- Safe line → *"Keep text and logos inside this line"*

Pro Tip — Default the bleed to **3 mm** and the safe inset to **3 mm** automatically for every product, so 90% of customers never configure anything. The best setting is the one nobody has to think about. This matches the platform principle of sensible defaults over configuration.

2. The designer should enforce, not just display

Showing guides isn't enough — the studio should actively help users stay safe:

- **Encourage fill to bleed:** when a background image or color rectangle is added, snap/stretch it to the bleed boundary, not the trim.
- **Warn on safe-zone crossings:** when a text or logo object's bounding box crosses the safe line, flag it live ("This text may get cut off — move it inward").
- **Protect against the cut zone:** ideally lock the layout so customers can't drop important content into the bleed/cut risk area in the first place.

3. Preflight must verify bleed actually exists

This is the deepest fix and the one that closes the silent-failure gap. When the PDF service generates output, a preflight step must:

- **Confirm artwork actually fills the bleed box** — not merely that a bleed *setting* exists. Using sharp, inspect the outer 3 mm ring of the rendered raster: if it's bare/white where it should be art, flag or reject the file before it reaches production. This catches the "bleed set but art not extended" bug that fools everyone.
- **Flag elements crossing the safe line.**
- **Write correct TrimBox + BleedBox metadata** (3 mm) into the exported PDF via PDFKit, so the print partner's imposition software reads the right geometry.
- **Optionally add crop marks, registration marks, and color bars** in the slug for offset-press partners.

Pitfall — borders near the edge. Warn customers away from thin decorative borders that run close to the cut. Because of the ± 1 mm cut tolerance, a border 2 mm from the edge will be 3 mm thick on one side and 1 mm on the other — it looks crooked even though the cut was perfectly "in spec." Borders are inherently risky in print; the safe move is a thick border or none.

Key takeaways —

- Four nested boxes, outer → inner: **Media/Slug > Bleed > Trim > Safe**. Trim is the source of truth; bleed extends out, safe sits in.
- Cutting has a normal ± 1 mm tolerance. **3 mm bleed + 3 mm safe** absorb it on both sides.
- No bleed (or bleed set but art not extended into it) → **white slivers** after trimming. The second case is the most common silent error.
- Keep all critical content inside the safe zone; never put important content in the bleed (it gets cut away).
- Printer's marks (crop, bleed, registration, color bars, page info) live in the slug, outside trim, and are cut off — they guide the operator and cutter.
- Pro PDFs (PDF/X) require correct **MediaBox $\geq$ BleedBox $\geq$ TrimBox**; GWG is the authority for preflight specs that include checking bleed.
- For PF360: the Fabric.js studio must show three plainly-labeled guides and default bleed/safe to 3 mm; the Node PDF service must *verify bleed art actually exists* (inspect the outer ring with sharp), write correct TrimBox/BleedBox via PDFKit, and is the natural home for the missing preflight step.

Section 11: Preflight — Validating a Print-Ready File

Imagine you hand a print shop a beautiful business-card design. The press operator loads the paper, makes the metal plates, and runs 5,000 cards. Then someone notices the text is fuzzy, the black looks washed-out, and there's a white sliver along one edge. The plates are scrap. The paper is wasted. The job is late. All of this was preventable — the file was broken before it ever reached the press, and nobody checked.

Preflight is the check that catches all of this *before* production starts.

Preflight

Automated quality-control of a print file before it goes into production. The software inspects the file against a set of rules and produces a pass/fail report listing every problem.

Analogy — The name comes from aviation. Before a plane takes off, the pilot runs a printed pre-flight checklist — fuel, flaps, instruments — because finding a problem on the ground is cheap and finding it in the air is catastrophic. Print preflight is the same idea: checking on the ground (the computer) instead of in the air (the press).

The two phases: Check vs. Fixup

Preflight has two distinct jobs, and confusing them causes real damage:

Preflight check (report)

Inspect the file against a profile and report what's wrong. Changes nothing. Output is a pass/fail report.

Fixup

Automatically correct a problem — convert RGB to CMYK, embed a font, add bleed, set black to overprint, flatten transparency.

Common Mistake — Auto-fixing everything. A fixup silently changes the file, and a "smart" fix can ruin the designer's intent (e.g. auto-converting a brand color and shifting it). Industry best practice: **fix only safe, deterministic things; report everything else for a human to decide.**

The standard preflight checklist (with real thresholds)

These are the items virtually every commercial print preflight checks. Learn the numbers — they're not arbitrary.

1. Color space (RGB vs. CMYK)

RGB

Red-Green-Blue — how screens make color with light. Huge, vivid range.

CMYK

Cyan-Magenta-Yellow-Key(black) — how presses make color with ink. Smaller range.

Process print needs CMYK or named spot colors. RGB left in a file converts unpredictably and prints "muddy." Convert using a destination **ICC profile** (a file describing a specific press's color behavior): North America uses GRACoL2013 or SWOP (coated stock); Europe uses FOGRA39 / FOGRA51 (PSO Coated).

2. Image resolution

PPI / DPI

Pixels (or dots) per inch — how much detail an image holds at its printed size.

Effective resolution

The resolution *after* scaling. A 300 PPI image enlarged to 200% becomes 150 PPI effective — half as sharp.

Standard: **300 PPI at final size** for color/grayscale photos; flag below ~250 PPI. Line art (1-bit) wants 600–1200 DPI. Large-format signage tolerates 72–150 PPI because you view it from far away. A 72 PPI web image is screen-only and fails.

3. Fonts embedded

Every font must be **embedded** (stored inside the PDF) or the text converted to **outlines** (turned into vector shapes). Otherwise the press substitutes a default font like Arial — and your layout reflows and breaks. A font with a "No Embedding" permission flag = automatic fail.

4. Bleed and safe zone

Bleed

Artwork extended **0.125 in (3 mm)** past every trim edge, so a slightly off cut never reveals white paper.

Safe / quiet zone

Keep text and logos 3–5 mm *inside* the trim line so they aren't sliced off.

```
+-----+ <- BleedBox (trim + 3mm)
| . . . . . |
| . +-----+ . | <- TrimBox (final cut size)
| . | +-----+ | . |
| . | | SAFE ZONE | | . | keep text here
| . | +-----+ | . |
| . +-----+ . |
| . . . . . |
+-----+ bleed art fills to here
```

5. Ink limit / TAC

TAC (Total Area Coverage)

Also called TIC. The sum of C+M+Y+K ink percentages at any single point. 100% of each = 400% maximum theoretically.

Too much ink doesn't dry — it smears and offsets onto the next sheet. Limits by process:

Process	Max TAC
Sheetfed offset, coated	300–340% (SWOP = 300%)
Heatset web (magazines)	300–320%
Non-heatset / uncoated / newsprint	240–260% (newsprint as low as 220%)
Gravure	~360%
Inkjet / digital	~300–350%

6. Rich black and black text

Rich black = a deep black built from multiple inks, e.g. C60 M40 Y40 K100 (240% — neutral, safe). Avoid "registration black" C100 M100 Y100 K100 (400% — over every limit). But small black text must be **pure K only** (C0 M0 Y0 K100): 4-color text gets a colored fringe if the plates misalign.

7. Overprint

Overprint

Printing an object on top of what's beneath it instead of knocking a hole out of the lower layers.

Small black text should overprint — so a slight registration shift won't leave a white halo around it. But **overprinting a white object makes it vanish** entirely — a classic silent error. Acrobat's "Simulate Overprinting" in Output Preview reveals both.

8. Spot colors, transparency, hairlines, geometry

- **Spot colors:** a stray "PANTONE 185 C" in a 4-color job becomes an unexpected 5th plate. Flag and convert unintended spots.
- **Transparency:** live transparency (soft shadows, see-through layers) is only valid in PDF/X-4 and later; older workflows need it **flattened**, which can leave white-box artifacts.
- **Hairlines:** a 0.25 pt or "0 pt" line may render as a single device pixel — sometimes invisible. Flag strokes thinner than ~0.25 pt.
- **Page geometry:** the TrimBox must equal the ordered product size, with correct orientation and page count.

The standards: PDF/X and the Ghent Workgroup

PDF/X (ISO 15930)

The print-ready PDF standard preflight validates against. All variants require embedded fonts, a declared output intent (target ICC), defined bounding boxes, and no encryption.

Flavor	What it allows
PDF/X-1a	CMYK + spot only. No RGB. No live transparency. Safest for legacy RIPs.
PDF/X-3	Adds color-managed RGB/Lab with ICC profiles.
PDF/X-4	Modern default — live transparency + layers + managed RGB.
PDF/X-6	Newest (PDF 2.0 era).

The **Ghent Workgroup (GWG, gwg.org)** publishes industry-consensus profiles built on top of PDF/X — per market (Sheetfed/Web offset, Magazine Ads, Packaging, Digital, Sign & Display). GWG CMYK profiles warn on images below ~250 PPI, require 3 mm bleed, and a 0.25 pt minimum line. GWG 2022's key goal was cutting **false positives** — errors reported that never actually cause press problems — and it introduced a universal "Proof of Preflight" report.

How this maps onto Print-Flow-360

PF360 has **no preflight at all** — it's the documented missing validation layer of the "order-to-production spine" (see [PLATFORM_GAP_ASSESSMENT](#) and [REAL_USER_QA_READINESS](#)). Today a customer's artwork — whether exported from the Fabric.js design studio in [designer/](#) or uploaded directly — flows into an order with zero quality checks. There's also no RGB → CMYK conversion. And the store owners reading the result are non-technical: they cannot interpret a prepress report full of jargon.

Pro Tip — Build preflight in the Node [pdf-service](#) (port 4000), which already has [sharp](#) for image inspection and [PDFKit](#) for PDFs. Gate it behind a feature flag per the project rules, run it deterministically on artwork submit, and snapshot the result onto the order for dispute resolution.

Real-world case — A proposed PF360 ruleset, producing a plain-language **traffic-light report** (Green pass / Amber warn / Red block) plus a thumbnail with problem areas circled:

1. Page size == ordered product size.
2. Bleed ≥ 3 mm present (auto-add if safe, else warn).
3. Effective resolution: block under 150 PPI, warn 150–300, ideal 300+.
4. Color space CMYK (auto-convert RGB via GRACoL/FOGRA, and *report the change*).
5. Fonts embedded or outlined (auto-outline from the designer export).
6. TAC $\leq 300\%$ coated / 240% uncoated (warn over).
7. Small black text = pure K + overprint.
8. Minimum line 0.25 pt.
9. Flag stray spot colors against the product's plate count.
10. Text inside the safe zone.

The message a customer sees is never `RESOLUTION_BELOW_THRESHOLD: 96ppi` — it's *"Your photo will look blurry when printed. Please upload a larger image."* with a button to re-upload.

Key takeaways —

- Preflight is automated QC of a print file before production — it catches errors while they're cheap to fix.
- Separate **check** (report only) from **fixup** (auto-correct); only auto-fix safe, deterministic items and report the rest for a human.
- The core checklist: CMYK color, 300 PPI effective resolution, embedded fonts, 3 mm bleed + safe zone, TAC under 300% (240% uncoated), rich black builds, pure-K overprinting text, no stray spots, flattened/legal transparency, ≥ 0.25 pt lines, correct page size.
- **PDF/X (ISO 15930)** is the print-ready target; **GWG** profiles refine it per market and fight false positives.
- PF360 has no preflight today — build a flag-gated, deterministic ruleset in the Node `pdf-service` (sharp + PDFKit) that returns a plain-language traffic-light report, never raw codes, and snapshot the result on the order.

Section 12: Trapping

When a printing press lays down color, it does not paint all the colors at once. Each ink — say cyan, then magenta, then a special blue — is applied by its own separate *plate* (a metal or rubber carrier that holds one ink) in its own separate pass or unit. The paper, and those plates, never line up perfectly. They shift by a fraction of a hair from one ink to the next. **Trapping** is the prepress trick that hides those tiny shifts so your final print does not show ugly white slivers where two colors meet.

Trapping

Deliberately making two adjacent colors overlap by a tiny amount, so that if the press misaligns the inks slightly, no bare paper shows at the seam between them.

Misregistration

When the ink layers do not line up exactly — the plates or paper shifted between passes. "Registration" means the inks are in perfect alignment; *misregistration* means they are not.

Plate / ink unit

The part of the press that applies one single color. Four-color print uses four plates (cyan, magenta, yellow, black); a job with spot colors adds one plate per spot color.

Why trapping exists: the white-gap problem

Imagine two flat blocks of color sitting edge to edge — a blue shape next to a yellow shape. To print this, the press normally **knocks out** the yellow: it cuts a yellow-shaped hole in the blue so the two inks do not mix and muddy each other. The blue plate prints around the hole; the yellow plate fills the hole. If both plates land in exactly the right spot, the edges meet perfectly. But they never do. The paper stretches, a plate is a thread off, a web press whips the paper through at speed. The result is a thin gap where neither ink printed — and the bare white paper peeks through.

Analogy — Think of a child coloring a cartoon. They draw the outline of a balloon and fill it in, but leave a thin unpainted ring just inside the outline. White paper shows through that ring. Trapping is like telling the child: "bleed your fill a tiny bit *past* the outline." Now even if your hand wobbles, no bare paper ever shows. Or picture hanging wallpaper: you overlap the seams slightly so the wall behind never peeks out, no matter how the paper settles.

NO TRAP — press shifts, white gap appears:

BLUE | | YELLOW (the "| |" = bare paper)

WITH TRAP — colors overlap, no gap can show:

BLUE [##] YELLOW ("[##]" = shared overlap zone)

Misregistration is worst on **offset**, **flexo** (flexible-plate printing, common for packaging and labels), and **screen** printing, and on stretchy stock like newsprint or thin film. It is least on single-pass digital/toner presses, because those lay all colors in one go with no plate-to-plate shift.

Spread vs. choke: two ways to make the overlap

There are only two ways to create that overlap zone, and they differ only in *which color moves*.

Spread ("fattie")

The foreground object is grown slightly so it bleeds *outward* over the background. The object gets fatter.

Choke ("skinny")

The foreground object stays put, but the background grows slightly *inward* over it. The object effectively shrinks.

Both produce the identical result: a thin shared band where the two inks overlap. The only question is which color grows into the other.

The decision rule: the lighter color always spreads into the darker one

This is the single most important rule in trapping, so memorize it:

Key rule — The **lighter** color always spreads into the **darker** color. The decision is based on *neutral density* (how light or dark a color is to the human eye). The lighter color is the one that grows.

Why? Because the **darker color defines the edge your eye sees**. Your eye reads the dark color as "the shape." If you move (fatten) the lighter color, the shape's perceived outline stays crisp and the overlap hides invisibly under the dark edge. If you moved the dark color instead, you would visibly distort and fatten the shape.

Worked example — Light-yellow text on a dark-blue background. The blue defines the letters' outline to your eye. So you *spread the yellow outward*. Even if the press shifts, the yellow now reaches under where the blue edge sits — no white gap, and the letters still look sharp. If you instead choked (spread the blue inward into the yellow), you would visibly fatten and smudge the letterforms.

How wide is a trap?

A trap is tiny — measured in fractions of a point. At 150 lpi (lines per inch, a common print resolution):

Measure	Typical trap width
Inches	1/300" to 1/150"
Points	0.24 pt to 0.48 pt
Millimeters	0.08 mm to 0.16 mm

A handy rule of thumb is "about half a dot" — roughly 1/300" (0.08 mm) at 150 lpi. **Black and dark traps can be 1.5× to 2× wider**, because dark ink masks any misalignment well and a wider trap simply won't be noticed. Trap width should scale with press quality: a tight, well-registered press needs a smaller trap; a sloppy press or stretchy stock (flexo, newsprint) needs a larger one. Too wide and the trap becomes a visible dark or colored outline; too narrow and gaps still slip through.

When you need trapping — and when you must NOT

Trap it	Do NOT trap it
Spot colors (Pantone/PMS) adjacent to anything — each is its own plate with zero shared ink, so any shift = white gap. Always trap.	Continuous-tone photographs (CMYK images) — soft edges and shared CMY inks already overlap. Trapping adds visible color fringes and degrades the image.
Sharp, hard edges between two solid flat colors — logos, vector art, text on a colored background, packaging. The classic trapping cases.	Two process colors that share enough common ink (e.g. both contain a lot of cyan) — the shared plate already bridges the gap. Little or no trap needed.
Spot color touching process color.	High-quality presses with tight registration , and most digital/toner/inkjet — trap artifacts can be worse than the rare misalignment.

Pro tip — Counter-intuitively, 4-color (process) art is often *easier* on trapping than 2-spot-color art, because the four process inks frequently share ink across the seam and bridge gaps on their own. Pure spot-to-spot is the hardest because the two inks share nothing.

The simpler alternative for black: overprint vs. knockout

Knockout (the default)

The top color punches a hole in the layer beneath so the inks don't mix. This is exactly where gaps appear — so knockouts are what need trapping.

Overprint

The top color prints directly *on top of* the background with no hole cut beneath it. No knockout, so no gap is possible — and no spread/choke math is needed.

Black text and thin black lines are routinely set to **overprint**. Black ink is opaque enough to cover whatever sits under it, so a shift can never reveal white. This is the everyday solution for small black type.

KNOCKOUT edge (gap-prone):

___YELLOW___ | GAP | ___BLUE___

OVERPRINT black (no gap possible):

___YELLOW___ [BLACK on top] ___BLUE___

Common mistake — Overprinting *light* colors causes color shifts (the background bleeds through and changes them). Worst of all, **overprinting white makes the white disappear entirely** — white "ink" with overprint on means nothing prints there. A classic, expensive disaster. Overprint is safe for black, dangerous for everything light. Note: "rich black" (black plus a CMY underlay for a deeper black) follows normal black *trapping* rules, not the simple overprint shortcut.

Automatic / in-RIP trapping

Nobody traps by hand anymore on real jobs. Trapping is computed automatically by software, either in the prepress/PDF workflow or — most commonly — at the **RIP**.

RIP (Raster Image Processor)

The engine that converts your page into the dot pattern the press actually prints. "In-RIP trapping" means the traps are calculated at this final raster stage.

Trap preset

A named bundle of trap settings (default trap width, black width, image-trapping on/off, thresholds) you can reuse across jobs.

The **Adobe In-RIP Trapping** engine detects contrasting color edges, reads the neutral density of the two colors, and automatically expands the lighter into the darker — even trapping one object against several different backgrounds at once. To use it you need a PostScript Level 2+ device with a RIP that supports Adobe In-RIP Trapping; in the print dialog you set Color to "In-RIP Separations" and Trapping to "Adobe In-RIP." Always confirm your print provider's RIP supports it.

Big gotcha — In Acrobat Pro, the "Trap Presets" feature does *not actually create traps in the PDF*. It only stores instructions that a compatible RIP applies later. If your printer's RIP ignores them, nothing is trapped. And if the designer manually traps *and* the RIP traps, you get a **double trap** — visibly fat, ugly seams. Coordinate with your printer on who does the trapping; do it once.

Where this lands in Print-Flow-360

Trapping is a press-side, separated-color concern — and Print-Flow-360 today works in **RGB only**, which keeps trapping out of the platform's direct scope but makes it a documented gap you should understand:

- The **Fabric.js design studio** ([designer/](#)) produces RGB vector and raster artwork. It has no concept of spot colors, overprint, knockout, or trap zones — a customer's two flat shapes are just RGB fills with no trapping intent attached.
- The **Node PDF service** ([pdf-service/](#) , port 4000) uses **sharp** for images and PDFKit for PDFs. These generate RGB output; PDFKit does not author trap presets or color separations. So any trapping must happen *downstream*, at the print provider's RIP — not here.
- The platform has **no RGB → CMYK conversion and no preflight** step. That means there is no stage that would separate colors into plates or flag "these two adjacent spot colors will gap." Combined with the **weak order-to-production spine**, trapping responsibility falls entirely on whichever press/RIP fulfills the order.

Actionable for PF-360 — Until CMYK separation and preflight exist, the safe, honest position is: artwork leaves the platform as RGB, and *trapping is the print provider's RIP responsibility*. If you later build a production handoff, the right place to add trap-preset metadata is at the PDF export/handoff layer (so the RIP knows what to do) — never hand-bake traps into the designer's RGB canvas, where they'd be wrong for any other output device.

Key takeaways —

- Trapping = a deliberate tiny overlap between adjacent colors so press misregistration never shows a white paper gap.
- **Spread** grows the foreground outward; **choke** grows the background inward — same overlap, different color moves.
- **The lighter color always spreads into the darker one**, because the dark color defines the edge your eye sees.
- Trap width is tiny: ~0.24–0.48 pt (0.08–0.16 mm) at 150 lpi; black traps run 1.5–2× wider.
- **Trap**: adjacent spot colors, and sharp edges between solid flat colors (logos, text, packaging). **Don't trap**: photographs, process colors sharing ink, tight digital presses.
- **Overprint black** instead of trapping it; never overprint white (it vanishes).
- Modern shops trap automatically **in the RIP** — coordinate so you don't double-trap.
- In Print-Flow-360, the RGB-only designer and PDF service can't trap; with no CMYK/preflight, trapping is the print provider's RIP job downstream.

Section 13: Raster vs Vector & Rasterization

Every image a computer can show or a press can print is stored in one of two fundamentally different ways. Understanding this split is the single most useful piece of foundational knowledge in print and graphics — it explains why a logo looks razor-sharp on a billboard but a photo turns into a blurry mess when enlarged, why some files are tiny and others huge, and exactly what the Print-Flow-360 design studio is doing every time a customer drags a shape. Let's build it up from zero.

The two ways to store a picture

Raster image (also called a bitmap)

A picture stored as a grid of tiny colored squares called pixels. Like a mosaic — the image *is* the arrangement of colored tiles.

Vector image

A picture stored as *math*: points, lines, and curves described by coordinates and equations. No pixels are saved — only instructions for how to draw the shapes.

Pixel

Short for "picture element." The smallest single dot of color in a raster image.

Analogy — A raster image is a *mosaic*: thousands of fixed colored tiles. To make it bigger you'd have to stretch each tile, and the picture gets chunky. A vector image is a *recipe*: "draw a circle here, radius 5cm, fill red." You can bake that circle the size of a coin or the size of a stadium — the recipe never changes and is always perfect.

Raster: pixels and resolution

A raster image's quality is fixed at creation. It depends on two things: the total number of pixels, and how densely they're packed when printed.

PPI (pixels per inch)

How many pixels of the digital image get packed into one inch when printed. This is the value you set or check in Photoshop. Higher PPI = more detail per inch.

Resolution-dependent

The defining weakness of raster: it has a fixed amount of detail. Enlarge it past its native size and you get pixelation — the visible "staircase" of chunky squares — or blur.

Critically, **you cannot add detail that was never captured**. "Upscaling" a small image asks the software to invent pixels by guessing the colors in between (interpolation), which produces soft, mushy results — never real sharpness. A 200×200 logo cannot become a crisp 2000×2000 logo.

Raster is the *right* choice for photographs, continuous-tone images, and complex gradients — anything where the color varies subtly from pixel to pixel. Cameras, scanners, and screens all produce rasters. Common formats:

Format	Best for	Notes
JPEG	Photos	Lossy (discards data to shrink size)
PNG	Web graphics, logos with transparency	Lossless + alpha (transparency)
TIFF	Prepress / print	Lossless, large files
PSD	Photoshop working files	Layered

Common mistake — A customer uploads a 72 PPI logo grabbed from a website. On their monitor (screens show ~72–96 PPI) it looks perfect, so they assume it's fine for print. It isn't — it simply lacks the pixels print needs and will print blurry. This is the #1 prepress complaint, and it's exactly the kind of issue a *preflight* check would catch (PF360 currently has no automated preflight — a documented gap).

Vector: shapes made of math

A vector image stores coordinates and equations, not pixels. Because it's a description rather than a fixed grid, it is **resolution-independent**: scale it infinitely with zero quality loss, and the file size stays small and constant no matter how big you display it. Vector art has *no PPI requirement* — it prints sharp at any output resolution.

That's why vector is the right choice for logos, icons, line art, typography (fonts are vectors!), and illustrations — anything that needs to work from a business card to a billboard. Common formats: **SVG**, **AI** (Illustrator), **EPS**, and **PDF** (a PDF can hold *both* vector and raster — which is why a print-final file mixes vector type and logos layered over raster photos).

Bézier curves and paths — the heart of vector editing

This is exactly what the Fabric.js design studio in **designer/** manipulates. A vector shape is a **path**.

Path

A sequence of **anchor points** joined by straight or curved segments.

Anchor point (node)

A fixed point the curve *must* pass through. The "pins" of the shape.

Control handle (control point)

A lever attached to an anchor that steers the curve. The handle's *angle* sets the curve's direction; its *length* sets how hard the curve is pulled that way. Control points do *not* sit on the curve itself.

Bézier curve

A smooth curve defined by anchors and handles. Named after Pierre Bézier (Renault, 1960s) who used it for car bodywork; the underlying algorithm came from Paul de Casteljau (Citroën, 1959).

Curves come in degrees by how many points define them:

- **Linear** — 2 points, a straight line. $B(t) = (1-t)P_0 + tP_1$
- **Quadratic** — 3 points (1 shared control). Used by TrueType fonts. $B(t) = (1-t)^2P_0 + 2(1-t)t \cdot P_1 + t^2P_2$
- **Cubic** — 4 points: 2 anchors (P_0, P_3) + 2 handles (P_1, P_2). The industry standard — used by Illustrator, the SVG `<path>` "C" command, PostScript Type 1 fonts, and PDF. $B(t) = (1-t)^3P_0 + 3(1-t)^2t \cdot P_1 + 3(1-t)t^2 \cdot P_2 + t^3P_3$, for $0 \leq t \leq 1$.

Here, t is a slider from 0 to 1 that walks along the curve (0 = start, 1 = end). Tools evaluate these stably via De Casteljau's algorithm (repeated linear interpolation). Here's a cubic Bézier:



Curve bows from P_0 toward P_1/P_2 and lands on P_3 .
Dashed lines $P_0 \rightarrow P_1$ and $P_3 \rightarrow P_2$ are the control handles (tangents) -- they steer but aren't drawn.

Anchors come in two flavors: **smooth** (handles collinear and equal length → seamless curve through the point) and **corner/cusp** (handles independent → a sharp angle). A typeface-design rule worth internalizing: use the *fewest* anchors with good handles. Over-adding points makes lumpy, hard-to-edit curves.

Fills and strokes

Fill

The interior color, gradient, or pattern of a closed path.

Stroke

The outline drawn *along* the path — with its own width, color, dashes, end caps, and corner joins.

A shape can have fill only, stroke only, or both. In Fabric.js these are object properties (`fill` , `stroke` , `strokeWidth`) on path and shape objects. The PF360 designer's whole job is letting customers manipulate anchor coordinates, handles, fills, and strokes — and the storefront brand-color theming feeds a store's primary color into these defaults so designs match the brand.

Rasterization: turning vectors into pixels

Rasterization (scan conversion)

Converting vector geometry into a pixel grid. Required because screens — and print devices via a RIP — are ultimately dot grids.

Aliasing ("jaggies")

The stair-step edges on diagonals and curves caused by forcing smooth geometry onto a finite grid (undersampling).

Anti-aliasing

Smoothing jaggies by giving edge pixels intermediate blended values, proportional to how much of the pixel the shape covers.

```
ALIASED edge      ANTI-ALIASED edge
###              ###
###              ###.
###              ###:
    ###          .###    (.,: = gray
    ###          :###    blend pixels)
```

Common anti-aliasing techniques: SSAA/supersampling (render big, shrink down — best, costly), MSAA (multiple samples per pixel), and post-process methods like FXAA and TAA. On a **screen**, anti-aliasing is excellent — it's why text looks smooth.

Why anti-aliasing is undesirable at the final print RIP

RIP (Raster Image Processor)

The software/hardware that converts a PDF or page-description file into the device's dot grid, then halftones it for the press.

Halftone

Simulating shades of gray/tone using patterns of tiny solid dots, since a press is essentially 1-bit (ink or no ink).

A RIP rasterizes vector art at very high device resolution — a 2400+ dpi imagesetter — so edges are already smooth to the naked eye. No anti-aliasing is needed. Worse, if AA *is* applied, the blended **gray edge pixels** along a saturated black stroke get halftoned at typical screen rulings (~130–150 lpi) and print as ugly, visible halftone dots or fuzzy seams. Unconstrained AA can turn a crisp sub-2-pixel black line into a 2-pixel-wide gray smear. A classic bug: thin light "hairlines" appearing along the edges of color fills in imported Illustrator art — the culprit is RIP anti-aliasing.

Pro tip — The rule: *keep vector elements vector all the way to the RIP*. Rasterize and anti-alias only for on-screen preview, never bake AA into print-final saturated edges or text. In PF360 terms: `pdf-service` (PDFKit, port 4000) should emit print-bound text and logos as true vectors in the PDF rather than flattening them to anti-aliased `sharp` raster bitmaps — that preserves crisp edges through the customer's print provider's RIP.

Resolution rules of thumb (print)

- **300 PPI** — baseline for raster images in standard print (cards, brochures, flyers). It's roughly 2× the halftone screen ruling, so at 150 lpi you want ~300 dpi of pixel data for accurate halftone dots.
- **72–96 PPI** — screen resolution. Fine for web, never for print.
- **~100 dpi at full size** — acceptable for large-format banners/signage viewed from a distance.
- Only *raster* elements are bound by the 300 dpi rule — vector type and logos stay sharp at any size. Use a PDF "Press Quality" preset to preserve resolution.

Real-world case — A customer designs a flyer in the PF360 studio: a photo background (raster, must be ≥300 PPI at print size), their logo (vector — sharp at any size), and headline text (vector font). Exported as a PDF, the photo stays raster while logo and text remain vector. If PF360 had preflight, it would warn that the photo is only 150 PPI at the chosen size — catching the blur *before* it reaches the press, not after a refund request. (Note: PF360 also lacks RGB → CMYK conversion, so screen colors may shift on press — a separate documented gap covered elsewhere.)

Key takeaways —

- **Raster = pixels**, resolution-dependent, best for photos; quality is fixed at creation and cannot be upscaled into existence.
- **Vector = math (paths)**, resolution-independent, best for logos/text/icons; scales infinitely with no quality loss and no PPI requirement.
- Vector paths are built from **anchor points** (curve passes through) and **control handles** (steer direction & pull). **Cubic Béziers** (4 points) are the design-tool standard — exactly what Fabric.js edits in `designer/`.
- **Fill** = interior; **stroke** = outline. Both are object properties in Fabric.js.
- **Rasterization** turns vectors into pixels; **anti-aliasing** smooths jaggies — great on screen, harmful at the print RIP, where gray edge pixels halftone into fuzzy, dotted edges.
- Print baseline: **300 PPI for raster**, vectors stay sharp at any size. Keep elements vector to the RIP; anti-alias only for preview.

Section 14: Image Resampling, Color Depth & Channels

Every photo, logo, or uploaded artwork is just a grid of tiny colored squares called pixels. This section is about two deep questions that decide whether a printed card looks crisp or muddy: **(1) what happens when we change how many pixels an image has** (resampling), and **(2) how much color information each pixel can hold** (color depth and channels). Get these wrong and a customer's beautiful design prints blurry, banded, or with the wrong colors — silently. Get them right and the same file looks professional.

Pixel

The smallest dot in a digital image. A 1000×500 image is a grid of 1000 dots wide by 500 tall = 500,000 colored dots.

Resampling

Recalculating pixel values when you change the number of pixels — i.e. resizing, rotating, or warping an image.

Resolution / PPI

PPI = "pixels per inch": how many pixels are packed into each printed inch. More pixels per inch = finer detail.

14.1 Resampling: making images bigger or smaller

There are two directions, and they have *opposite* problems.

Upsampling (enlarging)

Asking for more pixels than you started with. The computer must **invent** the new pixels.

Downsampling (shrinking)

Asking for fewer pixels. The computer must **throw away or combine** existing pixels.

Key Takeaway — Enlarging cannot create real detail — it can only guess between the pixels you already have. Shrinking destroys detail, and if you do it carelessly it creates ugly artifacts. Both directions need a good *algorithm* to do the math well.

The algorithm is just the rule for how many neighboring source pixels you look at, and how heavily you weight each one, when computing a new pixel. That "how many neighbors" is called the **kernel support**. Here are the four you must know.

Algorithm	How it works	Look	Use it for
Nearest neighbor	Copies the single closest pixel. No averaging, no new info.	Blocky, jagged "staircase" edges.	Pixel art, screenshots, 1-bit line art, exact integer scaling. Never photos.
Bilinear	Weighted average of the nearest $2 \times 2 = 4$ pixels.	Smooth but blurry; too soft for enlarging.	Real-time / GPU previews where speed beats quality.
Bicubic	Weighted cubic curve over $4 \times 4 = 16$ pixels.	Sharper and smoother than bilinear; the classic photo default.	General photo resizing. Variants: Catmull-Rom (sharper), Mitchell-Netravali (balanced).
Lanczos	A "windowed sinc" — the mathematically ideal filter — over 4×4 (lanczos2) or 6×6 (lanczos3) pixels.	Sharpest, best detail retention; but can create faint "halos" (ringing) at hard edges.	High-quality downscaling and final output. lanczos3 is the industry standard.

Analogy — Imagine asking witnesses to describe a crime. **Nearest neighbor** asks one person and repeats them word-for-word. **Bilinear** averages 4 people into a vague, mushy consensus. **Bicubic** interviews 16 people and weighs them intelligently. **Lanczos** interviews even more with a near-perfect method — but occasionally one loud witness causes an exaggerated detail (that's "ringing").

ENLARGING a hard edge (black|white):

```
Original:      [B][W]
Nearest:      [B][B][W][W]  <- crisp but doubled/blocky
Bilinear:     [B][gray][W]   <- soft, blurry transition
Lanczos:      [B][darker][gray][lighter][W]  <- sharp, but the
                                     "darker/lighter" overshoots = a faint halo (ring)
```

Why upscaling invents (and degrades) detail

When you enlarge, every brand-new pixel sits *between* real samples, so its value is a mathematical guess. Bicubic/bilinear guess by smoothing (result: soft). Lanczos guesses with sharper math (result: crisp but sometimes ringy). None of them adds true new texture — a blurry license plate stays unreadable. Only **AI/ML upscalers** (Photoshop's "Preserve Details 2.0", ESRGAN-class models) *hallucinate* plausible detail. That can look amazing, but it is *invented* — it can fabricate the wrong text or a slightly wrong face, so never trust it for legal/contractual artwork.

Common Mistake — Taking a 72-PPI web image, telling the export "300 DPI", and calling it print-ready. Changing the DPI number does *not* add pixels — you just stretched the same guesses across a bigger area. It will still print soft.

Why downscaling needs good filtering

When you shrink, several source pixels must collapse into one. If you just grab one and ignore the rest (no averaging), fine patterns — pinstripes, brick walls, small text, dense logos — produce **aliasing**: shimmering moiré, jaggies, and sparkle. The fix is **low-pass filtering**: averaging across *all* the source pixels being merged so nothing is missed.

Aliasing

False patterns (wavy moiré, jagged stair-steps) that appear when you sample fine detail too coarsely.

Low-pass filter

Averaging/blurring step that removes detail too fine to survive at the smaller size, *before* discarding pixels.

For huge reductions, simple box/area averaging is excellent; for moderate reductions, lanczos3 or bicubic. The pro workflow is: **downscale, then apply a light "unsharp mask"** to restore a touch of crispness.

14.2 What the Print-Flow-360 PDF service actually does

The Node `pdf-service` (port 4000) processes uploaded artwork with **sharp**, which is built on the fast **libvips** engine. Its `resize()` defaults are sensible — know them, because they decide the quality of every thumbnail, proof, and processed upload:

- **kernel:** `'lanczos3'` by default — the high-quality choice. Other options: `lanczos2`, `mitchell`, `cubic` (Catmull-Rom), `linear`, `nearest`, plus `mks2013` (Magic Kernel Sharp, used by Facebook) and `mks2021` (less over-sharpening — good for thumbnails).
- **fit:** `'cover'` by default — this *crops* the image to fill the box. Alternatives: `contain` (letterbox), `fill` (stretch, breaks aspect ratio), `inside`, `outside`.
- **withoutEnlargement:** `false` — meaning sharp *will* upscale by default. Set it `true` to forbid faking resolution on small artwork.
- **fastShrinkOnLoad:** `true` — uses JPEG/WebP shrink-on-load (decoding fewer DCT blocks) for speed; can add slight aliasing at extreme ratios. Disable for maximum quality.

Under the hood libvips does a fast integer pre-shrink, then a high-quality residual resample — efficient even on huge files. On *upscale*, the downsample-only kernels fall back to cubic interpolation.

Common Mistake — Leaving `fit:'cover'` on customer uploads silently chops off edges of their design. And leaving `withoutEnlargement:false` means a tiny logo gets blown up into soft mush instead of triggering a "too small" warning. Both are exactly the kind of silent quality loss this platform must avoid.

14.3 Color depth: how much color a pixel can hold

Bit depth

The number of bits used to store *one channel* of *one pixel*. More bits = more possible levels. Levels per channel = 2^{bits} .

Channel

One color "layer" of an image. RGB has 3 (Red, Green, Blue); CMYK has 4; an alpha channel adds transparency.

Bit depth (per channel)	Levels per channel	Common name & total colors (RGB)	Used for
1-bit	2 (black/white)	Bitmap / line art	Pure line art, fax, screened output
8-bit	256 (0–255)	"24-bit color" $\approx$ 16.7 million	Web, JPEG, PNG, final print files
16-bit	65,536	"48-bit color" $\approx$ 281 trillion	Editing master files (Photoshop's "16-bit" is really 15-bit+1 = 32,769 levels)
32-bit float	(continuous)	HDR / scene-referred	High-dynamic-range, linear compositing

Note the shorthand confusion: "32-bit color" usually means 24-bit RGB + 8-bit alpha; "30-bit" = 10 bits/channel; "48-bit" = 16 bits/channel. The number is bits *per pixel*, not per channel — always check which is meant.

The three channel systems

RGB

3 channels, *additive* (mixing light). This is how screens work — and it is sharp's working space and the web's space.

CMYK

4 channels — Cyan, Magenta, Yellow, Key (black) — *subtractive* (mixing inks). This is how presses work. CMYK has a **smaller gamut** (range of reproducible colors) than RGB, so bright screen blues/greens can't be printed exactly.

Alpha

An extra channel storing transparency (0 = fully see-through, 255 = fully opaque at 8-bit), at the same bit depth as the color channels.

Common Mistake (a real Print-Flow-360 gap) — The platform has **no RGB → CMYK conversion and no preflight**. Customers upload bright RGB artwork; if it's sent to a CMYK press without an **ICC profile** conversion (which controls rendering intent and black generation), colors shift and out-of-gamut areas go flat. A naive convert ruins color. sharp/libvips *can* handle CMYK, but proper separation needs the profile. Until that conversion + preflight exists, the honest move is to *warn the customer* at upload that screen colors may shift in print, never silently pass RGB through.

Pro Tip — When sharp resizes an image with transparency, it **premultiplies the alpha** first (multiplies color by opacity) so edges don't get dark or colored "fringe" halos. Hand-rolled resize code that skips this step produces ugly halos around cut-out logos. This is a quiet reason to keep image work inside sharp.

14.4 Bit depth vs banding — why 16-bit matters for editing

Banding (posterization)

Visible stepped stripes in what should be a smooth gradient — skies, sunset vignettes, soft shadows — because there aren't enough tonal levels to blend smoothly.

Smooth sky gradient:

8-bit, after heavy edits:  <- visible "steps" = banding
16-bit, same edits:  (smooth, no steps)
8-bit histogram after edit:  <- "comb": gaps = lost levels

256 levels (8-bit) sounds like plenty, but a very gradual gradient across a wide area can need finer steps than 256, so you see bands. The real danger is **editing**. Every curve, levels, exposure, or color-grade adjustment *redistributes and discards* tonal values. In 8-bit, this thins your 256 levels into gaps — a "comb" histogram — and banding appears. 16-bit has ~256× more headroom, so cumulative edits don't break the gradient.

Key Takeaway — The golden rule: **edit in 16-bit, export/flatten to 8-bit for delivery**. Adding a touch of output *dithering* (tiny random noise) masks any residual 8-bit banding.

14.5 Print numbers you must know

Effective PPI

The resolution an image *actually* prints at = native pixel width ÷ placed size in inches. This — not the file's DPI tag — is what determines sharpness.

LPI (lines per inch)

The fineness of the halftone screen a press uses.

- **Target 300 PPI** at final size for close-viewed work: business cards, brochures, packaging.

- **Large-format/posters** are fine at 100–150 PPI because you stand back to view them.
- **Nyquist rule for halftones:** $\text{PPI} \approx 2 \times \text{LPI}$. A 150-LPI press $\rightarrow$ 300 PPI; 200 LPI $\rightarrow$ ~300–400 PPI.
- **Prepress downsampling:** on PDF export, downsample images above ~450 PPI down to 300; never set the threshold below 300 for final output (lower is only for proofs).

Real-world case — A customer places a 1000-pixel-wide logo into a design and stretches it to fill 10 inches. Effective PPI = $1000 \div 10 = 100 \text{ PPI}$ — far below the 300 needed for a card. It looked fine on screen (where the canvas is small) but prints visibly soft. This "scaling art up in the layout drops effective PPI" trap is the #1 quiet quality killer, and it's exactly what a preflight check in the **designer/** studio or **pdf-service** should catch.

14.6 Practical pipeline guidance for uploaded artwork

1. **Validate actual pixels, not the DPI tag.** Compute effective PPI from real dimensions $\div$ required print size. If it falls below ~150–300, *warn or block* — never silently upscale to fake the resolution. Set sharp's `withoutEnlargement: true` as the safety net.
2. **Downscale well.** Use the default `lanczos3` (or `mks2021` for thumbnails/proofs); consider area/box averaging for very large reductions; optionally apply a light unsharp afterward.
3. **Keep 16-bit through intermediate steps**, output 8-bit (plus dither) for proofs, and convert to CMYK via an ICC profile for press — not silently.
4. **Preserve and premultiply alpha** on every resize so cut-out logos don't get edge halos.
5. **Use 1-bit / nearest neighbor only for true line art** — never on photos, and never use smooth interpolation on pixel art or crisp line art (it blurs the very edges you need sharp).
6. **Speak human.** Surface "This image is low resolution for the size you chose — it may look blurry when printed" with a fix ("upload a larger file"), not a raw "97 PPI" number. This is the platform's non-technical-user standard, and it's exactly where the weak order-to-production spine needs a preflight gate.

Common Mistakes recap — (1) Upscaling web images and claiming print quality. (2) Nearest-neighbor on photos / smooth filters on line art. (3) Downsampling without low-pass filtering $\rightarrow$ moiré. (4) Heavy 8-bit edits $\rightarrow$ banded skies. (5) Resizing RGBA without premultiplying $\rightarrow$ fringe halos. (6) RGB $\rightarrow$ CMYK with no ICC profile $\rightarrow$ color shift. (7) Trusting the DPI metadata tag instead of pixel count. (8) Leaving `fit:'cover'` and silently cropping the customer's design.

Key takeaways —

- Resampling = recomputing pixels when you resize. Upscaling *invents* detail (a guess, never real); downscaling *destroys* detail and needs low-pass filtering to avoid aliasing/moiré.
- Algorithm quality ladder: nearest (line art only) → bilinear (fast/blurry) → bicubic (good photo default) → **lanczos3** (sharpest, slight ringing, the standard). Only AI upscalers add fake-but-plausible detail.
- Print-Flow-360's `pdf-service` uses sharp/libvips: default kernel `lanczos3`, default `fit:'cover'` (crops!), and `withoutEnlargement:false` (upscales by default — flip it to `true` for artwork).
- Bit depth = levels per channel: 1-bit (2), 8-bit (256), 16-bit (65,536). Channels: RGB (screen, additive), CMYK (print, subtractive, smaller gamut), alpha (transparency).
- Banding comes from too few levels under heavy editing. Edit in 16-bit, export 8-bit + dither.
- Print at 300 PPI for close work ($\text{PPI} \approx 2 \times \text{LPI}$). Effective PPI = pixels ÷ placed inches — scaling art up in a layout silently lowers it. Trust pixels, not the DPI tag.
- Platform reality: no RGB → CMYK conversion and no preflight yet — so warn customers about color shift and low resolution at upload rather than failing silently.

Section 15: Alpha Compositing, Blend Modes & Transparency Flattening

Every time you stack a logo over a photo, fade a watermark to 50%, or add a soft drop shadow, your computer is doing math to decide what color each pixel ends up. That math is called *compositing*. This section teaches you how transparency actually works, why "see-through" effects can blow up when you send a file to a printing press, and exactly what the Print-Flow-360 design studio and PDF service must do to avoid ugly halos and color shifts. We start from the ground up.

15.1 The Alpha Channel: pixels that are partly see-through

Channel

One layer of information per pixel. A normal color image has three: Red, Green, Blue (RGB). Each says how much of that color is in the pixel.

Alpha channel

A fourth value per pixel that stores **opacity** — how solid or see-through the pixel is. 0 = fully transparent (invisible), 1 = fully opaque (solid). In 8-bit images this is the range 0–255 instead of 0–1.

So a pixel isn't just "red." It's "red, and 40% solid." That extra alpha value is what lets one image sit on top of another and let the lower one peek through.

Analogy — Think of each layer as a sheet of colored glass. Clear glass (alpha 0) shows everything behind it; frosted glass (alpha 0.5) tints what's behind; painted-solid glass (alpha 1) hides it. Stacking glass sheets to see the final picture is exactly what compositing does.

15.2 The "over" operator — the rule for stacking

The most common compositing operation is "**A over B**": put foreground A on top of background B. This is the everyday "normal" stacking you see in any layers panel — the painter's algorithm, painting back-to-front.

The formula was formalized in the foundational 1984 paper by Porter & Duff, "*Compositing Digital Images*" (SIGGRAPH), which introduced 12 operators (over, in, out, atop, xor, and more). Bruce Wallace independently derived the same "over" math in 1981 from a physical model of how light reflects and passes through surfaces. For "over," the resulting opacity is:

$$\text{alpha_out} = \text{alpha_a} + \text{alpha_b} * (1 - \text{alpha_a})$$

Read that in plain words: the result is as opaque as the top layer, *plus* however much of the bottom layer still shows through the part the top didn't cover. If the bottom is fully solid (`alpha_b = 1`), the

color simplifies to the familiar blend (a "lerp," linear interpolation): $C_{out} = C_a * \alpha_a + C_b * (1 - \alpha_a)$. A logo at 30% over a photo = 30% logo color + 70% photo color. That's it.

15.3 Straight vs. premultiplied alpha (the source of most "halo" bugs)

There are two ways to store a transparent pixel's color, and confusing them causes one of the most common real-world graphics bugs.

Straight (a.k.a. unassociated / non-premultiplied) alpha

RGB stores the object's *true* color, independent of its opacity. A 30%-opaque red pixel is stored as (full red, alpha 0.3). **PNG files use straight alpha.**

Premultiplied (a.k.a. associated) alpha

RGB is already multiplied by alpha — it stores "emitted color" (color × alpha) and alpha stores "occlusion." That same 30% red is stored as (0.3 × red, alpha 0.3). Many GPU and compositor pipelines (OpenGL, After Effects internals, Cairo, Direct2D, Apple's `.premultipliedLast`) use premultiplied.

The two "over" formulas:

Form	Color formula for "over"	Note
Straight	$C_{out} = (C_a * \alpha_a + C_b * \alpha_b * (1 - \alpha_a)) / \alpha_{out}$	needs a divide
Premultiplied	$C_{out} = C_a + C_b * (1 - \alpha_a)$	no divide; cleaner

Both produce the same output alpha. Premultiplied wins for three reasons: (1) the Porter–Duff operators have their simple clean form *only* in premultiplied; (2) it makes scaling, blurring, mipmapping, and anti-aliasing correct — no colored halos; (3) "over" becomes **associative**, meaning $(A \text{ over } B) \text{ over } C = A \text{ over } (B \text{ over } C)$. That last point is huge for print: it lets you pre-flatten a *group* of layers into one and get the identical result, which is the whole basis of flattening (15.6).

Common mistake — the dark-fringe / halo bug. With straight alpha, fully transparent pixels (alpha 0) *still carry an RGB color*, usually black. When a filter or GPU blends neighboring pixels (resize, anti-alias, mipmap), it drags those invisible black pixels into the visible edge → a dark or colored halo around a cutout logo or sprite. Premultiplying first zeroes those contributions, killing the halo. This is THE classic "why does my transparent PNG have a black outline" problem.

Pitfall — double premultiplication. Premultiplying an already-premultiplied image darkens edges. Unpremultiplying needs $C_{straight} = C_{pre} / \alpha$, which is undefined at alpha 0 and loses precision for tiny alphas. Because PNG is straight and most GPU stacks are premultiplied, mixing them up is a constant source of edge artifacts.

15.4 Blend modes — beyond plain stacking

A **blend mode** changes *how* the top layer's color combines with the layer below, instead of just covering it. Most are "separable" (computed per channel, on values normalized to 0–1). The math below follows the PDF 1.4 imaging model, standardized in the W3C "Compositing and Blending Level 1" spec. Let b = top value, a = base value.

Mode	Formula	Effect & everyday use
Normal	$f = b$	plain stacking
Multiply	$f = a \cdot b$	always darker; white is invisible, black stays black. Shadows, ink-on-paper feel.
Screen	$f = 1 - (1-a)(1-b)$	always lighter; the inverse of multiply. Glows, highlights.
Overlay	multiply if $a < 0.5$, else screen	boosts contrast; 50% gray = invisible. Depends on the BASE (non-commutative).
Hard Light	Overlay with layers swapped (keys off b)	like shining a harsh light through the top art.
Darken / Lighten	$\min(a, b)$ / $\max(a, b)$	keep the darker / lighter pixel.
Color Dodge / Burn	$a / (1-b)$ / $1 - (1-a)/b$	strong brighten / darken.
Difference	$ a - b $	black is invisible; used to spot changes.

Multiply and screen are *commutative* (order of the two layers doesn't matter). Overlay and hard light are not — they depend on which layer is the base. **Soft Light** (Photoshop): if $b < 0.5$, $2ab + a^2(1-2b)$; else $2a(1-b) + \sqrt{a} \cdot (2b-1)$. Four modes — **Hue, Saturation, Color, Luminosity** — are *non-separable*: they work in HSL space across all channels at once, not per channel.

Analogy — Multiply = stacking two highlighter-marked transparencies on an overhead projector: each one only subtracts light, so it gets darker. Screen = pointing two slide projectors at the same wall: light adds up, so it gets brighter.

Print color-shift trap. Blend modes are computed in a specific color space. Adobe's "Transparency Blend Space" can be set to Document RGB or Document CMYK, and the SAME multiply can produce different output in each. A multiply over rich black or a spot color is a frequent prepress surprise — pick the wrong blend space and colors shift on press.

15.5 Two different "flattenings" — don't confuse them

Layer flattening (Photoshop sense)

Merge all stacked layers into one raster (pixel) image. You lose the ability to edit layers separately. Purely about editability.

Transparency flattening (vector workflow — Illustrator/InDesign/ Acrobat)

Convert "live" transparency — opacity below 100%, blend modes, drop shadows, feathering, glows, transparent gradients/mesh, knockout groups — into plain **opaque** objects that *look* identical but contain no transparency. This is what print prep does.

How transparency flattening works: it finds every place where transparent objects overlap and slices the artwork into **atomic regions** — the distinct intersection patches of all overlapping objects. Each patch is then rebuilt as either a solid vector fill (kept as crisp vector when the look allows) or a rasterized image (when the effect can't be drawn as vector). Because "over" is associative (15.3), the merged result matches the live preview — in theory.

15.6 Why old RIPs and PDF/X-1a can't keep live transparency

RIP (Raster Image Processor)

The software/hardware at a print shop that turns your PDF into the dots the press lays down. Old RIPs predate the transparency era.

Transparency only entered PDF in **PDF 1.4** (Acrobat 5, 2001). The print-exchange standards built on older PDF can't represent it:

Standard	Built on	Transparency	Color
PDF/X-1a (2001/2003)	PDF 1.3	None — must flatten	DeviceCMYK / spot only, all fonts embedded, no ICC, explicit overprint
PDF/X-3	PDF 1.3	None — flattens	color-managed (ICC allowed)
PDF/X-4 (2008/2010)	PDF 1.6	Live transparency kept	ICC-managed RGB + spot, layers (OCGs)

Pro tip — rule of thumb. Send **PDF/X-4** whenever the printer's RIP accepts it (modern Adobe PDF Print Engine RIPs do) — it avoids flattening artifacts entirely. Use **PDF/X-1a** only when an old RIP explicitly requires it, and then carefully tune the flattener and verify overprints.

15.7 Flattening artifacts — the failures you'll actually see

1. **White box / white seams / stitching:** thin white hairlines along the borders between atomic regions (where a raster patch meets a vector patch) caused by sub-pixel rounding gaps. Often screen-only,

but some RIPs *print* them. Fix: enable *Clip Complex Regions* (snaps boundaries to exact vector paths), raise flattener resolution, or shift the raster/vector balance.

2. **Color shifts:** a rasterized patch gets re-converted in a different blend/color space, so a rectangle of slightly-off tone appears exactly where a drop shadow or multiply overlapped.
3. **Text reflow / jaggies:** "Convert All Text to Outlines" thickens small type and drops hinting; rasterized text near transparency looks fuzzy.
4. **Stroke weight change:** "Convert All Strokes to Outlines" can shift hairline widths.
5. **Unexpected rasterization** of large areas → bloated files and resolution-limited gradients (banding).

Adobe flattener preset numbers (memorize these). Raster/Vector Balance slider: 100 = keep maximum vector. Line Art & Text Resolution: High = **1200 ppi** (Medium 300, Low 288). Gradient & Mesh Resolution: High = **300 ppi** (Medium 150, Low 144). Options: Convert All Text to Outlines, Convert All Strokes to Outlines, Clip Complex Regions, Anti-alias. Use the *High Resolution* preset for press; Low/Medium for web proofs only.

15.8 Tie-in: Print-Flow-360's designer and PDF service

The Fabric.js **design studio** ([designer/](#)) composites layers on an HTML canvas using per-object alpha/opacity — that is live alpha compositing running in the customer's browser. A customer can set object opacity, add overlapping art, and (potentially) blend modes. The browser shows the premultiplied, anti-aliased result instantly.

When the Node **PDF service** ([pdf-service/](#) , port 4000, using **sharp** for images and PDFKit for PDFs) turns that canvas into a print file, two concrete things matter:

- **Premultiply correctly.** The canvas export is a PNG with *straight* alpha. When **sharp** resizes or composites it, handle alpha correctly so cutout art doesn't pick up a black/white fringe (15.3). A transparent canvas background must not bleed into edge pixels.
- **Choose the right output target.** Emit **PDF/X-4** to preserve live transparency and ICC color when the print partner's RIP supports it (best fidelity). Emit **PDF/X-1a** (flattened, DeviceCMYK) only for legacy RIPs — and set the flattener to 1200 ppi text / 300 ppi gradient with Clip Complex Regions on to avoid white seams.

Best practice — the honest-UX moment. If a design uses opacity, blend modes, or drop shadows AND the chosen print format is X-1a, surface a plain-language note: "Transparency effects will be merged for this print format and may shift slightly — please review the proof." That respects the platform's UX-first rule and turns a silent color shift into an informed decision. Remember this connects to a documented gap: PF360 has **no built-in RGB → CMYK conversion and no preflight**, so blend-space and overprint surprises are not yet caught automatically — flag them, don't hide them.

Key takeaways —

- Alpha = per-pixel opacity (0–1); the **"over"** operator (Porter–Duff, 1984) is normal layer stacking:
`alpha_out = alpha_a + alpha_b(1-alpha_a)` .
- **Premultiplied** alpha makes filtering, scaling, and Porter–Duff math clean and associative; **straight** alpha (used by PNG) causes dark/colored edge halos if you blend before premultiplying.
- Blend modes: **multiply** darkens, **screen** lightens, **overlay** boosts contrast and depends on the base. They're computed in a color space, so RGB vs CMYK blend space can shift press color.
- **Transparency flattening** slices overlaps into atomic regions and rebuilds them as opaque vector or raster — needed because **PDF/X-1a (PDF 1.3) has no transparency**; **PDF/X-4 (PDF 1.6) keeps it live**.
- Flattening artifacts: white seams/stitching, rasterized color shifts, fuzzy outlined text. Mitigate with Clip Complex Regions + high resolution (1200 ppi text / 300 ppi gradient).
- In PF360: the Fabric.js studio composites live; the `pdf-service` must premultiply on export to avoid fringes and prefer X-4, warning users when X-1a will flatten their effects — especially since PF360 has no automatic CMYK conversion or preflight yet.

Section 16: Inside a PDF — Document Model & Content Streams

You have probably opened thousands of PDFs and never wondered what is inside one. But if you build a print platform, the PDF is your *final product* — it is the file that goes to the press, the thing your customer pays for. Understanding how a PDF is assembled lets you reason about why files are huge or tiny, why text is sometimes searchable and sometimes a blurry picture, and why a beautiful design on screen can print wrong. In Print-Flow-360, the Node `pdf-service` (port 4000) builds these files with a library called **PDFKit**, so this is not academic — it is the machine room of the product.

Let us start from the outside and work in.

What a PDF file actually is

A PDF is a plain-text-ish container of numbered building blocks, plus an index that lets a reader jump straight to any block. Think of it less like a Word document and more like a tiny database with a table of contents at the very end.

PDF (Portable Document Format)

A file format that describes exactly how a page should look — text, shapes, images, fonts, colors — so it renders identically everywhere. Standardized as ISO 32000 (PDF 1.7 = ISO 32000-1:2008; PDF 2.0 = ISO 32000-2).

Every PDF has **four parts, in this order**:

```
+-----+
| 1. HEADER   %PDF-1.7 (+ binary-marker comment) |
+-----+
| 2. BODY     the numbered objects (the content)  |
+-----+
| 3. XREF     byte-offset index of every object   |
+-----+
| 4. TRAILER  points to Root + xref; ends %%EOF   |
+-----+
```

1. **Header** — one line like `%PDF-1.7` telling the reader the version. A second comment line of high-value bytes usually follows so that file-transfer tools treat the file as binary, not text (which could corrupt it).
2. **Body** — all the real content, stored as *objects* (next section).
3. **Cross-reference table (xref)** — a map of where each object lives, measured in bytes from the start of the file.
4. **Trailer** — the entry point. It names the root object and says where the xref table is, then ends with `%%EOF`.

Key idea — a PDF reader parses *end-first*: it finds `%%EOF`, reads `startxref` to locate the index, jumps to the xref table, then follows `/Root` to the document. It never has to read the whole file to find one page — that is what makes a 500-page PDF open instantly.

Objects — the eight building blocks

Everything in the body is built from eight basic object types:

Type	Looks like	Plain meaning
Boolean	<code>true</code>	yes/no
Numeric	<code>612</code> , <code>0.75</code>	integer or decimal
String	<code>(Hello)</code> or <code><48656C></code>	text, literal or hex
Name	<code>/Type</code> , <code>/Helvetica</code>	a fixed keyword/label (starts with <code>/</code>)
Array	<code>[1 0 R 2 0 R]</code>	an ordered list
Dictionary	<code><< /Key value >></code>	key/value map (the workhorse)
Stream	dictionary + raw bytes	a dictionary plus a blob of data (drawing instructions, an image, a font)
Null	<code>null</code>	nothing

An **indirect object** is one that is numbered so other objects can point at it:

```
5 0 obj << ... >> endobj
^ ^
| +-- generation number (almost always 0)
+---- object number
```

Anywhere else in the file, a reference to that object is written `5 0 R` (the `R` means "reference"). The generation number only changes when an object slot is freed and reused, which is rare in freshly generated files — so you will see `0` nearly everywhere.

The xref table, concretely

The cross-reference table is a fixed-width index. Each entry is exactly **20 bytes**: a 10-digit byte offset, a 5-digit generation number, a flag (`n` = in use, `f` = free), and a 2-byte line ending.

```

xref
0 6          <- start at obj 0, 6 entries
0000000000 65535 f      <- obj 0, head of the free list
0000000009 00000 n      <- obj 1 lives at byte 9
0000000074 00000 n      <- obj 2 at byte 74
0000000120 00000 n
...

```

Because every entry is the same length, the reader can compute exactly where object 5's entry sits and jump there — no scanning. PDF 1.5+ can replace this human-readable table with a compressed *cross-reference stream* and pack many small objects into *object streams*, producing smaller files at the cost of readability.

The document tree: Catalog → Pages → Page

The trailer's `/Root` points to the **Catalog**, the top of a tree that organizes the document:

```

Catalog      << /Type /Catalog /Pages 2 0 R >>
  |
Pages node    << /Type /Pages /Kids [3 0 R ...] /Count N >>
  |
Page leaf     << /Type /Page /Parent 2 0 R
               /MediaBox [0 0 612 792]
               /Resources << /Font << /F0 4 0 R >> >>
               /Contents 5 0 R >>

```

MediaBox

The physical page size, in **points** (1 point = 1/72 inch). `[0 0 612 792]` = US Letter (8.5×11"). A4 = `[0 0 595 842]`.

CropBox / BleedBox / TrimBox / ArtBox

Additional rectangles. For print specifically, **TrimBox** (the final cut line) and **BleedBox** (artwork that runs past the cut so there is no white edge) are what a prepress operator cares about.

Resources

A lookup table mapping short names used in the drawing (like `/F0` or `/Im0`) to the heavy objects they refer to (fonts, images). This decoupling means a font used on 50 pages is stored *once*.

Analogy — the content stream is a recipe that says "add 2 cups of *flour*." The `/Resources` dictionary is the pantry that says "*flour* = that 20 kg sack in the corner." The recipe stays short and readable; the bulky ingredient is stored once and reused.

The content stream — the drawing program

The `/Contents` of a page is a **stream object**: a little program of drawing instructions. It uses *postfix* (RPN) notation — operands come first, the operator comes last — and the coordinate origin is the

bottom-left corner, with y increasing upward.

Worked example — a "Hello, World!" page body:

```
5 0 obj << /Length 44 >>
stream
BT                % begin text
/F0 36 Tf         % use font /F0 at 36 pt
50 700 Td         % move to x=50, y=700
(Hello, World!) Tj % show this string
ET                % end text
endstream endobj
```

The stream's dictionary must carry `/Length` (the exact byte count of the data) and often a `/Filter` such as `/FlateDecode` (zlib/deflate compression — the most common). Get `/Length` wrong and readers throw parse errors.

The most important operators to recognize:

Group	Operators	What they do
Graphics state	<code>q</code> / <code>Q</code> , <code>cm</code> , <code>w</code>	save/restore state (must nest evenly), apply a transform matrix, set line width
Color	<code>rg</code> / <code>RG</code> , <code>g</code> / <code>G</code> , <code>k</code> / <code>K</code>	fill/stroke in RGB, gray, or CMYK — all values are 0–1 , not 0–255
Path build	<code>m</code> , <code>l</code> , <code>c</code> , <code>re</code> , <code>h</code>	moveto, lineto, cubic Bézier curve, rectangle, close path
Path paint	<code>S</code> , <code>f</code> , <code>f*</code> , <code>B</code> , <code>W</code>	stroke, fill (nonzero), even-odd fill, fill+stroke, clip
Text	<code>BT</code> / <code>ET</code> , <code>Tf</code> , <code>Td</code> , <code>Tj</code> , <code>TJ</code>	begin/end text, set font, move, show string, show with kerning
Images	<code>Do</code>	paint a named XObject (e.g. <code>/Im0 Do</code>)

A filled blue rectangle is just: `q 0 0 0.75 rg 100 100 200 100 re f Q` — save state, set fill to RGB(0, 0, 0.75), build a rectangle, fill it, restore state.

Common mistake — using color values like `255 0 0 rg` . PDF expects **0–1**, so `255` is clamped to `1` and your channels go haywire. The other classic bug is unbalanced `q` / `Q` : every `q` needs a matching `Q` , or graphics state (color, clip, transform) "leaks" into everything drawn afterward.

Vector vs raster inside the PDF

This is the distinction that decides whether your print file is crisp and small or blurry and huge.

Vector

Content described as math: paths (`m / l / c`) and text operators living directly in the content stream. Infinitely scalable — sharp at 100% or 800% zoom — tiny in bytes, and the text stays selectable and searchable.

Raster (an image XObject)

A stored grid of pixels: a stream object with `/Subtype /Image`, `/Width`, `/Height`, `/ColorSpace`, `/BitsPerComponent` (usually 8), and a `/Filter` like `/DCTDecode` (JPEG) or `/FlateDecode` (PNG-like). Fixed resolution — it pixelates once you zoom past its native DPI.

For print, the rule of thumb is **300 DPI / 300 PPI at final size** (150 line-screen × 2). Upscaling a 72-DPI web image to 300 DPI cannot invent detail that was never captured — it just gets blurry.

	Vector text/shapes	One flattened page image
File size (Letter page)	a few KB	~19 MB raw (6.5M px × 3 bytes at 300 DPI RGB)
Sharp at any zoom?	Yes	No — pixelates past native DPI
Text searchable/selectable?	Yes	No — it is now just pixels
CMYK plate accuracy	Clean	Degraded, with transparency baked in

The big print-shop mistake — rasterizing the *entire* page to a single bitmap. It bloats the file, destroys searchable/accessible text, softens fine line art, and discards the live transparency the press's RIP (the software that converts the PDF to printer dots) needs to blend colors correctly. Keep text and vectors as vectors even if the file is slightly larger.

Pro tip — rasterize ≠ flatten. *Flattening* (transparency flattening) merges overlapping semi-transparent objects into opaque ones while *keeping text and vectors as vectors*. *Rasterizing* turns things into pixels at a chosen DPI. Prepress guidance: do not pre-flatten or rasterize prematurely — hand a clean vector PDF with live transparency to the RIP and let it do the final blending and color at output.

How PDFKit emits all this in PF360's pdf-service

PDFKit (the foliojs library) is a **streaming generator**. A `PDFDocument` is a readable stream you pipe to a file or directly to an HTTP response, so the service can build a large PDF without holding the whole thing in memory — which matters for a multi-tenant service handling many jobs on port 4000.

Your high-level calls compile down to the exact operators above:

PDFKit call	Emits
<code>doc.text("Hi")</code>	<code>BT ... /Fn s Tf ... Td (Hi) Tj ET</code> (stays vector)
<code>doc.rect(x,y,w,h).fill()</code>	<code>re ... f</code>
<code>doc.moveTo().lineTo()</code>	<code>m ... l</code>
<code>doc.image("logo.png")</code>	embeds an image XObject + <code>Do</code> (the only call that introduces raster)

PDFKit auto-manages object numbers, the page tree, the `/Resources` dictionaries, font subsetting and embedding, FlateDecode compression, and finally writes the xref table and trailer when you call `doc.end()`. The takeaway for PF360: **text and shapes you draw with PDFKit are vector by default**; raster only enters when you call `doc.image(...)` (where `sharp` typically prepares/resizes the bitmap first).

PF360 in practice — when the Fabric.js **design studio** (`designer/`) exports a customer's artwork, the dangerous shortcut would be to flatten the whole canvas to one big PNG and drop it into the PDF — small effort, terrible output (no crisp text, bloated file, no clean CMYK separation). The right approach is to translate Fabric text objects into PDFKit `doc.text()` calls and Fabric shapes into vector paths, reserving `doc.image()` only for genuine photos the customer uploaded. That keeps the print file crisp, small, and prepress-friendly.

Connecting to PF360's documented gaps — the platform currently has *no RGB → CMYK conversion* and *no preflight* step. In content-stream terms that means PDFKit is emitting `rg` (RGB fill, 0–1) where a press wants `k` (CMYK fill, 0–1), so brand colors and storefront theme colors can shift on press. There is also no automated check for low-DPI images, missing TrimBox/BleedBox, or unembedded fonts before the PDF reaches production. Knowing the object model tells you *exactly* where a future preflight check would live: inspect each page's `/MediaBox` vs `/TrimBox`, scan image XObjects' `/Width` ÷ physical-size for effective DPI, and verify the color operators and font dictionaries.

Key takeaways —

- A PDF = header, body of numbered **objects**, an **xref** byte-offset index, and a **trailer** pointing to the Catalog; readers parse it *end-first* for instant random access.
- Objects come in 8 types; **dictionaries** and **streams** do the heavy lifting. `5 0 obj` defines, `5 0 R` references.
- The document tree is **Catalog** → **Pages** → **Page**; the page's `/MediaBox` is in **points** (612×792 = Letter), and `/Resources` maps short names to fonts/images.
- The **content stream** is a postfix drawing program (origin bottom-left); colors are **0–1**, and `q / Q` must balance.
- **Vector** (paths + text ops) stays crisp, small, and searchable; **raster** (image XObjects) is fixed-DPI — aim for **300 DPI at final size**.
- Never flatten a whole page to one image; **flatten ≠ rasterize** — keep vectors and live transparency for the RIP.
- PDFKit streams these objects out incrementally; `doc.text()` / `doc.rect()` stay vector, only `doc.image()` adds raster. PF360's missing RGB → CMYK and preflight steps map directly onto color operators and object inspection.

Section 17: Fonts — Embedding, Subsetting & Outlines

Type — the actual letters in your design — is where print files quietly betray you. A flyer that looks perfect on your screen can come back from the printer with the wrong typeface, words shoved off the edge, or little empty boxes where characters should be. Almost always the cause is the same: the file *named* a font but did not *carry* it. This section explains why that happens and the two ways professionals make sure it never does — **embedding (with subsetting)** and **converting to outlines**.

Font

A file that contains the drawn shapes of letters, numbers, and symbols (the "glyphs") plus rules for spacing them.

Glyph

One drawn character — the shape of "a", "5", "&", or "あ". A font is a collection of glyphs.

RIP

Raster Image Processor — the printer's software/hardware that turns your PDF into the dots it actually prints. It runs on the print shop's machine, not yours.

Why text breaks when a font isn't embedded

A PDF or EPS can refer to a font just by *name* — like a recipe that says "add cheese" without including any cheese. If the device opening the file doesn't have that exact font installed, it cannot draw your shapes. So it does **font substitution**: it swaps in whatever fallback it has, usually Helvetica/Arial, Times, or an Adobe "Multiple Master" simulation font that tries to fake the right widths.

Substitution causes three distinct failures:

1. **Wrong look** — your elegant script becomes plain Helvetica. The brand is gone.
2. **Reflow** — the substitute font's letters are different widths and spacing (its "metrics" differ), so the text rewraps, overflows its box, pushes line and page breaks around, and collides with images. The simulation fonts try to hold the line breaks by matching widths, but it's approximate and ugly.
3. **Wrong or missing glyphs** — the substitute may not contain the characters you used. You get `.notdef` boxes (□, often called "tofu"), blank gaps, or garbage. Accented letters, non-Latin scripts, and the classic Symbol & ZapfDingbats fonts are the usual casualties.

Common Mistake — Trusting your own screen. The file proofs perfectly on *your* computer because *you* have the font installed. The print shop's RIP almost never has it. This is a *silent failure*: nothing warns you until the wrong job is already printed.

Analogy — Emailing a document that says "use the handwriting font from my laptop." It looks great when you open it, because the font lives on your laptop. Everyone else sees boring default text. Embedding is attaching the actual font to the email so it travels with the file.

Embedding — pack the font inside the file

Embedding stores a full copy of the font program inside the PDF itself. Now the file is self-contained: it renders identically on any machine, regardless of what fonts are installed. PDF can embed Type 1, TrueType, Type 3, OpenType (PDF 1.6+), and composite Type 0 / CID fonts for large character sets like Chinese, Japanese, and Korean (CJK).

The cost is file size. A full CJK font holds tens of thousands of glyphs and can be many megabytes. That's where subsetting comes in.

Subsetting — embed only the glyphs you actually used

Subsetting

Embedding only the specific glyphs your document uses, instead of the whole font.

If a poster uses 40 characters out of a 600-glyph font, the subset embeds roughly those 40 — a tiny fraction of the size. You can spot a subset in any PDF: its font name gets a six-uppercase-letter prefix plus a "+", e.g. `ABCDEF+Garamond` or `E00DIA+TimesNewRoman`. Adobe Distiller's default rule is "subset embedded fonts when the percent of characters used is less than 100%."

Pitfall — A subset is not a usable full font. If you reopen the PDF and try to type a character the subset doesn't contain, you get `.notdef`. And merging two PDFs that each subset "the same" font can clash — each carries a different `ABCDEF+` subset, forcing messy re-subsetting. For a *final* print file this is fine; just make sure every glyph you need is present before you export.

Licensing — you may not always be allowed to embed

Fonts are licensed software. OpenType/TrueType fonts carry a 16-bit flag called **fsType** (in the font's OS/2 table) that declares what embedding the foundry permits:

fsType level	What it allows
Installable (0)	Embed and even permanently install on another machine. Most permissive.
Editable (0x0008)	Embed; recipient may view, print, and edit.
Preview & Print (0x0004)	Embed for viewing and printing only — no editing or extraction.
Restricted / No embedding (0x0002)	May NOT be embedded without explicit foundry permission. Tools refuse with "font could not be embedded due to licensing restrictions."

Extra bits exist too: `0x0100` forbids subsetting, `0x0200` allows bitmap embedding only.

Key point — fsType is *advisory metadata, not DRM*. It communicates the license; software *may* honor it (Adobe apps do) but nothing technically enforces it. The real authority is the **EULA** you bought. Critically: a **web font license usually does NOT grant PDF-embedding or print rights** — those are separate scopes. The "base-14" fonts (Helvetica, Times, Courier variants, Symbol, ZapfDingbats) need no embedding because every PDF viewer ships them.

Converting text to outlines (a.k.a. "create outlines", "curves", "vectorize")

Outlining

Replacing live, editable text with vector path shapes — the letters become drawings, not text.

Vector

Artwork defined by math (points and curves) so it scales to any size without blurring — unlike a photo, which is fixed pixels.

Outlining turns each glyph into anchor points and curves. No font is referenced anymore, so there's zero substitution risk and zero embedding-license problem at print time, and it looks pixel-identical everywhere. But you pay for it:

- **No longer editable** — you can't fix a typo, change size, or swap the font. Always keep an editable master file.
- **No longer searchable, selectable, or copyable**, and it breaks screen readers — it fails the PDF/UA accessibility standard. Fine for art and logos, bad for documents.
- **Loses live-text features** — underline, bullets/numbering, and OpenType features bake in or drop.
- **Tiny text suffers** — each small glyph becomes hundreds of anchor points, loses its "hinting" (instructions that keep small text crisp), can look thicker or muddy, and bloats the file. Outline headlines and logos, never body copy.

Common Mistake — Using a sketchy online "outline my text" tool that secretly *rasterizes* (turns text into pixels) instead of making true vectors. Result: jagged, blurry edges at print resolution. Real outlining stays vector.

When to embed+subset vs outline (for print)

Situation	Do this
Normal print job	Embed + subset — crisp, small, searchable, standards-compliant. The default.
Font is restricted (fsType) / can't embed	Outline it (or get a print license).
Logo or wordmark	Outline — appearance is sacred, editing isn't needed.
Printer explicitly demands outlines	Outline, but keep an editable master.
Body copy / small text	Embed + subset — never outline.

Many modern printers now *prefer* embedded fonts because outlines bloat files and harm small text. The **PDF/X** standards for print (PDF/X-1a, X-3, X-4) *require* every font to be fully embedded (subset is allowed) — no unembedded fonts permitted. Preflight checks (the Ghent Workgroup PDF/X-Plus specs, founded 2002) automatically reject files with missing fonts. So a properly produced print PDF solves this problem by design.

Pro Tip — Whether you embed or outline, always keep a separate *editable master* with live text. The print file is a one-way export; you never want it to be your only copy.

How this plays out in Print-Flow-360

PF360 keeps text **live** in the Fabric.js design studio (`designer/`) so customers can edit it. When an order exports, the Node **pdf-service** (port 4000) writes the text as SVG `<text>/<tspan>` (from Textbox / IText / CurvedText) and renders it with PDFKit via `svg-to-pdfkit` , **embedding the actual font binary** through `doc.registerFont` (see `pdf-service/src/services/pdf/fontManager.js` and `pdfGenerator.js`). So PF360 takes the embed+subset path, not outlining.

It resolves each font family in order: a tenant's uploaded `designer_fonts` (read from storage) → a stored DB URL → auto-fetching Google Fonts by name. It deliberately downloads static TTF/OTF/WOFF and avoids variable WOFF2 (the `fontkit` 1.9.x library crashes embedding variable glyphs), even spoofing a Firefox-2.0 user-agent to force a static TTF.

Critical failure mode (real, in the code) — If a family can't be resolved, the service logs "Helvetica fallback will be used" and `pdfGenerator` falls back to `'Helvetica'` (`fontManager.js:455` , `pdfGenerator.js:173-174`). That is exactly the substitution/wrong-glyph bug from the top of this section — a custom font that isn't in `designer_fonts` silently becomes Helvetica in the printed file, with no warning to the buyer.

Actionable mitigation — Make sure every font a customer can pick is actually resolvable: present in `designer_fonts` with a real upload, or a valid Google Fonts name. Treat "Helvetica fallback will be used" in the logs as a production alert, not noise. Because PF360 has *no preflight step* (a documented platform gap), there's nothing downstream to catch a fallback before it prints — so the check has to happen at upload/selection time. Outlining is only a last resort here for a font that genuinely can't be embedded.

Key takeaways —

- A font that's only *named* in a file carries no shapes — viewers substitute, causing wrong typeface, reflowed/overflowing text, or tofu boxes. On your screen it looks fine; at the RIP it prints wrong (a silent failure).
- **Embedding** packs the font into the file for identical rendering everywhere; **subsetting** embeds only used glyphs to shrink it (look for the `ABCDEF+` name prefix).
- **fsType** declares embedding permission (Installable → Editable → Preview&Print → Restricted) but is advisory metadata, not enforcement — the EULA rules, and web-font licenses rarely cover print embedding.
- **Outlining** turns text into vector shapes: no font needed, pixel-perfect, but not editable/searchable/accessible and bad for tiny body text. Use it for logos/headlines or restricted fonts only.
- Default for print = embed + subset; PDF/X standards require all fonts embedded and preflight rejects missing ones.
- PF360 embeds fonts in pdf-service via `doc.registerFont`, but unresolved custom fonts silently fall back to Helvetica — guarantee every selectable font resolves, since there's no preflight to catch it.

Section 18: PDF/X Standards — The Print-Ready Format

You have learned about CMYK color, ICC profiles, embedded fonts, bleed, and why RGB has no place at the press. Now we tie it all together with one idea that *enforces* every one of those rules in a single file: **PDF/X**. Think of this section as the "graduation exam" for everything else in the guide.

What problem does PDF/X solve?

Imagine you email a normal PDF to a print shop. It might look perfect on your screen, yet hide problems the printer only discovers *after* they've started: a font isn't included, a logo is secretly in RGB, an image links to a file that wasn't sent, or there's no information about which paper and press the colors were meant for. Each problem means a phone call, a delay, or a reprint someone has to pay for.

PDF/X exists to make those surprises impossible.

PDF/X

Short for "PDF for eXchange." It is a strict, validated *subset* of the ordinary PDF format — the same file type, just with the risky and ambiguous features forbidden. A valid PDF/X file opens in any normal PDF reader; it has simply promised to leave nothing to chance.

Subset

A smaller, rule-limited version of something bigger. PDF/X removes features from PDF rather than adding new ones — like a "child-safe" kitchen that's still a kitchen, just with the sharp knives locked away.

RIP (Raster Image Processor)

The software/hardware at the print shop that turns your PDF into the dot pattern the press actually lays down. The RIP is the audience PDF/X is written for.

PDF/X is defined by the ISO 15930 family of standards (multiple parts, one per flavor). Its guiding philosophy is **blind exchange**: the printer can drop your file into the RIP and print it with no questions, no missing pieces, no guessing.

Analogy — A normal PDF is a recipe that says "add some cheese." PDF/X is the same recipe rewritten so it *must* say "add 200g of aged cheddar, included in this box." Nothing is left to the cook's interpretation, and all ingredients ship inside the package.

The rules every PDF/X flavor enforces

No matter which flavor you pick, all of these are mandatory:

- **All fonts embedded** (full or subset). The file may never rely on the printer happening to own a typeface. If a font's license forbids embedding, it can't be used — that's the licensing trap.
- **An output intent is required** (more on this below — it's the single most defining rule).
- **Page boxes defined:** a `TrimBox` (or `ArtBox`) marking the final cut size on every page, a `BleedBox` extending past it for bleed (industry standard **3 mm / 0.125 in**), and a `MediaBox` for the full sheet.
- **No encryption or passwords** — the RIP must be able to read everything.
- **No interactive junk:** no forms, JavaScript, audio, video, or file attachments.
- **No external/linked content** (except the special X-2/X-5 partial-exchange variants). Everything is self-contained.
- **Every image has a defined, valid color space** — no ambiguous "device" data left untagged.

Bleed

Extra artwork extending past the trim line so that when the paper is cut, no thin white sliver appears at the edge if the cut drifts by a hair.

Output intent — the heart of PDF/X

If you remember one new concept from this section, make it this one.

Output intent

A piece of metadata embedded in the PDF that names the *intended printing condition* — the specific press process and paper the file was prepared for — usually via an ICC profile. It tells the RIP, "this file's colors were built for THIS press on THIS paper."

An output intent has two parts: an `OutputConditionIdentifier` (a registered name like `FOGRA39` or `CGATS TR 001`) and, in most cases, an **embedded ICC profile**. PDF/X permits only *output* (printer) profiles — unlike PDF/A, which also allows monitor profiles. The ICC profile version must be **2.0 or newer**; older profiles are rejected by the standard.

Common output-intent profiles you'll see in the wild:

- **Europe, coated offset:** Coated FOGRA39 / `ISOcoated_v2_eci` (ISO 12647-2:2004); newer Fogra51 / `PS0coated_v3`.
- **North America:** `GRACoL2006_Coated1v2`, US Web Coated SWOP v2 / `SW0P2006_Coated3v2`.

ECI (the European Color Initiative) gives these ICC profiles away free, matched to ISO 12647-2.

Key Takeaway — A missing output intent is the most common reason a file is rejected as "not valid PDF/X." Without it, the RIP has no color target — it knows the numbers but not the destination.

The flavors

PDF/X-1a — the bulletproof classic

(ISO 15930-1:2001 on PDF 1.3; revised ISO 15930-4:2003 on PDF 1.4.) The most restrictive and most universally compatible. **CMYK + spot colors only** — no RGB, no Lab, no ICC-tagged color. **No live transparency** (it must be flattened). No layers. Because it assumes nothing modern, it runs on the oldest RIPs, and many US commercial and newspaper printers still demand it.

Flattened transparency

Effects like drop shadows or "see-through" overlaps are pre-baked into solid, opaque artwork before saving — so the RIP never has to figure them out. Safe, but irreversible.

PDF/X-3 — color-managed, European favorite

(ISO 15930-3:2002 on PDF 1.3; revised ISO 15930-6:2003 on PDF 1.4.) Same as X-1a but lifts the CMYK-only rule: it *permits* calibrated RGB, Lab, and grayscale alongside CMYK/spot — provided each color carries an ICC profile or is converted through the output intent at the RIP. Still **no live transparency** (flattened) and **no layers**. Popular in Europe.

PDF/X-4 — the modern default

(ISO 15930-7:2008, base PDF 1.6, often described as roughly PDF 1.7.) Inherits X-3's color-managed rules *and* adds the two things X-1a/X-3 lack:

- **Live transparency** — blend modes and drop shadows are kept intact, with flattening deferred to the RIP. Fewer artifacts, smaller files for complex art.
- **Layers** (optional content / OCGs) — e.g. a separate die-cut or varnish layer.

In X-4 the output-intent ICC profile **must be embedded** (not just named).

A word on the rest

X-2 was never widely published and is effectively withdrawn. X-5 (ISO 15930-8:2008) is X-4 plus external references and extra (n-colorant) colors. X-6 (ISO 15930-9:2020) is the newest, built on PDF 2.0. The "**p**" variants (X-4p, X-5p, X-6p) are "partial exchange" — they let the ICC profile live *externally*, referenced rather than embedded, handy when many files share one profile.

Comparison: X-1a vs X-3 vs X-4

Property	PDF/X-1a	PDF/X-3	PDF/X-4
Color allowed	CMYK + spot only	+ RGB, Lab, ICC	+ RGB, Lab, ICC
Transparency	Flattened	Flattened	Live
Layers (OCG)	No	No	Yes
Fonts embedded	Required	Required	Required
Output intent	Required (may be named)	Required (embed if color-managed)	Required (must embed)
Base PDF	1.3 (/1.4)	1.3 (/1.4)	1.6
ISO part	15930-1:2001 (/4:2003)	15930-3:2002 (/6:2003)	15930-7:2008
Best for	Legacy / safest	European color-managed	Modern default

Why Print-Flow-360's pdf-service should target PDF/X

This is where the whole guide pays off. The Node `pdf-service` (port 4000, using sharp for images and PDFKit for PDFs) is what turns a customer's design into the file a print partner receives. Today the platform has documented gaps: **no RGB → CMYK conversion**, **no preflight checking**, and a **weak order-to-production spine**. PDF/X is the standard that forces those gaps closed, because a file can't validate as PDF/X while they exist.

Producing a plain PDF means trusting the print partner to catch every error. Producing PDF/X means the file *self-validates* and is blind-exchangeable. Concretely, PDF/X mechanically enforces the four pillars taught earlier:

- **CMYK/spot only (X-1a) or color-managed (X-4)** → no surprise RGB reaching the press.
- **Embedded ICC output intent** → the press/paper color target travels with the file.
- **All fonts embedded** → no substituted typefaces in a customer's business-card name.
- **TrimBox + BleedBox** → bleed and final cut size are explicit, not assumed.

Best Practice — Target **PDF/X-4** as the default output and keep **PDF/X-1a** as a fallback. X-4 preserves the live transparency (drop shadows, blends) that the Fabric.js design studio (`designer/`) can produce, so nothing is prematurely flattened, and it suits modern digital presses. X-1a is the safe option for legacy/offset partners that reject transparency — flatten and strip to CMYK for them.

Real-world case — A customer designs a flyer in the studio with a drop-shadowed logo and a photo. The brand-color theming gave everything tidy on-screen RGB values. If `pdf-service` exports a plain PDF, the print partner may flatten the shadow badly and the RGB photo prints dull and color-shifted. If it instead exports PDF/X-4 with an embedded `ISOcoated_v2_eci` output intent, the shadow stays live, the photo carries its profile, fonts are embedded, and a 3 mm bleed box is declared — the partner RIPs it with zero questions.

Top real-world mistakes

Common Pitfalls —

- Leaving RGB images inside an X-1a file (invalid — X-1a is CMYK/spot only).
- Using a font whose license forbids embedding → substitution at the press.
- No bleed — elements get cut off at trim. Always extend 3 mm / 0.125 in.
- **Missing output intent** — the RIP has no color target.
- Low resolution — aim for 300 DPI photos, 1200 DPI line art; 72–96 DPI prints pixelated.
- Sending X-4 live transparency to a RIP that only accepts X-1a.
- External/linked images that break in transit.
- Expecting "PDF/X" alone to fix bad source color — it validates *structure*, not *design quality*.

Key takeaways —

- PDF/X is a strict, ISO-15930 subset of PDF that guarantees a file is print-ready for "blind exchange" — same file type, fewer risky features.
- Every flavor requires embedded fonts, an output intent, and TrimBox/BleedBox; none allow encryption, JavaScript, or unresolved external content.
- The output intent — an embedded printer ICC profile (version ≥ 2.0) naming the press/paper — is the defining, most-often-missed requirement.
- X-1a = CMYK/spot only, flattened, safest/legacy. X-3 = adds color-managed RGB/Lab, still flattened. X-4 = color-managed + live transparency + layers = modern default.
- PF360's `pdf-service` should target PDF/X-4 (fallback X-1a): it mechanically enforces CMYK, ICC profiles, font embedding, and bleed — exactly the platform's documented production gaps.
- PDF/X validates the file's structure, not the quality of the design — garbage in still prints as garbage.

Section 19: Typography & Text Rendering

Text is the part of a design people actually *read*. A logo can be vague and still work; a price, a phone number, or a paragraph of legal fine print cannot. This section teaches you how text really works inside a computer and on paper — from the letters you type, to the shapes that get drawn, to the pixels and ink that finally appear. We start with the most basic and most-confused idea: the difference between a character and a glyph.

1. Characters vs glyphs — the foundation

Character

The abstract *meaning* of a letter or symbol — the thing you type and store. The computer records it as a **Unicode code point**, a universal ID number. The capital letter A is `U+0041` everywhere on Earth.

Glyph

The actual *drawn shape* on screen or paper. One character can be drawn many different ways depending on the font.

The relationship between characters and glyphs is **not one-to-one**, which trips up almost every beginner:

- **One character** → **many glyphs**: the letter "a" can be drawn as a normal "a", a small-cap "A", or a fancy swash alternate.
- **Many characters** → **one glyph**: the two characters "f" and "i" often join into a single *ligature* shape "fi".
- **One character** → **zero glyphs**: a combining accent mark may have no standalone shape; it just modifies its neighbour.

Inside the font, a table called the **cmap** ("character map") translates each Unicode code point into a default glyph. Then the font's layout features can substitute or reposition glyphs from there.

Common Mistake — Assuming "the font has the letter" is enough. A code point can be mapped but the font may have no glyph for it — you then see `.notdef`, the "tofu" box `□`. A storefront brand that uses an emoji or a rare currency symbol in a font that lacks it will print a box, not the symbol.

2. Font formats — what those file extensions really mean

A font is a file full of glyph outlines plus metric tables. Over 40 years the industry settled on a family of formats:

Format	Outline math	Notes
PostScript Type 1 (.pfb/.pfm)	Cubic Bézier	Adobe, 1984. The deprecated ancestor — end-of-life in Adobe apps since Jan 2023 ; these files no longer render. You'll only meet them in old archives.
TrueType (.ttf)	Quadratic Bézier (B-spline)	Apple + Microsoft, late 1980s. Famous for powerful "bytecode" hinting.
OpenType (.otf or .ttf)	Either type	Adobe + Microsoft, 1996. The modern umbrella format.
WOFF / WOFF2	Either	Compressed OpenType for the web. WOFF2 uses Brotli compression, ~30% smaller than WOFF.

Bézier curve

A smooth curve defined by a few "control points" — the math used to describe every letter outline. *Cubic* curves use more control points (smoother, heavier); *quadratic* use fewer (lighter, faster).

The key teaching point about OpenType: OpenType is a *container*, not a competitor to TrueType. An OpenType font wraps *either* outline type: TrueType/quadratic outlines (stored in a `glyf` table, usually saved as `.ttf`) or PostScript/CFF cubic outlines (Compact Font Format, usually saved as `.otf`). So:

Key Takeaway — `.ttf` vs `.otf` tells you the *outline type inside*, not "TrueType vs OpenType." Both files are OpenType. OpenType added Unicode (65,536+ glyphs, vs the old 256-glyph limit), one cross-platform file for Mac and Windows, and the advanced layout engine described next.

Variable fonts — one file, infinite weights

Added to OpenType in v1.8 (September 2016), a **variable font** stores a whole continuous "design space" in a single file. Instead of shipping 12 separate files (Light, Regular, Bold, ...), one file interpolates between master designs along named **axes** (lowercase 4-character tags):

- `wght` — Weight (1–1000)
- `wdth` — Width (percentage)
- `ital` — Italic (0 or 1)
- `slnt` — Slant (degrees, typically –90 to 90)
- `opsz` — Optical Size (in points)

Custom axes use UPPERCASE tags. Adoption climbed from ~11% of web pages in 2020 to ~33–34% in 2024 because one small file replaces dozens. For PF360's storefront, a variable font lets a theme nudge heading weight without downloading another file — fewer requests, faster pages.

3. The OpenType layout engine — how features work

Two tables do the clever typography, both keyed by 4-character *feature tags*:

GSUB (Glyph Substitution)

Decides *which* glyphs to use — swap "f"+"i" for the "fi" ligature, swap digits for small caps, etc.

GPOS (Glyph Positioning)

Decides *where* glyphs sit — kerning pairs, stacking accents onto letters.

The text shaper runs **GSUB first, then GPOS** (pick the glyphs, then position them). Useful tags:

- **GSUB**: `liga` standard ligatures (fi, fl — on by default), `dlig` discretionary ligatures (st, ct — off by default), `smcp` small caps, `onum` / `lnum` oldstyle vs lining figures, `frac` fractions, `salt` / `ssNN` stylistic alternates/sets, `swsh` swashes.
- **GPOS**: `kern` kerning, `mark` accent placement. Modern kerning lives in GPOS, not the old legacy `kern` table.

4. From outlines to pixels — the rendering pipeline

```
[glyph outline]  ->  scale to size  ->  apply HINTS
  (Bézier pts)      (ppem grid)      (snap to grid)
      |              |
      v              v
  RASTERIZE  ->  anti-alias / subpixel  ->  composite
  (fill to pixels)  (ClearType RGB ~3x)  (on screen)
```

em / UPM (units per em)

The font's internal design grid. All measurements are in these units — typically 1000 (PostScript/CFF) or 2048 (TrueType) — then scaled by your point size.

ppem (pixels per em)

How many pixels tall the em box becomes at the chosen size. Small ppem = few pixels = hard to render cleanly.

Rasterize

Convert the vector outline into a grid of filled pixels.

Hinting

Hinting means instructions baked into the font that gently distort outlines to snap onto the pixel grid at small sizes / low resolution, so stems stay even and letters stay readable. TrueType hinting is a stack-based assembly *bytecode* run by an interpreter (a tiny virtual machine), stored in `glyf` / `prep` / `fpgm` tables — fonts like Verdana have thousands of hand-tuned lines. PostScript/CFF hinting is lighter and more automatic.

Common Mistake — Worrying about hinting for *print*. Hinting matters below ~24px on screens and low-DPI displays. At print resolution the rasterizer has so many pixels that hinting is nearly irrelevant. PF360's [pdf-service](#) (PDFKit + sharp at port 4000) renders at high resolution — don't burn time tuning hints there.

5. Typographic geometry — the text-tool vocabulary

```
ascender --- b d l <- tops above x-height
cap-height- B D H <- top of capitals
x-height --- x o n e <- top of lowercase
baseline === x o n e B D b d l <- glyphs sit here
descender -- g p y <- tails below baseline
```

Baseline

The invisible line glyphs rest on.

x-height

Height of a lowercase "x". A tall x-height reads better at small sizes.

Cap-height

Top of capital letters (usually a touch below the ascender height).

Ascender / Descender

Parts rising above x-height (b, d, h, k, l) / dropping below baseline (g, j, p, q, y).

Counter, bowl, overshoot

The enclosed space in "o"; the round stroke of "b"; the tiny amount round glyphs exceed the lines so they look optically equal to flat ones.

The classic spacing trio (clear this up once and for all)

Leading (line-spacing)

Vertical distance from one baseline to the next. Named after the lead metal strips printers slid between lines. Rule of thumb: point size + 2–4 pt, or 120–150% of size (12 pt body → ~14.4–18 pt). Longer lines need *more* leading.

Tracking (letter-spacing)

Uniform horizontal space across a whole run/word/line. Tighten big display headlines; add some to CAPS and small caps; never heavily track lowercase body text.

Kerning

Space between one *specific pair* of letters — fixing optical gaps in "AV", "To", "WA", "Ty", "Yo", "LT". *Metric* kerning uses the font's GPOS pairs; *optical* kerning is computed by the app.

Workflow order: leading → tracking → kerning (macro to micro).

Common Mistake — Calling tracking "kerning." Kerning is *pairwise only*. A control that spaces the whole line evenly is tracking, no matter what it's labelled.

6. Print legibility — the numbers that matter

- **Body text:** 9–12 pt (10–11 pt is common). Around 8 pt is the floor where reading speed drops.
- **Line length (measure):** 50–75 characters per line, 66 ideal. Too long and the eye loses the line on return.
- **Leading:** 120–150% of size. Size, measure, and leading are coupled — change one, retune the others.
- **Black ink:** body/small text should print as **100% K only** (single-channel black), not rich 4-color black — registration misalignment fuzzes small 4-color text. Reserve rich black for large display type.
- Avoid thin serifs and hairlines in reverse (white on dark) — ink gain fills the counters. Keep legal fine print to ~5–6 pt minimum.
- **Embed or outline all fonts** in the print PDF (PDF/X) so the print engine (RIP) never substitutes a wrong font. Ligatures and kerning must survive flattening.

Real-world case — A PF360 customer types a phone number in 7 pt black using a default rich-black fill. On the proof it looks crisp (screen is RGB). At the press, the cyan/magenta/yellow/black plates are a hair out of register and the number prints fuzzy and ghosted. The fix: 100% K only, bump to 9 pt. Because the platform currently has **no RGB → CMYK conversion and no preflight**, nothing catches this automatically — it must be caught by design guidance and review.

7. PF360 designer text tool — the controls users see

The design studio (`designer/`) is built on Fabric.js. The text controls live in

`designer/src/views/Editor/CanvasRight/Components/ElementText.vue` (toolbar) and `ElementStylePanel/TextboxStylePanel.vue` (style panel), with config lists in

`src/configs/texts.ts` (`FontSizeLibs` , `LineHeightLibs` , `CharSpaceLibs`). Visible controls: font color picker; Font size +/-; Bold / Italic / Underline; horizontal align Left/Center/Right; vertical align Top/Middle/Bottom; and a **"Spacing"** row exposing line height and character spacing.

UI label / control	Typographic term	Implementation note
Spacing → line height	Leading	Fabric <code>lineHeight</code> is a <i>unitless multiplier</i> (default 1.16) — relative leading, not absolute points.
Spacing → char spacing	Tracking	Fabric <code>charSpacing</code> is measured in 1/1000 em , not px or pt; PF360 steps it ± 10 . Per-character overrides are cleared (<code>clearPerCharCharSpacing</code>) so the global value applies uniformly.
(none)	Kerning	No true pairwise kerning UI — Fabric kerns by font metrics only. The "Spacing" control is <i>tracking</i> .

Common Mistake / UX flag — Per the project's plain-language rule, a single "Spacing" control hides two different concepts (leading and tracking) and risks being mistaken for kerning, which PF360 doesn't expose at all. Clearer labels would be "Line spacing" and "Letter spacing" — never "Kerning," since no pairwise control exists.

Best Practice — When preparing print-bound text in the designer: keep body text 9 pt+, set line spacing around 1.3–1.5, use letter spacing sparingly (tighten headlines, loosen CAPS), and choose a font with a generous x-height for small labels. Because there's no automated preflight or font-embedding guarantee in the order-to-production spine yet, treat these as manual checks before a job is sent to print.

Key takeaways —

- **Characters are meaning, glyphs are shapes** — the mapping is not 1:1; a missing glyph shows a ☐ tofu box.
- **OpenType is the umbrella format.** `.ttf` vs `.otf` = which outline math is inside (quadratic/glyf vs cubic/CFF), not "TrueType vs OpenType." PostScript Type 1 is dead since 2023.
- **Variable fonts** pack many weights/widths into one file along axes (`wght` , `wdth` , `ital` , `slnt` , `opsz`) — fewer downloads for the storefront.
- **GSUB picks glyphs, GPOS positions them;** rendering goes outline → scale → hint → rasterize → anti-alias. Hinting is a screen/low-DPI concern, not a print one.
- **Leading is vertical, tracking is uniform horizontal, kerning is pairwise.** Don't confuse them; work macro to micro.
- **For print:** 9–12 pt body, 50–75 chars/line, 120–150% leading, 100% K for small black text, embed fonts. PF360 has no auto RGB → CMYK or preflight — these are manual checks.
- **In the PF360 designer,** "Spacing" = Fabric `lineHeight` (unitless leading multiplier) + `charSpacing` (tracking in 1/1000 em); there is no kerning control — label accordingly.

Glossary of Terms

This glossary collects every important term from the guide in one place, in alphabetical order. Each definition is written in plain English so you can look something up without re-reading a whole chapter. Where it helps, the entry connects the concept to the Print-Flow-360 stack: the Fabric.js **design studio** ([designer/](#)), the Node **PDF service** ([pdf-service](#)), port 4000, using [sharp](#) for images and [PDFKit](#) for PDFs), and the storefront's brand-color theming.

Additive color

Making colors by adding beams of light together. Start with black (no light) and add red, green, and blue to build up to white. This is how screens work — the model behind RGB.

Alpha channel

An extra channel stored alongside the color channels that records how opaque or transparent each pixel is, from fully see-through to fully solid. A logo on a transparent background owes that transparency to its alpha channel.

Anti-aliasing

Smoothing the jagged "staircase" edges of text and shapes by adding partly-shaded pixels along the boundary, so a curve looks smooth instead of blocky.

Baseline

The invisible line that letters sit on. Most characters rest on it; parts like the tail of a "y" hang below it (the descender).

Bézier curve

A smooth curve defined by a few control points and "handles" rather than by listing every pixel. It is the math behind vector shapes and the outlines of every font glyph.

Bicubic resampling

A high-quality way to resize an image: each new pixel is calculated by blending a neighborhood of surrounding pixels with a smooth curve. It looks better than simpler methods, which is why [sharp](#) in the PDF service uses high-quality resampling by default.

Bleed

Extra artwork that extends past the final trimmed edge of the page (usually [3 mm](#) or [0.125 in](#)). It exists so that slight cutting inaccuracies never leave a thin white sliver at the edge.

Blend mode

A rule for how a layer's colors mix with the layers beneath it — for example *Multiply* darkens, *Screen* lightens. Most blend modes assume RGB math, which is one reason they need careful handling before print.

Choke

A type of trap where the lighter background is pulled slightly inward over a darker foreground object, hiding gaps caused by misaligned print plates. The opposite of a spread.

CMM (Color Management Module)

The software engine that does the actual math of converting color from one device's profile to another's, using the PCS as the meeting point. Your operating system, Photoshop, and a RIP each contain a CMM.

CMYK

The four printing inks: Cyan, Magenta, Yellow, and Key (black). It is a subtractive model — inks absorb light, and more ink means a darker result. All physical printing presses think in CMYK.

Color depth (bit depth)

How many bits are used to record each pixel's color, which sets how many distinct shades are possible. 8 bits per channel gives 256 levels each (about 16.7 million colors); 16-bit gives far more, useful for smooth gradients without banding.

Color management

The whole system of profiles and conversions that keeps a color looking consistent as it moves from camera to screen to printer, instead of shifting at every step.

Color space

The specific range and arrangement of colors a device or standard can represent, such as sRGB or US Web Coated (SWOP) for offset printing. Two devices can both be "RGB" yet have different color spaces.

Colorimetric intent

A rendering intent that keeps in-gamut colors exactly accurate. *Relative* colorimetric remaps the white point and clips out-of-gamut colors to the nearest reproducible ones (best for most print work); *absolute* reproduces the paper white too (used for proofing one paper on another).

Content stream

The part of a PDF that holds the actual drawing instructions — "draw this line, place this text here, fill this shape." The page you see is the result of the viewer executing this stream.

Crop marks (trim marks)

Short thin lines printed just outside the page corners that show the cutter exactly where to slice the paper to the final size.

Dot gain

The tendency of halftone dots to grow as ink spreads and soaks into paper, making printed areas darker than intended. Profiles compensate by laying down slightly less ink in advance.

DPI (dots per inch)

How many ink dots a printer lays per inch. It describes the output device, and is often higher than the image's PPI because many tiny dots combine to simulate one pixel's color.

Flattening (transparency flattening)

Converting overlapping transparent layers, blend modes, and soft shadows into plain opaque shapes (or a single rasterized image) that older print workflows can handle, since classic CMYK printing has no concept of transparency.

Font embedding

Packing the actual font file inside the PDF so the document prints with the exact typeface the designer chose, even on a machine that doesn't have that font installed.

Gamut

The complete set of colors a device or color space can actually produce. Screens have a wide gamut; CMYK printing has a smaller one, so some bright screen colors simply can't be printed.

Glyph

The drawn shape of a single character in a font — the visible form of the letter "a." One character can map to several glyphs (for example, alternate styles or ligatures).

Halftone

The trick of simulating shades of gray or smooth color tones using a grid of tiny dots of solid ink that vary in size. From a distance your eye blends them into continuous tone.

ICC profile

A standardized data file that describes exactly how one device reproduces color, defined by the International Color Consortium. It is the "dictionary" a CMM uses to translate colors between devices.

Kerning

Adjusting the space between two specific letters so the spacing looks even — for example tucking a "T" and "o" closer together. Different from tracking, which spaces a whole run uniformly.

Knockout

Removing (cutting a hole in) the background color wherever a foreground object sits, so the inks don't overlap and mix. The default behavior, and the opposite of overprint.

Lab (CIELAB)

A device-independent color model that describes color the way the human eye perceives it, using Lightness plus two color axes (a, b). Because it doesn't depend on any device, it is used as the neutral reference for conversions (the PCS).

Leading

The vertical space between lines of text, measured baseline to baseline. Named after the strips of lead once placed between rows of metal type.

LPI (lines per inch)

The resolution of a halftone screen — how many rows of dots per inch. Newspapers use a coarse screen (~85 LPI); glossy magazines use a fine one (~150–175 LPI).

Moiré

An unwanted wavy or grid-like pattern that appears when halftone screens are angled badly or when a printed pattern is re-scanned. Correct screen angles for each ink prevent it.

OpenType

A modern, cross-platform font format that can hold thousands of glyphs plus smart features like ligatures and alternates. It supersedes older formats and works the same on Windows and Mac.

Outlines (converting text to outlines)

Turning live text into plain vector shapes so it no longer needs the font. It guarantees the look but makes the text uneditable and unsearchable, so it's a last resort, not a substitute for embedding.

Output intent

A tag inside a PDF/X file that declares the exact printing condition (paper and press profile) the file was prepared for, so everyone downstream knows how the CMYK numbers should look.

Overprint

Printing one ink directly on top of another instead of knocking out the background. Used deliberately for effects and trapping; a common cause of surprises when "overprint preview" is off.

Pantone (PMS)

The Pantone Matching System — a catalog of premixed spot-color inks, each with a number (like **PMS 185**). Brands specify them so a logo color stays identical across every printer and material.

PCS (Profile Connection Space)

The neutral, device-independent color space (Lab or XYZ) that ICC profiles translate through. Color goes from the source profile into the PCS, then out to the destination profile — like translating two languages through a common one.

PDF content stream

See *Content stream* — the sequence of drawing operators that paints a PDF page.

PDF/X

A family of restricted PDF standards built specifically for reliable printing. They forbid risky things (like missing fonts or unspecified color) and require an output intent, so the file prints predictably anywhere.

PDF/X-1a

A strict, older PDF/X flavor that requires all color to be already converted to CMYK (or spot) and all fonts embedded, with no transparency. Very safe but inflexible — everything must be print-ready up front.

PDF/X-4

A modern PDF/X flavor that allows live transparency and color management (so files can stay smaller and editable longer) while still embedding fonts and declaring an output intent. The preferred target for most current print workflows.

Perceptual intent

A rendering intent that gently compresses the whole image's colors to fit the smaller print gamut, preserving the relationships between colors. Best for photographs, where smooth gradients matter more than exact values.

PPI (pixels per inch)

How many pixels of an image are packed into each printed inch. It describes the image data; 300 PPI is the standard for sharp print because at that density individual pixels become invisible to the eye.

Preflight

An automated check of a file before printing that flags problems — low resolution, missing fonts, RGB images, no bleed, ink too heavy. Print-Flow-360 currently has no built-in preflight, so customer files can reach production with these defects undetected.

Premultiplied alpha

A way of storing transparent images where each pixel's color values are already multiplied by its opacity. It speeds up compositing but must be "un-multiplied" correctly, or semi-transparent edges pick up dark or bright fringes.

Process color

Color built from the four standard CMYK inks, as opposed to a premixed spot ink. Photographs and full-color artwork are reproduced with process color.

Raster (bitmap)

An image made of a fixed grid of pixels, like a photo. It has a set resolution, so enlarging it past that point makes it look blurry or blocky.

Rasterization

Converting vector shapes or text into a grid of pixels for display or output. It happens whenever a resolution-independent design is turned into something a screen or print head can actually paint.

Registration black (registration color)

A special "color" set to 100% of every plate (C, M, Y, and K) used only for printer's marks so they print on every plate. Never use it for actual design elements — it dumps far too much ink.

Registration marks

Small crosshair-in-a-circle targets placed outside the page that let the press align each color plate exactly on top of the others.

Rendering intent

The chosen strategy for handling colors that exist on the source device but not on the destination — whether to shift everything to keep relationships natural (perceptual) or keep what fits exact and clip the rest (colorimetric).

Resampling

Changing an image's pixel count — adding pixels when enlarging or removing them when shrinking — by calculating new pixel values from the old ones. Upsampling can't invent real detail, so it never truly fixes a low-resolution image.

RGB

Red, Green, Blue — the additive color model used by screens, cameras, and the web. Print-Flow-360's design studio and theming work in RGB, and the platform does not convert RGB to CMYK, so color shifts can occur at the printer.

Rich black

A deeper black made by adding some cyan, magenta, and yellow under the black ink (for example **C60 M40 Y40 K100**) instead of plain **K100** . Used for large solid areas so they look truly dark rather than washed-out gray.

Safe zone (safe area)

An inner margin (commonly **3–5 mm** from the trim) where you keep important text and logos, so nothing critical gets clipped if the cut drifts slightly inward.

Screening

The overall process of converting continuous tones into a printable pattern of dots — including the choice of halftone shape, frequency (LPI), and angle for each ink.

Slug

An area outside the bleed used for production notes, job names, or instructions to the printer. It is trimmed off and never appears on the finished piece.

Soft proof

Simulating on your screen how a file will look once printed, by applying the destination print profile to the on-screen preview. It lets you catch gamut and color problems before wasting paper.

Spot color

A single premixed ink printed from its own plate, rather than built from CMYK. It gives exact, consistent color (Pantone inks are spot colors) and can produce colors CMYK can't, like bright oranges or metallics.

Spread

A type of trap where a lighter foreground object is expanded slightly outward into the darker background, overlapping the edge to hide registration gaps. The opposite of a choke.

Subsetting

Embedding only the specific glyphs a document actually uses instead of the entire font, which keeps the PDF small while still printing correctly.

Subtractive color

Making colors by subtracting (absorbing) wavelengths of light with pigments or inks. Each layer of ink removes more light, heading toward black — the model behind CMYK printing.

TAC (Total Area Coverage / total ink coverage)

The sum of all four CMYK percentages in the heaviest part of a file (so the maximum is 400%). Presses cap it — often around **240–300%** — because too much ink won't dry, smears, and sticks pages together.

Tracking

Adjusting the spacing evenly across a whole word, line, or block of text — for example loosening letterspacing on a small-caps heading. Unlike kerning, it isn't pair-specific.

Trapping

Deliberately overlapping the edges of adjacent colors by a tiny amount so that if the print plates shift slightly, no white gap shows through between them.

Trim

The final cut line — the actual finished size of the printed piece after the bleed has been sliced away.

TrueType

A long-established outline font format (originally from Apple and Microsoft) that describes glyphs with quadratic curves. Reliable and widely supported, though newer designs often ship as OpenType.

Vector

Artwork defined by math — points, lines, and Bézier curves — rather than pixels. It scales to any size without losing sharpness, which is why logos and type are kept as vectors. The Fabric.js design studio works largely with vector objects.

xref (cross-reference table)

An index near the end of a PDF that lists the byte position of every object in the file, letting a viewer jump straight to any page or resource without reading the whole document. A corrupt xref is a common cause of "damaged PDF" errors.

Pro Tip — Treat this glossary as a checklist when reviewing a print file: confirm the color space (RGB vs CMYK), the resolution (aim for 300 PPI), the presence of bleed and trim marks, embedded and subsetted fonts, a sensible TAC, and a declared output intent. Those six checks catch the majority of real-world print failures.

Common Mistake — Assuming a file that "looks fine on screen" will print fine. Screens are additive RGB with a wide gamut; print is subtractive CMYK with a smaller one and ink limits. Without color management, a soft proof, and preflight, the safe-on-screen file can shift color, run too heavy on ink, or lose thin text at the trim edge.

Key takeaways —

- **Two color worlds.** Additive RGB (light, screens, the design studio) versus subtractive CMYK (ink, presses). Lab is the neutral referee, and ICC profiles plus a CMM translate between them through the PCS.
- **Gamut and intent drive accuracy.** When colors don't fit the print gamut, the rendering intent (perceptual for photos, relative colorimetric for most else) decides how they're remapped — and a soft proof lets you preview the result.
- **Resolution is two numbers.** PPI describes the image (target 300), DPI/LPI describe the output device; halftoning and screening turn smooth tone into printable dots, with TAC and dot gain limiting how much ink lands.
- **Geometry and safety.** Bleed, trim, safe zone, and the various marks exist because cutters and plates are never perfectly precise; trapping (spread and choke) covers the same imperfection for adjacent colors.
- **The PDF must be self-contained.** Embedded and subsetted fonts, flattened or properly tagged transparency, a valid xref, and a PDF/X output intent are what make a file truly print-ready — exactly the validations a preflight step (a documented gap in Print-Flow-360) would enforce.

Frequently Asked Questions

This section answers the questions that come up again and again when people first move from "designing on a screen" to "printing on paper." Screens and presses are two very different machines, and most print headaches come from forgetting that. We'll start with the basics, then work up to the file-format and production questions a developer building a print SaaS like Print-Flow-360 will hit. Every technical word is defined the first time it appears.

Color and how it changes when printed

Q: Why do my colors look duller when printed?

Because your screen and your printer make color in opposite ways. A screen *emits* light and mixes red, green and blue (**RGB**) to build bright, glowing color. A press *absorbs* light: it lays down cyan, magenta, yellow and black inks (**CMYK**) on paper that reflects whatever the ink doesn't soak up. Ink simply can't reproduce the most vivid screen colors — especially bright greens, oranges and electric blues — so they shift toward a muddier, calmer version. This isn't a defect; it's the physical limit of ink on paper, called a smaller **gamut** (the range of colors a device can produce).

RGB

Red/Green/Blue — the additive color model screens use. Mixing all three at full strength makes white.

CMYK

Cyan/Magenta/Yellow/Black — the subtractive model presses use. The "K" (key) is black ink. Mixing all makes a muddy dark, so black is added separately.

Gamut

The full set of colors a given device can actually produce. Screens have a wide gamut; CMYK printing has a narrower one.

Analogy — RGB is like stage lighting (start in darkness, add colored light to brighten). CMYK is like painting on white paper (start bright, add pigment that darkens and dulls). You can't paint your way to a color brighter than the paper.

Q: RGB or CMYK — which should I use for my files?

If a file is going to print, the press ultimately needs **CMYK**. The safest workflow is to design in CMYK from the start so what you see roughly matches what prints. If you design in RGB (which most web and screen tools, including Fabric.js, do), *something* has to convert RGB to CMYK before printing — and that conversion decides how your bright colors get tamed. For web, app and email use, keep RGB. For anything physically printed, plan for CMYK.

Common Mistake — Print-Flow-360 today — The design studio ([designer/](#) , built on Fabric.js) works in RGB, and the platform has **no automatic RGB → CMYK conversion**. That means RGB values get handed straight toward production, and the printer's **RIP** (Raster Image Processor — the software that turns a file into press-ready dots) does an uncontrolled, last-second conversion. Colors can shift unpredictably between proof and print. Until a managed conversion step exists, expect customer color complaints and set expectations in the UI.

Q: Why is my black not really black on press?

There are two kinds of black. **100% K** means only black ink — it looks flat and slightly gray over large areas. **Rich black** adds cyan, magenta and yellow underneath the black to deepen it (a common recipe is C40 M30 Y30 K100). Use 100% K for small body text so it stays crisp; use rich black for large solid areas and big headlines so they look truly dark. If your "black" prints washed out, you probably sent 100% K where rich black was needed — or the reverse caused fuzzy text from **misregistration** (the four ink plates not lining up perfectly).

Q: Why did my white text disappear?

On a press, white is usually *the absence of ink* — it's just bare paper showing through. So "white" only works when there's a colored background to surround it. If white text sits on a white or empty background, nothing prints and the text vanishes. (True white ink exists, but it's a special process you must request explicitly.) A second cause is **overprint**: a setting that tells an object to print *on top of* what's beneath instead of knocking a hole for itself. White set to overprint becomes invisible because it mixes into the background.

Resolution and image quality

Q: What resolution do I need for print?

The standard for print is **300 PPI** (pixels per inch) at the final printed size. Screens only need about 72–96 PPI, which is why a web image that looks sharp online turns into a blurry, blocky mess when printed large. Always check resolution *at the size it will be printed*: a 1000×1000-pixel image is 300 PPI at about 3.3 inches, but only 100 PPI (too low) if stretched to 10 inches.

PPI / DPI

Pixels Per Inch (image) / Dots Per Inch (printer). Loosely used interchangeably; both describe how much detail is packed into each printed inch.

Vector vs Raster

Raster images (photos, PNG, JPG) are a fixed grid of pixels and blur when enlarged. Vector art (logos, type, SVG) is math-based and stays razor-sharp at any size.

Pro Tip — Print-Flow-360 — The PDF service (`pdf-service` , port 4000) uses **sharp** for image work, so it's the right place to add a server-side resolution check: read each uploaded image's pixel dimensions, compare against the placement size in the design, and warn the customer before checkout if it falls below ~300 PPI. Catching low-res uploads early prevents reprints and refunds.

Layout: bleed, trim and safe area

Q: What is bleed, and why do I keep getting thin white edges?

Presses print on large sheets and then **cut** them down to the final size — and that cut is never perfectly accurate (it can drift by a millimetre or two). **Bleed** is extra artwork extended *past* the cut line so that, even if the blade lands slightly inside your design, ink still reaches the very edge. White slivers appear when your background stops exactly at the trim line; the cutter strays outward and reveals bare paper. The standard bleed is **0.125 inch (3 mm)** on every side. Keep important text and logos inside the **safe area** (about 3 mm inside the trim) so nothing critical gets sliced off.

```
+-----+ <- Bleed edge (art extends to here)
| . . . . . |
| . +-----+ . | <- Trim line (final cut)
| . | +-----+ | . |
| . | | SAFE AREA | | . | <- Keep text/logos inside
| . | | (text here) | | . |
| . | +-----+ | . |
| . +-----+ . |
+-----+
```

Real-world case — A 3.5"×2" business card with a colored background should be built at 3.75"×2.25" (adding 0.125" bleed on all four sides). The PDF service's PDFKit step should generate the document at that bleed-inclusive size and ideally draw **crop marks** (small corner lines showing where to cut), so the print partner knows the exact trim.

PDF, fonts and text problems

Q: My text reflowed or changed font in the PDF — why?

A PDF only displays a font correctly if that font travels *inside* the file. This is called **embedding**. If a font isn't embedded, the viewer (or the press's RIP) substitutes a different font it does have, which changes letter widths and makes text reflow, overlap, or shift. The fix is to embed every font when generating the PDF. With PDFKit in the PDF service, register the actual font file with `registerFont()` and use it — don't rely on a font merely being installed on the rendering machine.

Q: Should I outline (convert to curves) my fonts?

Outlining turns each letter into a vector shape, so it no longer needs the font file at all — it's now just artwork. This guarantees the text looks identical everywhere, which is handy for logos and when you're unsure the print shop has your font. The trade-offs: outlined text can't be edited or spell-checked later, and very small outlined type can lose the subtle hinting that keeps it crisp. Rule of thumb: **embed by default; outline display type and logos when font portability is a risk.**

Q: What's the difference between PDF/X-1a and PDF/X-4?

Both are **PDF/X** standards — versions of PDF specifically locked down for reliable printing (all fonts embedded, color spaces defined, no risky interactive junk). The difference is what they allow:

Feature	PDF/X-1a	PDF/X-4
Color	CMYK + spot only (no RGB)	CMYK, spot <i>and</i> managed RGB
Transparency	Must be flattened (merged into flat layers)	Live transparency kept
Best for	Older, simplest, most foolproof workflows	Modern RIPs, color-managed shops
Era / PDF base	Older (PDF 1.3)	Newer (PDF 1.6, ISO standard)

Use **PDF/X-1a** when you want maximum compatibility and zero surprises (everything pre-converted to CMYK). Use **PDF/X-4** when your print partner has a modern color-managed setup. Always ask the printer which they prefer.

Spot color

A pre-mixed named ink (e.g. a Pantone) printed from its own plate, used for exact brand colors that CMYK can't hit.

Flattening

Merging transparent/overlapping objects into solid layers so old RIPs render them predictably.

Catching problems before they print

Q: What does preflight check?

Preflight is an automated inspection of a file *before* it goes to press — like a pilot's pre-flight checklist, hence the name. A good preflight flags: low-resolution images, fonts that aren't embedded, RGB or spot colors where CMYK is required, missing bleed, hairline rules too thin to print, total ink coverage that's too heavy (over ~300% causes smudging), and overprint/white-ink traps. It turns "we'll find out when it prints" into "we caught it at upload," saving wasted material and angry customers.

Common Mistake — Print-Flow-360 gap — The platform currently has **no preflight step**, so broken files (low-res photos, non-embedded fonts, RGB-only color, no bleed) can pass straight through checkout into production. The natural home for preflight is the PDF service: sharp can audit image resolution and color mode, and PDFKit/PDF metadata can verify embedding and page geometry. Even a minimal "block on low-res + missing bleed" check would prevent the most common reprints.

Q: Why does the right file still print wrong — isn't a clean PDF enough?

A clean file is necessary but not sufficient. Print is a *production chain*: file → preflight → color conversion → imposition (arranging multiple copies on a sheet) → press → cut → finishing → ship. A weak handoff anywhere breaks the result. In Print-Flow-360 today the **order-to-production spine is thin** — there's limited automated flow from a placed order to a press-ready, verified job. Practically, that means a developer should treat preflight, color conversion, and a structured production handoff as connected work, not three unrelated features.

Q: My storefront brand color looks fine on the website but wrong on print proofs — why?

The storefront's brand-color theming is an **RGB hex value** (e.g. **#1A73E8**) chosen to look good on screens. That same hex has no guaranteed CMYK equivalent on paper, and without managed conversion it'll shift. If brand-color accuracy matters on printed products, you need either a defined CMYK recipe or a spot color for that brand — not just the website hex. This is the same root cause as the duller-colors question, applied to branding.

Key takeaways —

- Screens (RGB, additive, wide gamut) and presses (CMYK, subtractive, narrow gamut) are different machines — duller print colors are physics, not a bug.
- Use **300 PPI at final size** for images; vectors and embedded/outlined fonts stay sharp.
- Add **0.125" (3 mm) bleed** and keep critical content in the safe area to avoid white edges and clipped text.
- Embed fonts (or outline logos), pick **PDF/X-1a** for foolproof CMYK or **PDF/X-4** for modern color-managed shops, and remember white is usually bare paper.
- Print-Flow-360 has real gaps to design around: no RGB → CMYK conversion, no preflight, and a thin order-to-production spine — the PDF service (sharp + PDFKit, port 4000) is the right place to build the resolution checks, bleed/crop-mark generation, and preflight that close them.

Revision Cheat Sheet

One-page fast recall for the whole guide. Skim before you design, while you preflight, or when something prints wrong. Every term is defined once in plain words, then used as shorthand. Numbers here are industry-standard defaults — verify against your printer's spec sheet when you have one.

The 30-Second Mental Model

RGB

Light added together. Red+Green+Blue. Used by screens, cameras, the web. Big, bright color range. This is what your design starts as.

CMYK

Ink that subtracts light. Cyan+Magenta+Yellow+Key(black). Used by printers. Smaller color range — bright RGB blues/greens can't be printed.

Lab

The neutral referee. A device-independent color space that describes color as a human eye sees it. Color management uses Lab as the "hub" to translate RGB ↔ CMYK.

Analogy — RGB is like mixing colored flashlights (more = brighter, all on = white). CMYK is like stacking colored filters or paint (more = darker, all on = muddy near-black). Lab is the dictionary both languages translate through.

The Print Color Pipeline (memorize this flow)

```
RGB design (screen, Fabric.js studio)
  |
  [ASSIGN source profile]  e.g. sRGB
  |
  [CONVERT via ICC profiles + Rendering Intent]
  |          (translates through Lab hub)
  v
CMYK separations + [OUTPUT INTENT embedded]
  |          (= which press/paper this is for)
  v
Preflight -> PDF/X -> RIP/screening -> Plates/Ink
```

- **ICC profile** = a file describing how one device renders color. **Source** profile = where the color came from (sRGB). **Destination** profile = where it's going (e.g. GRACoL, FOGRA39).
- **Rendering intent** = the rule for squeezing out-of-range colors into the smaller destination.
- **Output intent** = the destination profile *embedded in the PDF* so the printer knows your intended press/paper.

PF-360 gap — The platform has **no RGB → CMYK conversion** and **no preflight**. Files exported from the Fabric.js studio are RGB; the PDF service ([pdf-service](#) , sharp + PDFKit, port 4000) builds RGB PDFs. Treat every output as "RGB, unverified" until a conversion/preflight step exists. Warn store owners that screen colors will shift on press.

Key Numbers (the ones you'll be asked about)

Rule	Value	Why
Print resolution	300 PPI at final size	Matches human eye at reading distance; below ~200 PPI looks soft
Large-format / banners	100–150 PPI	Viewed from far away — less detail needed
Bleed	~3 mm (0.125 in) each edge	Cutting drifts; bleed prevents white slivers at the trim
Safe zone (margin)	~3 mm (0.125 in) <i>inside</i> trim	Keeps text/logos away from the cut
Total Area Coverage (TAC)	240–340% max	Sum of C+M+Y+K%; too much ink = smearing, won't dry. Coated ~300–340%, newsprint ~240%
Rich black (deep black)	e.g. C40 M30 Y30 K100	Plain K100 looks grey on big areas; rich black is denser
Registration black	Avoid for text/fills	100% of <i>all four</i> inks (400% TAC) — over ink limit, smears. It's only for crop marks
LPI → PPI	PPI ≈ 1.5–2× LPI	LPI = halftone screen frequency. 150 LPI press → supply 225–300 PPI

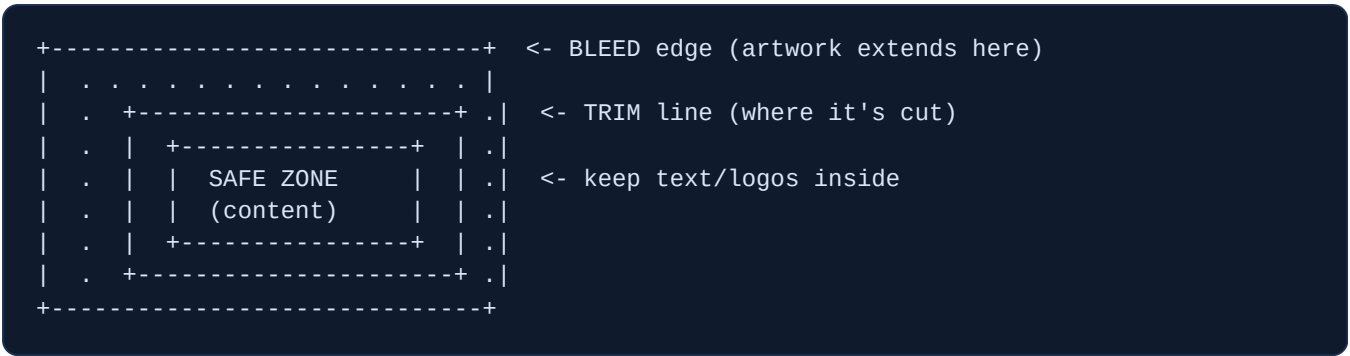
PPI vs DPI

PPI (pixels per inch) describes the image file. **DPI** (dots per inch) describes the printer's ink dots. People say "DPI" loosely but you control *PPI* in your file.

LPI

Lines per inch — how fine the halftone dot grid is. Higher = sharper but needs better paper.

Bleed / Trim / Safe — the three lines



- **Bleed:** background art runs PAST the trim, so a slightly-off cut still has color.
- **Trim:** the finished size — where the blade aims.
- **Safe:** the "don't put anything important here" buffer inside trim.

Rendering Intents — quick pick

Intent	Use for	What it does
Perceptual	Photos	Shifts the whole color range to keep relationships smooth (no banding). Default for images.
Relative Colorimetric	Logos, most print	Keeps in-range colors exact; clips out-of-range. Most common print choice.
Saturation	Charts, graphics	Maximizes vividness over accuracy.
Absolute Colorimetric	Proofing/brand	Simulates the target paper's white exactly (for proofs).

PDF/X-1a vs PDF/X-4 — when to use which

	PDF/X-1a	PDF/X-4
Color	CMYK + spot only (no RGB)	CMYK, spot, <i>and</i> RGB+ICC (color-managed)
Transparency	Must be flattened	Live transparency kept
Layers	No	Yes
Best for	Old/strict workflows, total control	Modern workflow, default choice today

Both require **fonts embedded** and an **output intent**. X-1a is the safe lowest-common-denominator; X-4 is the modern preferred standard.

Fonts, Raster vs Vector, Transparency

- **Fonts:** always **embed + subset** (include only used glyphs) OR **convert to outlines** (turn letters into vector shapes). Never ship a PDF that relies on the printer having your font — it'll substitute and reflow.
- **Vector** (paths/math: logos, type, line art) = scales infinitely, stays crisp. **Raster** (pixels: photos) = fixed resolution, blurs when enlarged. Pick vector for anything sharp-edged; raster for photographic.

Transparency flattening — Drop shadows, blend modes, and partial opacity are "live" effects. When output requires flattening (PDF/X-1a, some RIPs), they get baked into pixels — this can cause **color shifts**, **thin white seams** ("stitching"), and text turning into images. Flatten at high resolution and proof the result. The Fabric.js studio uses RGB blend modes that may not survive a CMYK flatten cleanly.

Preflight Checklist (run before "send to print")

- ☐ Resolution ≥ 300 PPI at final size (no upscaled images)
- ☐ Bleed present (~3 mm) and content inside safe zone
- ☐ Color mode = CMYK (or color-managed RGB for X-4); no stray RGB if X-1a
- ☐ TAC under the press limit ($\leq 300\%$ coated; check rich/registration black)
- ☐ Black text is K100 only (not 4-color, which causes mis-registration fuzz)
- ☐ All fonts embedded/subsetted or outlined
- ☐ Spot/Pantone colors named correctly or converted intentionally
- ☐ Overprint set deliberately (no accidental "overprint white" = invisible)
- ☐ Transparency flattening proofed; correct PDF/X profile + output intent
- ☐ Trim size and page count correct

Spot color & overprint one-liners

Spot color

A pre-mixed named ink (e.g. Pantone 185) run on its own plate — exact, consistent brand colors, used for metallics/fluorescents CMYK can't make.

Knockout

Default: top object cuts a hole in what's behind, so inks don't mix.

Overprint

Top object prints *on top of* the background (inks combine). Used for trapping; dangerous if set on white (it vanishes).

Trapping

Tiny overlap between adjacent colors so a slightly-misaligned press doesn't leave a white gap.

Top 8 Common Mistakes

1. **RGB into print** — bright colors go dull/wrong. Convert and proof. (Live PF-360 gap.)
2. **Low resolution** — 72 PPI web images placed at full page = blurry. Need 300 PPI at final size.
3. **No bleed** — white slivers appear at the cut edge.
4. **Text in the safe zone / at the edge** — gets trimmed off.
5. **Too much ink (TAC over limit)** — smears, offsets, won't dry; registration black on fills.
6. **Fonts not embedded** — printer substitutes a font, layout reflows.
7. **Accidental overprint / wrong knockout** — white text disappears, colors mix unexpectedly.
8. **Unflattened/mis-flattened transparency** — color shifts and white stitching seams.

Key takeaways —

- Color travels **RGB** → **(ICC + rendering intent)** → **CMYK + output intent**, with **Lab** as the translation hub.
- Burn in the numbers: **300 PPI**, **3 mm bleed**, **TAC 240–340%**, **PPI $\approx$ 1.5–2 $\times$ LPI**, and **never use registration black for fills/text**.
- A file is print-ready only after **preflight**: resolution, bleed/safe, CMYK, ink limit, K100 text, embedded fonts, intentional spot/overprint, proofed flattening, correct PDF/X.
- Prefer **PDF/X-4** (modern, keeps transparency & color management); use **X-1a** for strict legacy workflows.
- Remember PF-360's real gaps: **no RGB → CMYK, no preflight, weak order-to-production** — set expectations and treat studio/PDF-service output as unverified RGB.